UPCT · GRADO EN INGENIERÍA BIOMÉDICA · 3.º CURSO

Última actualización: 8 de noviembre de 2026

Electrónica Digital

Guía para aprobar

14	11	18	225
temas (T0–T13)	problemas de examen resueltos	problemas de hojas resueltos	comprobaciones por código

PORTADA

Empieza aquí

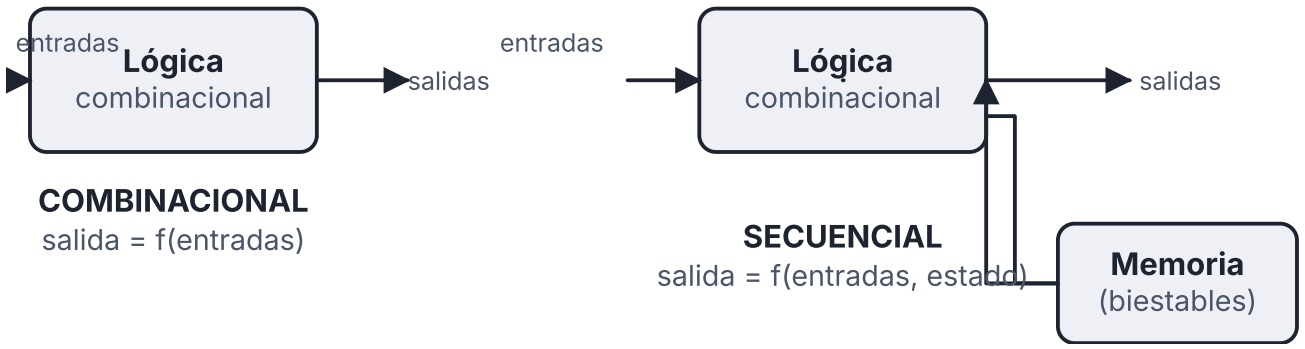
Qué es la asignatura, lo que necesitas saber antes de empezar y cómo usar esta guía.

Qué vas a aprender

Electrónica Digital es una asignatura obligatoria de **3.º de Ingeniería Biomédica** (4,5 ECTS, primer cuatrimestre). Aprendes a **diseñar circuitos que trabajan con unos y ceros**: desde una puerta que decide si se abre un toldo hasta una máquina que reconoce la combinación de una caja fuerte.

Todo lo que se construye en electrónica digital se reduce a dos tipos de circuitos:

- **Combinacionales**: la salida depende *solo* de las entradas de ahora. No tienen memoria. Ejemplo: un sumador.
- **Secuenciales**: la salida depende de las entradas de ahora *y de lo que pasó antes*. Tienen memoria. Ejemplo: un contador.



Todo circuito digital es una de estas dos cosas, o una mezcla: lógica combinacional sola, o lógica combinacional más memoria (biestables) realimentada.

El temario oficial se reparte en cuatro unidades:

Unidad	Temas	Qué trabajas
UD1 · Introducción	0 a 3	Numeración, códigos, álgebra de Boole
UD2 · Combinacional	4 y 5	Familias lógicas, sumadores, multiplexores, decodificadores, comparadores
UD3 · Secuencial	6 a 11	Biestables, memorias, contadores, registros, autómatas
UD4 · Lógica programable	12 y 13	FPGA y lenguaje VHDL

La evaluación se reparte en **80 % examen escrito** y **20 % prácticas**, y hay que sacar **al menos un 4,0 en cada parte** para aprobar. Todos los detalles están en El examen y cómo estudiar.

Repaso desde cero

No hace falta saber electrónica. Hace falta tener claras cinco ideas.

1. Un bit es un interruptor

Un **bit** vale 0 o 1. En un circuito real, 0 y 1 son dos rangos de tensión (por ejemplo, cerca de 0 V y cerca de 5 V). Con n bits puedes escribir 2^n combinaciones distintas.

$$N = 2^n \qquad \Longleftrightarrow \qquad n = \lceil \log_2 N \rceil$$

Con 3 bits hay 8 combinaciones; con 4 bits, 16. Si necesitas distinguir 10 cosas (los dígitos del 0 al 9), hacen falta 4 bits, porque con 3 solo llegas a 8.

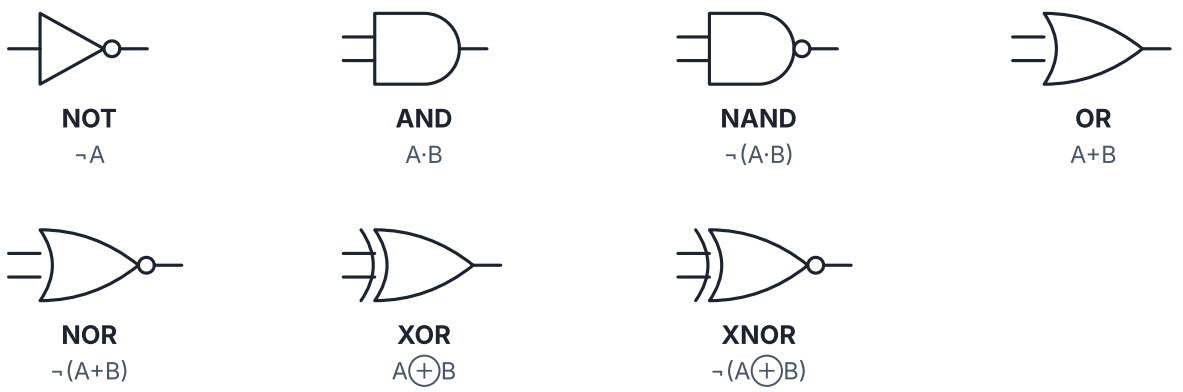
2. Contar en binario es contar con dos dedos

En decimal cada posición vale 10 veces la anterior. En binario, 2 veces.

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

3. Las tres puertas básicas

- **AND** ($A \cdot B$): vale 1 solo si *todas* las entradas valen 1.
- **OR** ($A + B$): vale 1 si *alguna* entrada vale 1.
- **NOT** ($\overline{A}$): invierte.



Las siete puertas que vas a usar. El círculo pequeño a la salida significa «negado».

4. Tabla de verdad

Una tabla de verdad es la lista de **todas** las combinaciones de entradas con la salida que quieres para cada una. Es el punto de partida de casi cualquier diseño combinacional: si no sabes qué circuito hacer, escribe la tabla.

5. El reloj

En los circuitos secuenciales hay una señal de **reloj** que sube y baja siempre igual. Los biestables solo miran sus entradas en un instante: el **flanco** (el momento en que el reloj sube, o baja). Entre flancos, lo que pasa en las entradas no cambia la memoria.

Cómo usar esta guía

1. **Empieza por la teoría de cada tema.** Está escrita con la intuición primero y las fórmulas después. Cada tema acaba con un recuadro de *Qué suele fallar* y otro de *Fórmulas clave*.
2. **Juega con los simuladores.** Si no entiendes algo, muévelo: cambia un bit, una secuencia, una tensión. Verás el resultado al momento.
3. **Haz los ejercicios sin mirar.** Cada ejercicio tiene el enunciado, un recuadro *Antes de empezar* y la resolución en pasos plegados. Abre un paso solo cuando estés atascado.
4. **Practica con exámenes reales.** En Exámenes resueltos tienes los que han salido, con búsqueda por tema.
5. **Cierra el círculo** con la autoevaluación, los simulacros y el plan de estudio.

> **⚠️ Cómo se ha comprobado todo**
>
> Cada resultado numérico, cada ecuación minimizada y cada secuencia de estados de esta guía se ha **comprobado con un programa** que simula el circuito entrada por entrada. Los ejercicios con la etiqueta «✔ verificado» pasan esa comprobación. Cuando una solución de referencia tenía un fallo, no se ha copiado: hay una nota **Ojo** que dice dónde está y por qué.

> **📘 Notación que usa la guía**
>
> - Decimales con coma en el texto ($0{,}5$) y con punto en el código (`0.5`).
> - $\overline{A}$ es «A negada». $A \cdot B$ se escribe a veces AB. $\oplus$ es la XOR.
> - Q_n es el estado actual y Q_{n+1} el estado siguiente.
> - Los números en base 2 llevan subíndice (1011_2); sin subíndice, son decimales.

EMPIEZA AQUÍ

El examen y cómo estudiar

Cómo se evalúa, qué cae más según los exámenes reales y una estrategia para sacar nota.

Cómo se evalúa la asignatura

Parte	Peso	Qué es
Exámenes escritos	**80 %**	Cuestiones conceptuales, análisis y síntesis de circuitos digitales, interpretación de diseños y resolución de casos prácticos.
Prácticas	**20 %**	Memorias e informes, y defensa del trabajo de laboratorio (individual o en grupo).

Reglas que conviene tener presentes:

- Para aprobar hace falta **un 4,0 como mínimo tanto en la parte teórica como en la práctica**.
- La asistencia a las prácticas es obligatoria: hace falta al menos un **80 %** para que cuente como hecha.
- Si ya superaste una actividad en la evaluación continua y quieres presentarte a esa misma actividad en la evaluación final, **debes renunciar a la nota que tenías**.
- Las prácticas 4 a 7 permiten usar inteligencia artificial como ayuda, pero tienes que **entender, revisar y justificar** el código. En la defensa te preguntarán por él.
- También hay pruebas cortas en clase, individuales o por parejas (evaluación formativa).

> ⚠️ **Fechas y ponderaciones**
>
> Esta guía no inventa fechas ni notas. Mira en tu calendario académico cuándo es la convocatoria. Lo que sí sabes: el examen escrito vale el 80 % y debes sacar un 4,0 como mínimo.

Cómo son los exámenes de otros años

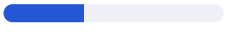
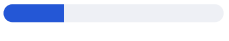
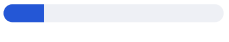
Los exámenes reales que han servido para preparar esta guía son de la **misma asignatura en la titulación de Ingeniería Electrónica Industrial y Automática** de la misma escuela. El contenido es esencialmente el mismo. La organización exacta del examen puede variar de un curso a otro, así que comprueba siempre la de tu convocatoria.

Lo que se repite en todos ellos:

Dato	Lo que dicen los exámenes
Número de problemas	4
Valor de cada problema	2,5 puntos (examen de febrero de 2023)
Tiempo	2 horas (examen de diciembre de 2024)
Cómo se corrige	«Se tendrá en cuenta la simplicidad de la solución propuesta, así como la claridad de realización del mismo» (febrero de 2023 y febrero de 2025)
Cronogramas	«No se contabilizarán los ejercicios que sólo dibujen las formas de onda sin explicar nada» (diciembre de 2024)
Notas de aprobado (febrero de 2023)	Examen escrito ≥ 5 y prácticas ≥ 1 sobre 2; nota mínima de la asignatura 6 puntos. En tu titulación rige el 80 %/20 % de arriba.

La última fila es una norma de *esa* convocatoria. Se incluye solo para que veas qué tipo de normas aparecen.

Qué cae más

Tipo de problema	Problemas	Peso	Aparece en
Bloques MSI y displays	6 de 11		3 de 3 exámenes
Autómatas / síntesis secuencial	5 de 11		3 de 3 exámenes
Aritmética y códigos (BCD, C2)	4 de 11		3 de 3 exámenes
Combinacional (puertas, Karnaugh)	3 de 11		3 de 3 exámenes
Contadores y biestables	2 de 11		1 de 3 exámenes

Recuento propio sobre los 11 problemas de los 3 exámenes resueltos aquí (un problema puede tener varias etiquetas). Son pocos exámenes: úsalo como orientación, no como regla.

Mira la forma que tiene un examen típico. Casi siempre hay:

1. **Un problema de diseño combinacional con bloques estándar** (sumadores, comparadores, multiplexores, displays): «haz la media de…», «calcula $A \cdot B + C$ y muéstralo en displays», «un recinto con dos puertas».
2. **Un problema pequeño de lógica o de contadores**: simplificar una función con Karnaugh y pasarla a NAND, o analizar un circuito dado y dibujar su cronograma.

3. **Un problema de síntesis de un contador** con biestables JK y puertas, con una entrada de control (sube/baja).
4. **Un autómata**: la caja fuerte, la noria, la máquina expendedora... Diagrama de estados, tabla de transiciones, ecuaciones y circuito.

El problema 4 suele ser el más largo. Reserva tiempo.

Estrategia para el examen

> 💡 **Reparto de las 2 horas**
>
> - **5 min**: lee los cuatro enunciados completos y marca qué te sabes mejor.
> - **25-30 min por problema**, empezando por el que tengas más claro.
> - **10 min al final** para revisar: ¿cada problema tiene diagrama/tabla, ecuaciones y circuito?
>
> Un problema empezado con buena idea y un esquema claro puntúa más que uno en blanco.

Reglas prácticas que se repiten en las correcciones:

- **Escribe los pasos.** Un cronograma sin explicar cómo se obtuvo no puntúa. Una tabla de verdad sin la ecuación tampoco.
- **Define tus entradas, salidas y estados** antes de dibujar. Escribe qué significa cada estado con palabras.
- **Si el enunciado es ambiguo, di qué supones** y sigue. Es lo que se espera.
- **Simplifica**: la solución más sencilla puntúa mejor. Usa los indiferentes (×) en Karnaugh.
- **Comprueba con un caso**: antes de entregar, mete una entrada concreta en tu circuito y mira que sale lo que debe.

Tu seguimiento

Marca lo que vas haciendo. Se guarda en este navegador.

> 📈 **Tu progreso** (se guarda en este navegador)
>
> 0 de 47 tareas (0 %)

Tema 0 · Introducción a la asignatura

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 1 · Introducción a los sistemas digitales

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 2 · Sistemas de numeración, operaciones y códigos

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 3 · Álgebra de Boole y funciones lógicas

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 4 · Familias lógicas

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 5 · Funciones de la lógica combinacional

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 7 · Memorias y circuitos lógicos programables

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 8 · Contadores

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 9 · Registros de desplazamiento

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 11 · Descripciones funcionales y estructurales

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 12 · Introducción a los dispositivos lógicos programables

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 13 · Introducción al diseño digital con VHDL

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Bloques de examen

- [ ] Resolver 2 exámenes completos en 2 horas (sin mirar la teoría)
- [ ] Rehacer los problemas que fallaste
- [ ] Hojas de problemas: combinacional
- [ ] Hojas de problemas: autómatas
- [ ] Repasar las prácticas y el código VHDL

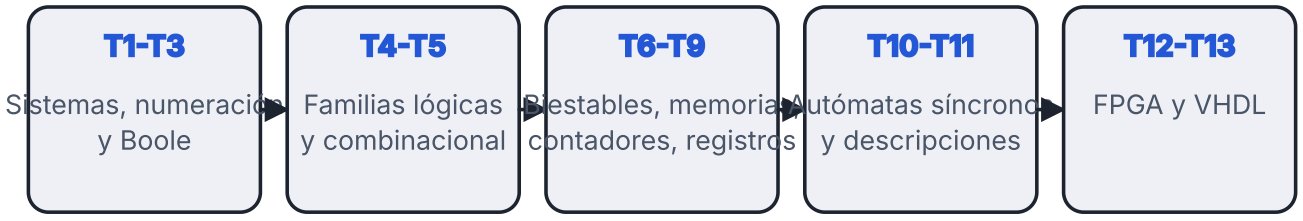
TEORÍA POR TEMAS

Tema 0 · Introducción a la asignatura

Qué se pide, con qué herramientas y cómo encaja cada tema.

El mapa en una página

La asignatura recorre este camino. Cada paso usa lo del anterior.



combinacional → secuencial → lógica programable

De los números al diseño con FPGA. Los temas de la izquierda son la base; los de la derecha son lo que se ve en el laboratorio.

Cuatro resultados de aprendizaje te dicen qué se espera de ti:

- **Explicar** cómo funcionan los sistemas digitales.
- **Analizar y sintetizar** circuitos digitales. *Analizar* es entender uno que ya existe. *Sintetizar* es diseñar uno a partir de una descripción.
- **Usar** instrumentación básica y software de simulación.
- **Programar** una FPGA con esquemáticos y con VHDL.

Dos verbos que lo resumen todo

💡 Analizar frente a sintetizar

- **Analizar**: te dan el circuito y te piden la función, la tabla o el cronograma. Camino: circuito → ecuaciones → tabla → comportamiento.
- **Sintetizar**: te dan una descripción en palabras y te piden el circuito. Camino: palabras → tabla o diagrama de estados → ecuaciones → circuito.

Son el mismo camino recorrido en sentidos opuestos. Si dominas uno, el otro es el mismo mapa al revés.

Herramientas del laboratorio

Se trabaja con placas de desarrollo **Basys** con una **FPGA de Xilinx** y el entorno **Xilinx ISE**. Una FPGA es un chip que puedes «reconfigurar» para que se comporte como el circuito digital que has diseñado. Se describe el circuito de tres maneras equivalentes:

1. **Esquemático**: dibujas las puertas y los bloques.
2. **Diagrama de estados**: dibujas los estados y las transiciones.
3. **VHDL**: lo escribes como texto.

El proyecto final es un sistema de monitorización de una **incubadora neonatal** usada como entorno de simulación. Lo verás en Prácticas.

> **🚧 Qué suele fallar**
>
> - Empezar a dibujar circuitos antes de escribir la **tabla de verdad o el diagrama de estados**. Todo diseño bueno empieza ahí.
> - Confundir *analizar* con *sintetizar* y calcular al revés.
> - No decir las suposiciones cuando el enunciado es ambiguo.

TEORÍA POR TEMAS

Tema 1 · Introducción a los sistemas digitales

Qué significa «digital», por qué se usa y cómo se representan 0 y 1.

Analógico frente a digital

Una señal **analógica** puede tomar cualquier valor entre dos extremos (la tensión de un micrófono). Una señal **digital** solo toma un conjunto finito de valores, casi siempre dos: 0 y 1.

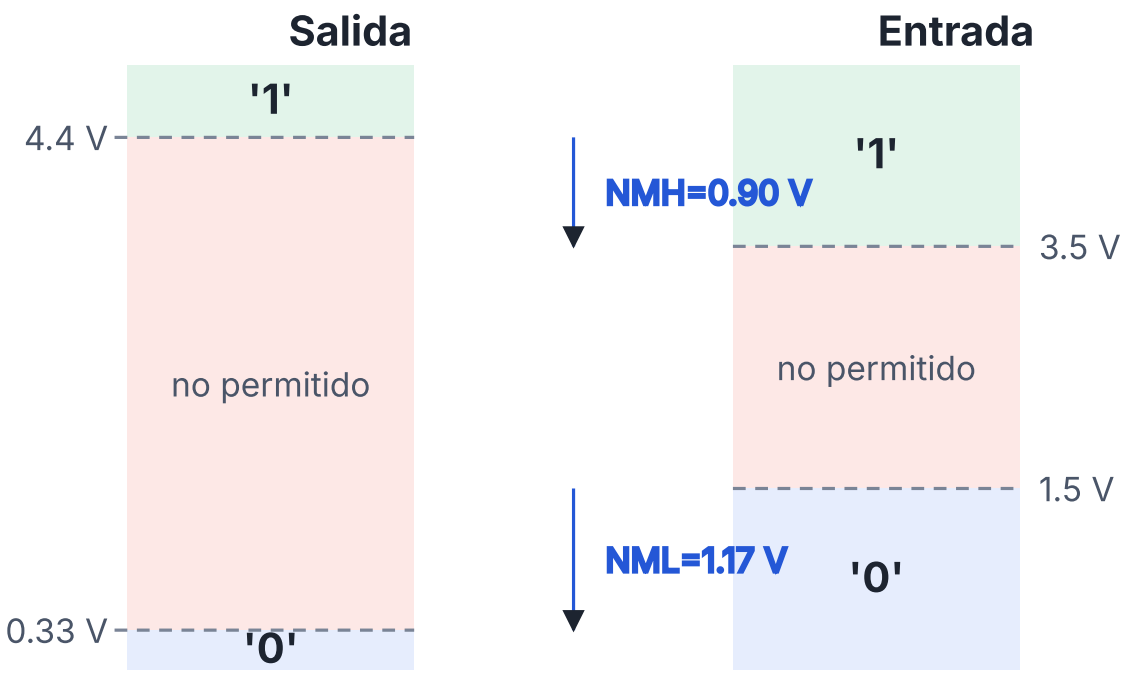
¿Por qué se prefiere lo digital?

- **Aguanta el ruido.** Si un 1 es «unos 5 V» y le llegan 4,6 V, sigue siendo un 1. El circuito siguiente lo limpia.
- **Se puede copiar sin pérdida.** Un fichero se duplica bit a bit.
- **Es programable.** El mismo hardware hace cosas distintas según el código.

El precio: hay que convertir lo real (temperatura, presión, señal ECG) en números. Eso lo hacen los convertidores A/D.

Representar 0 y 1 con tensiones

Un 1 y un 0 son **rangos** de tensión, no valores exactos. Entre ellos hay una zona prohibida. Lo verás en detalle en el Tema 4.



Niveles lógicos de una familia CMOS de 5 V

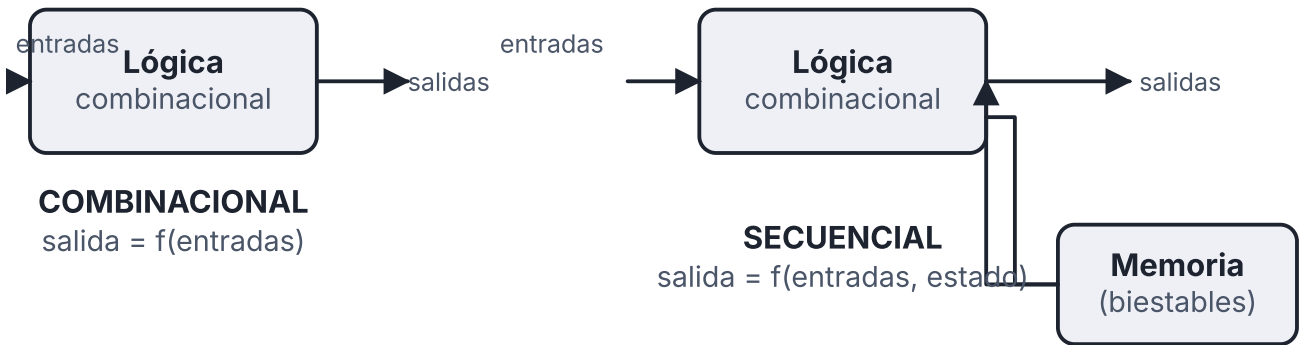
A la salida, un 1 es cualquier tensión por encima de 4,4 V. A la entrada, el circuito acepta como 1 todo lo que supere 3,5 V. La diferencia (0,9 V) es el margen de ruido.

Lógica positiva y negativa

En **lógica positiva** el nivel alto significa 1. En **lógica negativa**, el nivel alto significa 0. Casi todo lo que verás es lógica positiva. Pero en las placas Basys los segmentos de los displays y algunos pulsadores son *activos a nivel bajo*: se encienden con un 0.

> 📘 **Señal activa a nivel alto o bajo**
>
> Una señal está **activa a nivel alto** si hace su trabajo cuando vale 1, y **activa a nivel bajo** si lo hace cuando vale 0. En los esquemas, la activa a nivel bajo lleva un círculo en su pin o una barra encima ($\overline{CLR}$, $\overline{LD}$).

Tipos de sistemas digitales



Un **sistema combinacional** no recuerda nada. Un **sistema secuencial** tiene memoria y un reloj. Esta división organiza todo el temario:

	Combinacional	Secuencial
Salida depende de	Entradas actuales	Entradas actuales y pasado
Elemento básico	Puerta lógica	Biestable
Se describe con	Tabla de verdad, ecuación	Diagrama de estados, tabla de transiciones
Ejemplos	Sumador, multiplexor, decodificador	Contador, registro, autómata

🚧 Qué suele fallar

- Pensar que una puerta «recuerda». No: una puerta solo calcula.
- Olvidar que un 0 o un 1 reales son *rangos*. Dos circuitos solo se pueden conectar directamente si sus rangos encajan (Tema 4).

TEORÍA POR TEMAS

Tema 2 · Sistemas de numeración, operaciones y códigos

Cómo escribe números un circuito: binario, complemento a dos, BCD, Gray y detección de errores.

💡 Desde cero

Un circuito solo guarda ceros y unos. Para guardar el número 156, el 37 negativo o la letra «A», hace falta un **acuerdo** sobre qué significa cada combinación de bits. Ese acuerdo es un **código**. Este tema recoge los códigos que vas a usar siempre: binario natural, hexadecimal, complemento a dos, BCD, Gray y paridad.

Binario, octal y hexadecimal

En decimal cada posición vale una potencia de 10. En binario, una potencia de 2:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}$$

Del decimal al binario: divide entre 2 una y otra vez y apunta los restos. Lee los restos de abajo arriba.

Hexadecimal: agrupa los bits de 4 en 4 desde la derecha. Cada grupo es una cifra (0-9, A=10, B=11, … F=15). Es la forma corta de escribir binario. **Octal:** agrupa de 3 en 3.

Ejemplo T2 **De decimal a binario, hexadecimal y octal** ✔ verificado

Pasa 156_{10} a binario, hexadecimal y octal.

🧭 Antes de empezar: qué piden y qué teoría usas

Usas la división sucesiva entre 2 y la agrupación de bits en bloques de 4 (hex) y de 3 (octal).

1 Divisiones entre 2

$156 \to 78 \to 39 \to 19 \to 9 \to 4 \to 2 \to 1 \to 0$ con restos $0, 0, 1, 1, 1, 0, 0, 1$. Leídos de abajo arriba: 10011100_2.

2 Agrupa de 4 en 4

$1001\ 1100$ = 9 C. Es decir, $9C_{16}$.

3 **Agrupa de 3 en 3**

$010\ 011\ 100$ = 2 3 4. Es decir, 234_8. (Se rellena con un 0 a la izquierda.)

✅ **Resultado final**

$156_{10} = 10011100_2 = 9C_{16} = 234_8$. Comprobación: $9 \cdot 16 + 12 = 156$.

Parte fraccionaria: multiplica por 2 y apunta la parte entera cada vez. Ejemplo: $0{,}6875 \to 1{,}375 \to 0{,}75 \to 1{,}5 \to 1{,}0$ da $0{,}1011_2$.

⚠️ **Muchos decimales no caben en binario**

$0{,}1_{10}$ en binario es periódico, igual que $1/3$ en decimal. Por eso un circuito nunca guarda $0{,}1$ exacto: guarda una aproximación.

Números con signo: complemento a dos

Con n bits hay 2^n combinaciones. Para repartirlas entre positivos y negativos se usa el **complemento a dos (C2)**. El bit más significativo (MSB) es el **signo**: 0 positivo, 1 negativo.

Rango con n bits:

$$-2^{n-1} \le N \le 2^{n-1} - 1$$

Con 8 bits: de -128 a $+127$. Hay un negativo más que positivos.

Cambiar de signo: invierte todos los bits y suma 1. Ejemplo con $37 = 00100101$: invierto, 11011010; sumo 1, $11011011 = -37$.

Intuición: en C2 el peso del MSB es -2^{n-1} (negativo). Por eso $11011011 = -128 + 64 + 16 + 8 + 2 + 1 = -37$.

Ejemplo T2 **Sumar en complemento a dos y detectar desbordamiento**

✔ verificado

Con 8 bits en C2: (a) escribe -37; (b) calcula $45 + (-20)$; (c) calcula $100 + 50$ y di qué pasa.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: C2 = invertir y sumar 1; la suma de C2 es la suma binaria normal; hay **desbordamiento** si el resultado no cabe en el rango $[-128, 127]$.

1 **(a)** -37

$37 = 00100101$. Invierto: 11011010. Sumo 1: 11011011.

2 **(b)** $45 + (-20)$

$45 = 00101101$, $-20 = 11101100$. Suma: $1\,00011001$. Se descarta el acarreo que sale del byte: queda $00011001 = 25$. Correcto.

3 **(c)** $100 + 50$

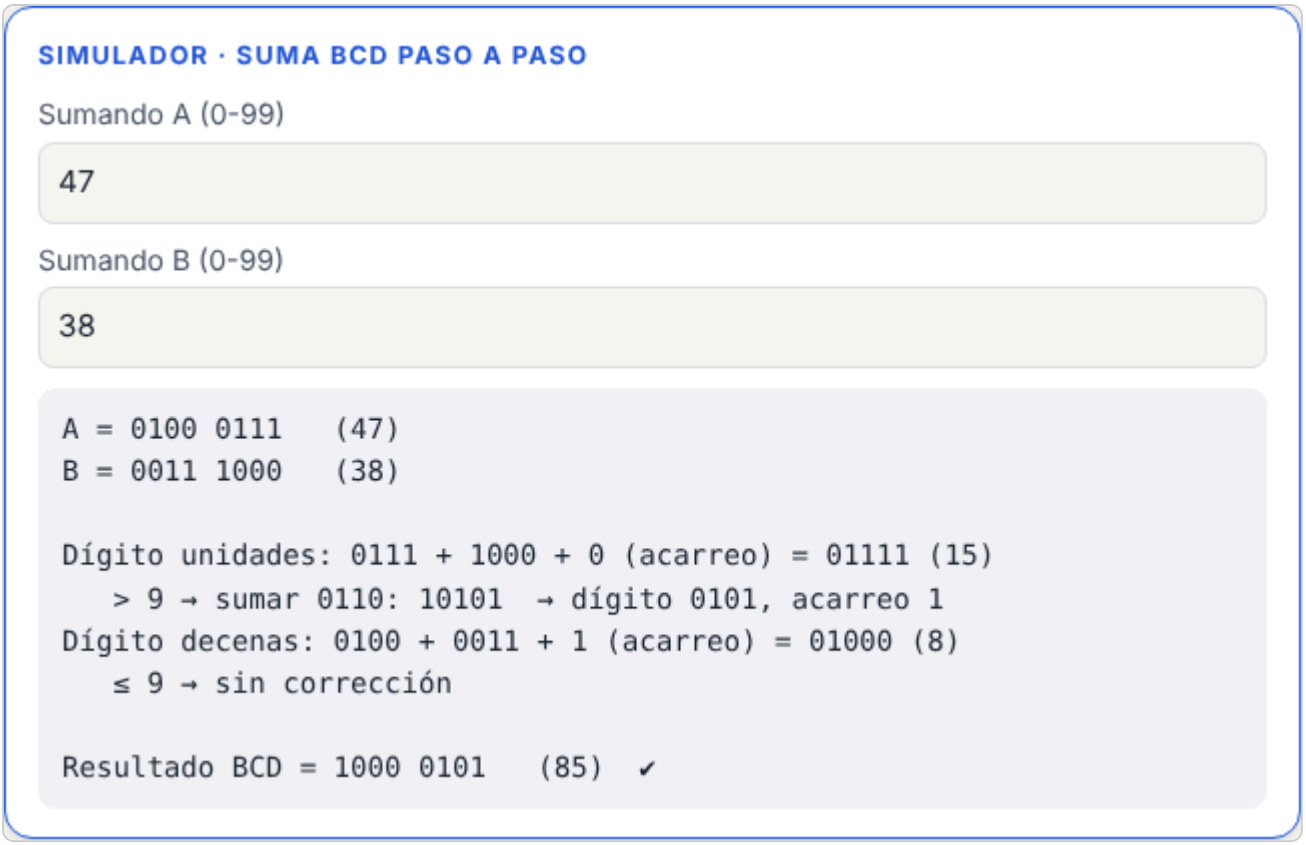
$100 = 01100100$, $50 = 00110010$. Suma: 10010110. El MSB vale 1, así que se lee como negativo: -106. Es **incorrecto**: $150 > 127$. Hay desbordamiento.

✅ Resultado final

(a) 11011011. (b) $00011001 = 25$. (c) 10010110 (se leería -106) y hay **desbordamiento**.

∑ Cómo detectar el desbordamiento en C2

- Dos positivos que suman un negativo, o dos negativos que suman un positivo: hay desbordamiento.
- Equivale a: el acarreo que entra al bit de signo es distinto del acarreo que sale de él.
- Un positivo más un negativo **nunca** desborda.



Captura del simulador interactivo. Pruébalo en la versión web.

BCD: decimal en cuatro bits

En **BCD** (decimal codificado en binario) cada cifra decimal se escribe con 4 bits, y solo se usan las combinaciones 0000 a 1001. Así $47_{10} = 0100\ 0111_{BCD}$. Es cómodo para mostrar números en displays, pero ocupa más bits y sumar es más difícil.

Suma en BCD: suma cada cifra en binario. Si el resultado de una cifra supera 9 (o hay acarreo), **suma 6** para saltarte las 6 combinaciones prohibidas (1010 a 1111).

Ejemplo T2 — Suma en BCD: 38 + 45

✔ verificado

Suma $38 + 45$ en BCD.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: BCD, y la corrección «+6» cuando una cifra pasa de 9.

1 Escribe los sumandos

$38 = 0011\ 1000$ y $45 = 0100\ 0101$.

2 **Suma binaria**

$0011\,1000 + 0100\,0101 = 0111\,1101$.

3 **Corrige la cifra de abajo**

Las cifras bajas suman $1101 = 13 > 9$: es inválido. Suma 0110: $0111\,1101 + 0110 = 1000\,0011$.

✅ **Resultado final**

$1000\ 0011_{BCD} = 83$. Comprobación: $38 + 45 = 83$.

Código Gray y detección de errores

Gray: una secuencia de códigos donde de uno al siguiente **cambia un solo bit**. Para 3 bits: $000, 001, 011, 010, 110, 111, 101, 100$. Se usa en codificadores de posición: si dos bits cambiaran a la vez y uno llegara antes, leerías una posición falsa. Con Gray el error máximo es una posición.

Paso de binario a Gray: $g_i = b_i \oplus b_{i+1}$ (el MSB se copia). Es decir, $g = b \oplus (b \gg 1)$.

Paridad: se añade un bit para que el número total de unos sea par (paridad par) o impar. Detecta **un** error, pero no sabe dónde está. Para 1011001 hay cuatro unos: el bit de paridad par es 0.

Hamming (7,4): añade 3 bits de paridad a 4 de datos de forma que se **corrige** un error. Los bits p_1, p_2, p_3 están en las posiciones 1, 2 y 4. Para los datos 1011 la palabra es 0110011. Si falla un bit, los tres chequeos de paridad forman un número binario que **es la posición del bit erróneo**.

🔭 **Más a fondo: distancia de Hamming**

La **distancia** entre dos palabras es el número de bits en que difieren. Un código con distancia mínima d detecta $d - 1$ errores y corrige $\lfloor (d-1)/2 \rfloor$. La paridad simple tiene $d = 2$ (detecta 1, corrige 0). Hamming (7,4) tiene $d = 3$ (corrige 1). Se ha comprobado por código que corrige cualquiera de los 7 posibles errores de un bit en las 16 palabras.

🚧 Qué suele fallar

- Olvidar que en C2 el rango **no es simétrico**: con 8 bits el mínimo es -128, y no tiene opuesto positivo.
- Descartar el acarreo final y creer que ya está: hay que mirar el **desbordamiento** aparte.
- En BCD, sumar 6 a todas las cifras en vez de solo a las que pasan de 9.
- Mezclar «mayor que 9» con «hay acarreo»: ambas condiciones obligan a corregir.

∑ Fórmulas clave

- N combinaciones $\Rightarrow n = \lceil \log_2 N \rceil$ bits.
- C2 de n bits: $-2^{n-1} \leq N \leq 2^{n-1} - 1$.
- $-N = \overline{N} + 1$ (invertir y sumar 1).
- Gray: $g = b \oplus (b \gg 1)$.
- Suma BCD: si cifra > 9, sumar 0110.

¿Rango de un número en C2 de n bits?

-2^{n-1} a $2^{n-1} - 1$.

¿Cómo se niega un número en C2?

Invertir todos los bits y sumar 1.

¿Cuándo hay desbordamiento en C2?

Cuando sumas dos números del mismo signo y el resultado tiene signo contrario.

¿Corrección en la suma BCD?

Si la cifra pasa de 9 (o hay acarreo), sumar 0110.

¿Qué caracteriza al código Gray?

Entre dos códigos consecutivos cambia un solo bit.

BCD

Decimal codificado en binario: cada cifra decimal en 4 bits.

Complemento a dos

Representación de enteros con signo: el MSB pesa -2^{n-1}.

Desbordamiento

El resultado de una operación no cabe en el número de bits disponible.

Código Gray

Código en el que dos palabras consecutivas difieren en un solo bit.

Distancia de Hamming

Número de bits en que difieren dos palabras de un código.

Paridad

Bit extra que hace par (o impar) el número de unos de una palabra.

¿Cuál es el rango de un entero en complemento a dos con 6 bits?

A de -31 a $+32$

B de -32 a $+32$

C de 0 a 63

D de -32 a $+31$

Con $n = 6$: $-2^5 = -32$ hasta $2^5 - 1 = 31$.

En C2 de 8 bits sumas $01100100 + 00110010$. ¿Qué ocurre?

A Hay un error de paridad

B Se produce acarreo y el resultado es correcto

C Hay desbordamiento: dos positivos dan un resultado con MSB igual a 1

D No hay problema: 150 cabe en 8 bits

$100 + 50 = 150 > 127$. El resultado 10010110 se lee como -106.

$0100\,0111$ en BCD representa:

A 47

B 0x47 en decimal, es decir, 71

C 71

D 4,7

Cada grupo de 4 bits es una cifra decimal: 4 y 7.

Para 3 bits, ¿qué código de Gray sigue a 011?

A 000

B 100

C 010

D 111

La secuencia es 000, 001, 011, 010, 110, 111, 101, 100.

TEORÍA POR TEMAS

Tema 3 · Álgebra de Boole y funciones lógicas

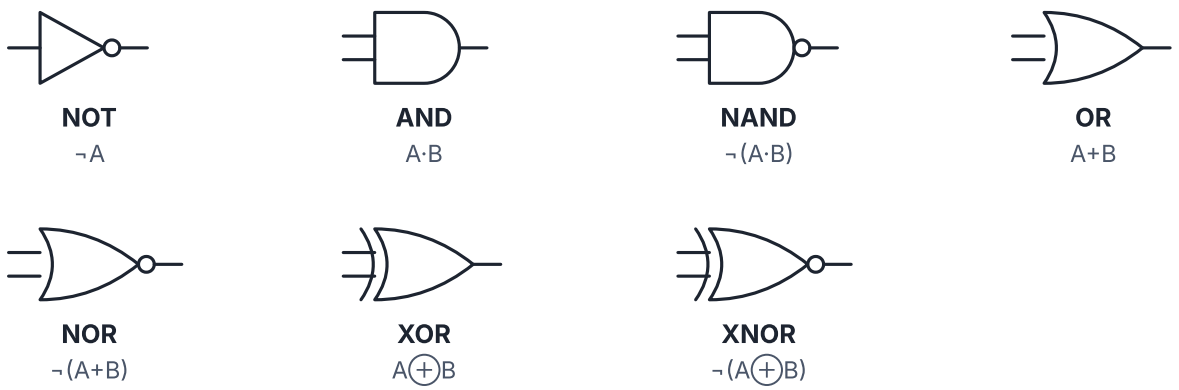
Puertas, tablas de verdad, formas canónicas y simplificación con mapas de Karnaugh.

💡 **Desde cero**

Una **función lógica** es una regla que dice, para cada combinación de entradas (0/1), si la salida es 0 o 1. Se puede escribir de tres formas equivalentes: **tabla de verdad**, **ecuación** y **circuito de puertas**. El álgebra de Boole es el conjunto de reglas para pasar de una a otra y, sobre todo, para **simplificar**: menos puertas es menos coste, menos consumo y menos retardo.

Las operaciones básicas

Operación	Símbolo	Se lee	Vale 1 cuando
NOT	$\overline{A}$	no A	$A = 0$
AND	$A \cdot B$	A y B	las dos entradas valen 1
OR	$A + B$	A o B	al menos una vale 1
XOR	$A \oplus B$	A o exclusivo B	las entradas **son distintas**
NAND	$\overline{A \cdot B}$		no las dos a 1
NOR	$\overline{A + B}$		las dos a 0



Símbolos de las puertas básicas. El círculo a la salida significa «negado».

Las puertas NAND y NOR son universales: con solo una de ellas puedes construir cualquier circuito. Se ha comprobado por código que con NAND se obtienen NOT, AND y OR.

Reglas que debes saber de memoria

Regla	Forma 1	Forma 2
Identidad	$A + 0 = A$	$A \cdot 1 = A$
Nulo	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotencia	$A + A = A$	$A \cdot A = A$
Complemento	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
Absorción	$A + AB = A$	$A(A + B) = A$
Adyacencia	$AB + A\overline{B} = A$	$(A + B)(A + \overline{B}) = A$
Consenso	$AB + \overline{A}C + BC = AB + \overline{A}C$	
De Morgan	$\overline{A + B} = \overline{A} \cdot \overline{B}$	$\overline{A \cdot B} = \overline{A} + \overline{B}$

Todas estas identidades se han comprobado por código con las tablas de verdad completas.

> 💡 **De Morgan en una frase**
>
> Para negar una expresión: **cambia cada AND por OR (y al revés) y niega cada variable**. Por eso una puerta NAND es lo mismo que una OR con las entradas negadas.

Formas canónicas

Un **minitérmino** es un AND que contiene todas las variables (cada una negada o no) y vale 1 en **una sola** fila de la tabla. Un **maxitérmino** es un OR con todas las variables que vale 0 en una sola fila.

- **Suma de minitérminos** (Σm): suma (OR) de los minitérminos de las filas donde $f = 1$.
- **Producto de maxitérminos** (ΠM): producto (AND) de los maxitérminos de las filas donde $f = 0$.

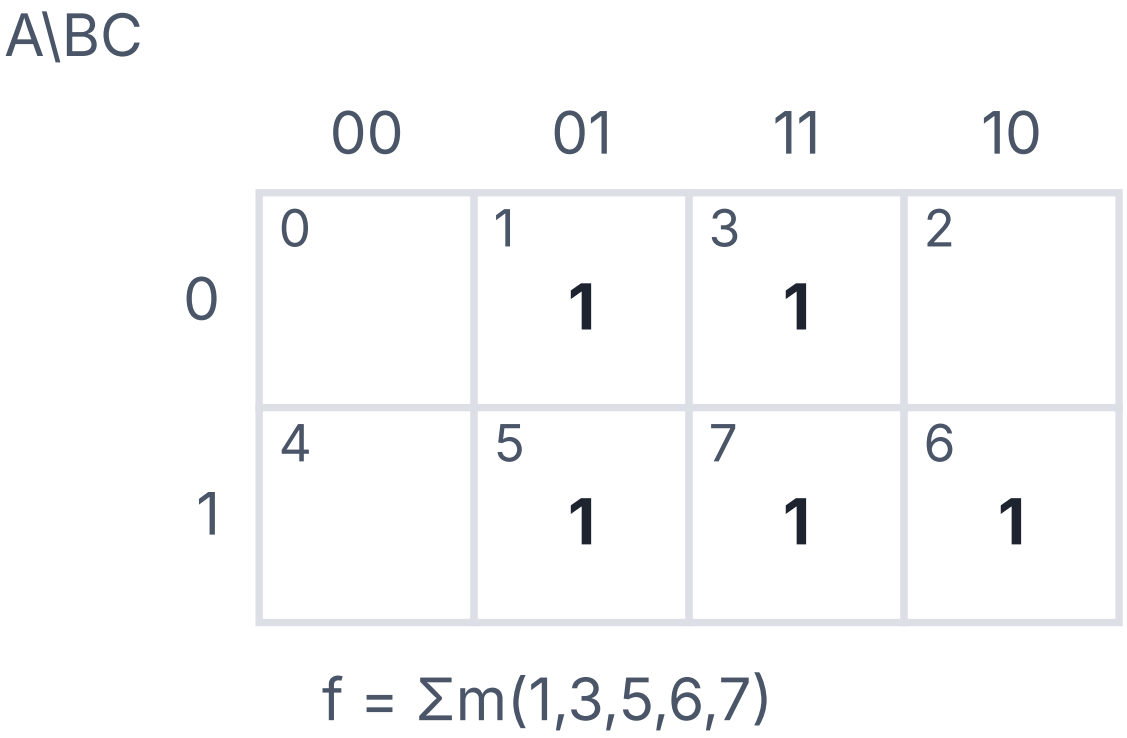
Ejemplo: $f(A, B, C) = \Sigma m(1, 3, 5, 6, 7)$ significa que $f = 1$ en las filas $001, 011, 101, 110, 111$.

Simplificar con mapas de Karnaugh

Un **mapa de Karnaugh** es la tabla de verdad colocada en una rejilla donde las casillas vecinas difieren en **un solo bit** (por eso las columnas van en orden Gray: 00, 01, 11, 10). Dos casillas vecinas con un 1 se pueden juntar y desaparece la variable que cambia.

Reglas para agrupar:

1. Grupos de 1, 2, 4, 8… casillas (potencias de 2), rectangulares.
2. Cuanto **más grandes**, mejor: eliminan más variables.
3. El mapa es **cíclico**: el borde izquierdo es vecino del derecho, y el de arriba del de abajo.
4. Cubre todos los 1 con el mínimo número de grupos. Un 1 puede estar en varios grupos.
5. Las casillas **indiferentes** (X, *don't care*) se usan solo si ayudan a hacer el grupo mayor.



Dos grupos de cuatro y de dos casillas cubren los cinco unos.

Ejemplo T3 **Simplificar $f = \Sigma m(1,3,5,6,7)$** ✔ verificado

Simplifica $f(A,B,C) = \Sigma m(1,3,5,6,7)$ como suma de productos y como producto de sumas.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: mapa de Karnaugh de 3 variables, agrupación en potencias de 2, y la dualidad: para el producto de sumas se agrupan los **ceros** y se niega el resultado.

1 Rellena el mapa

Pon un 1 en las casillas $1, 3, 5, 6, 7$. Los ceros están en $0, 2, 4$.

2 Agrupa los unos

Grupo de 4: m_1, m_3, m_5, m_7 (la columna $C = 1$): queda solo C. El 1 que falta, $m_6 = 110$, se agrupa con m_7: queda AB.

3 Escribe la suma de productos

$f = C + AB$.

4 Producto de sumas

Ahora agrupa los **ceros** (m_0, m_2, m_4). m_0 y m_2 (la fila $A = 0, C = 0$) dan $\overline{A}\,\overline{C}$. m_0 y m_4 ($B = 0, C = 0$) dan $\overline{B}\,\overline{C}$. Así $\overline{f} = \overline{A}\,\overline{C} + \overline{B}\,\overline{C}$.

5 Niega con De Morgan

$f = \overline{\overline{A}\,\overline{C} + \overline{B}\,\overline{C}} = (A + C)(B + C)$. Es la misma función: $(A + C)(B + C) = AB + C$.

✅ **Resultado final**

$f = C + AB = (A + C)(B + C)$.

SIMULADOR · MINIMIZADOR DE FUNCIONES (2-4 VARIABLES)

Nº de variables · clic en una salida: 0 → 1 → × (indiferente)

4

— ejemplos —

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

```
Minterms (1): —
Indiferentes: —

Suma de productos : F = 0
Producto de sumas : F = ()

Puertas: 0 producto(s) + 1 OR  ·  0 literales
NAND-NAND: se obtiene negando dos veces la SOP (De Morgan);
inversores solo para las variables que aparezcan negadas.
```

Captura del simulador interactivo. Pruébalo en la versión web.

🔭 Más a fondo: método de Quine–McCluskey

Cuando hay más de 4 variables el mapa se vuelve incómodo. El algoritmo de **Quine–McCluskey** hace lo mismo con tablas: (1) agrupa minitérminos que difieren en un bit para obtener los **implicantes primos**; (2) elige un conjunto mínimo que cubra todos los minitérminos, empezando por los **implicantes primos esenciales** (los únicos que cubren algún minitérmino). Todas las simplificaciones de esta guía se han obtenido con una implementación propia de este método y se han comprobado contra la tabla de verdad.

🚧 Qué suele fallar

- Hacer grupos de 3, 5 o 6 casillas. Solo valen 1, 2, 4, 8, 16.
- Olvidar que el mapa es cíclico y no ver el grupo de las esquinas.
- Usar una casilla X como si fuera un 1 obligatorio: las X solo ayudan, nunca hay que cubrirlas.
- Dejar un grupo redundante (todos sus unos ya están cubiertos por otros).
- Negar mal con De Morgan: hay que cambiar la operación **y** negar las variables.

∑ Fórmulas clave

- $\overline{A+B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.
- $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$.
- $A \oplus B = A\overline{B} + \overline{A}B$.
- n variables: 2^n filas y 2^{2^n} funciones distintas (con 4 variables, 65536).

¿Enuncia De Morgan?

$\overline{A+B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.

¿Cuántas funciones lógicas distintas hay de n variables?

2^{2^n}.

¿Qué son puertas universales?

NAND y NOR: con una sola se construye cualquier función.

¿Por qué las columnas de Karnaugh van en orden 00,01,11,10?

Para que las casillas vecinas difieran en un solo bit (código Gray).

¿Cómo se usan las casillas indiferentes?

Solo si permiten hacer un grupo mayor; no hay que cubrirlas.

Minitérmino

AND con todas las variables que vale 1 en una sola fila de la tabla.

Maxitérmino

OR con todas las variables que vale 0 en una sola fila de la tabla.

Implicante primo

Grupo de unos que no se puede ampliar más en el mapa de Karnaugh.

Indiferente (X)

Combinación de entradas que no ocurre o cuya salida da igual.

Puerta universal

Puerta con la que se puede construir cualquier función lógica (NAND o NOR).

Según De Morgan, $\overline{A \cdot B + C}$ es igual a:

A $\overline{A} + \overline{B} + C$

B $(\overline{A} + \overline{B})\,\overline{C}$

C $(\overline{A} + \overline{B}) + \overline{C}$

D $\overline{A}\,\overline{B} + \overline{C}$

Se cambia + por · y se niega cada término: $\overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B})\overline{C}$.

En un mapa de Karnaugh, ¿cuántas casillas puede tener un grupo válido?

A 1, 2, 4, 8...

B Solo números pares

C Cualquier número

D Hasta un máximo de 4

Los grupos son potencias de 2.

¿Cuál es la forma simplificada de $AB + A\overline{B} + \overline{A}B$?

A $\overline{A} + \overline{B}$

B $A \oplus B$

C $A + B$

D AB

$AB + A\overline{B} = A$, y queda $A + \overline{A}B = A + B$.

TEORÍA POR TEMAS

Tema 4 · Familias lógicas

Qué hay dentro de una puerta real: niveles de tensión, márgenes de ruido, fan-out, consumo y retardo.

> 💡 **Desde cero**
>
> Hasta ahora una puerta era un símbolo perfecto. En la realidad es un circuito con transistores, con **tensiones** que no son exactamente 0 y 5 V, que **consume** energía y que **tarda** un poco en responder. Una **familia lógica** es un conjunto de puertas fabricadas con la misma tecnología (TTL, CMOS…) y diseñadas para conectarse entre sí. Este tema responde: ¿se pueden conectar estas dos puertas? ¿Cuántas? ¿Qué velocidad y consumo tendrá el conjunto?

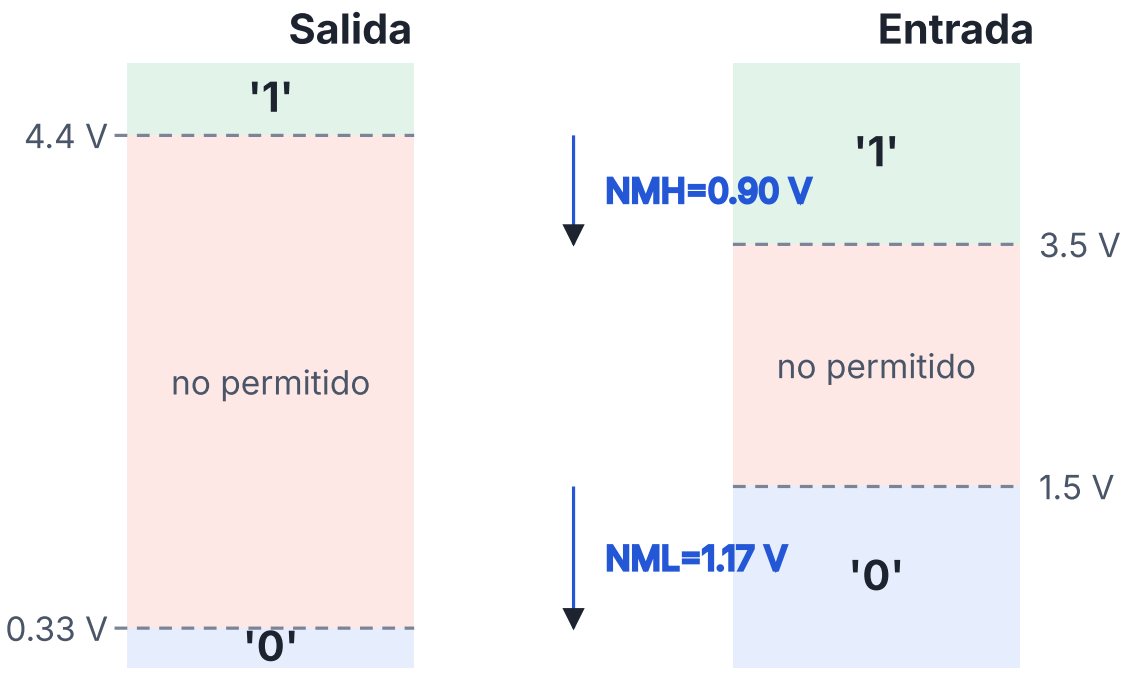
Niveles lógicos y margen de ruido

Un 0 o un 1 es un **rango** de tensión. Cada familia da cuatro números (valores típicos de un 74LS de TTL y de un 74HC de CMOS a 5 V):

Parámetro	Significa	TTL (74LS)	CMOS (74HC)
V_{OH}	salida mínima en nivel alto	2,7 V	4,4 V
V_{OL}	salida máxima en nivel bajo	0,5 V	0,33 V
V_{IH}	entrada mínima que se reconoce como alta	2,0 V	3,5 V
V_{IL}	entrada máxima que se reconoce como baja	0,8 V	1,5 V

La salida de una puerta alimenta la entrada de la siguiente. Para que funcione, lo que **garantiza** la salida debe cumplir lo que **exige** la entrada. La diferencia es el **margen de ruido**: cuánto ruido puede sumarse al cable sin que se estropee el dato.

$$NM_H = V_{OH} - V_{IH} \qquad NM_L = V_{IL} - V_{OL}$$



74HC a 5 V

Márgenes de ruido de CMOS: 0,9 V en alto y 1,17 V en bajo.

Ejemplo T4 **Márgenes de ruido de TTL y CMOS** ✔ verificado

Calcula los márgenes de ruido de la familia TTL (74LS) y de la CMOS (74HC) de la tabla anterior. ¿Cuál tolera más ruido?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: $NM_H = V_{OH} - V_{IH}$ y $NM_L = V_{IL} - V_{OL}$. El margen **efectivo** de una conexión es el menor de los dos.

1 TTL

$NM_H = 2{,}7 - 2{,}0 = 0{,}7$ V y $NM_L = 0{,}8 - 0{,}5 = 0{,}3$ V.

2 CMOS

$NM_H = 4{,}4 - 3{,}5 = 0{,}9$ V y $NM_L = 1{,}5 - 0{,}33 = 1{,}17$ V.

3 Compara

El peor caso de TTL es $0{,}3$ V; el de CMOS, $0{,}9$ V. CMOS aguanta el triple de ruido en el peor caso.

✅ **Resultado final**

TTL: $0{,}7$ V y $0{,}3$ V. CMOS: $0{,}9$ V y $1{,}17$ V. Gana CMOS.



Captura del simulador interactivo. Pruébalo en la versión web.

Fan-out: cuántas puertas puede mover una salida

Cada entrada **absorbe** o **suministra** algo de corriente. Una salida solo puede dar una corriente máxima. El **fan-out** es el número de entradas que una salida puede alimentar sin salirse de los niveles garantizados. Se calcula para cada nivel y vale el **menor**:

$$\text{fan-out} = \min\left(\frac{I_{OL}}{I_{IL}},\ \frac{I_{OH}}{I_{IH}}\right)$$

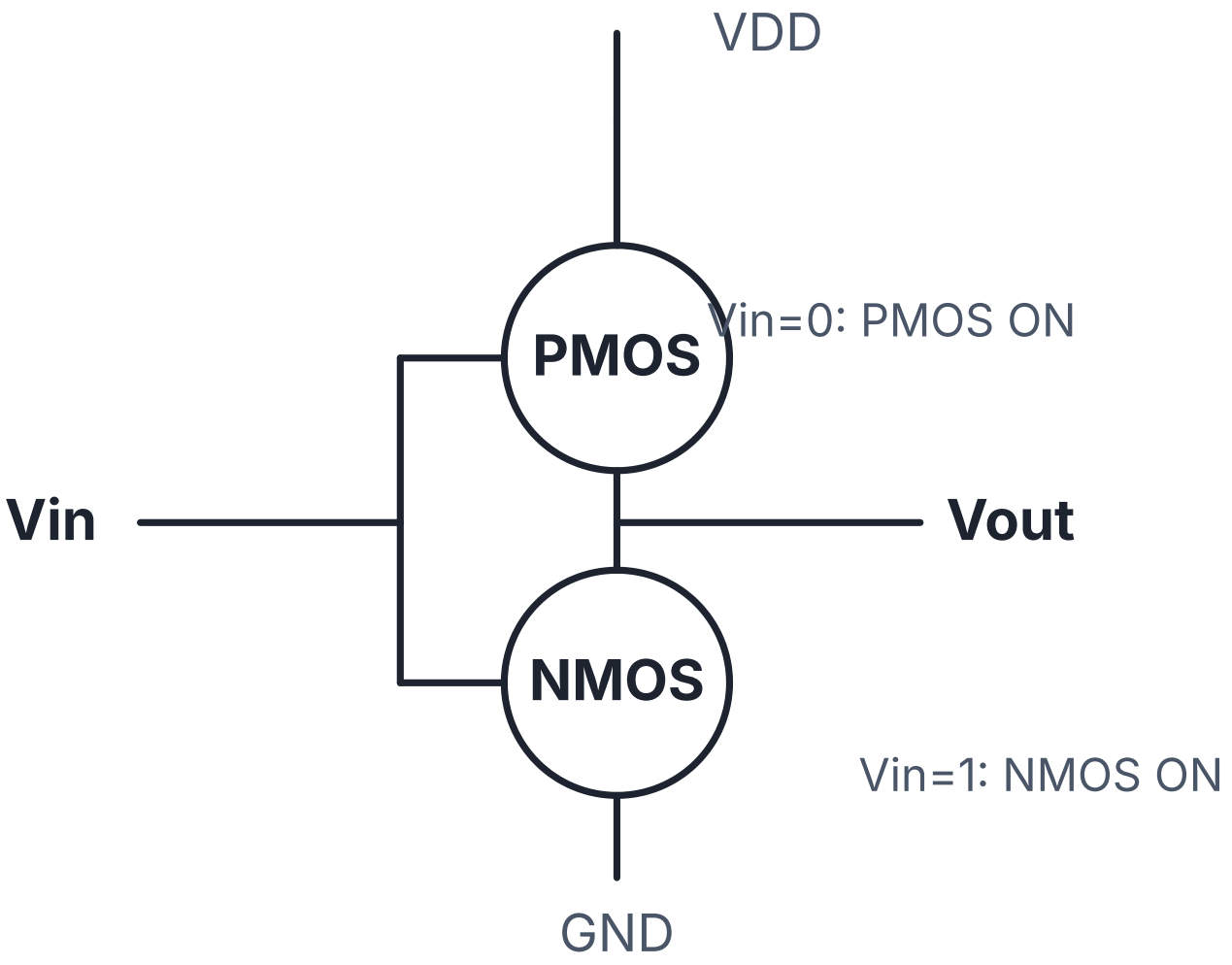
Para un 74LS: $I_{OL} = 8$ mA e $I_{IL} = 0{,}4$ mA dan 20; $I_{OH} = 0{,}4$ mA e $I_{IH} = 20$ μA dan 20. El fan-out es **20**. En CMOS las entradas casi no toman corriente, así que el límite real lo pone la capacidad (retardo), no la corriente.

Retardo y potencia

- **Retardo de propagación t_{pd}**: lo que tarda la salida en reaccionar cuando cambia la entrada. En una cadena de puertas, **los retardos se suman**: 5 puertas de 10 ns dan 50 ns.
- **Potencia en CMOS:** casi no consume en reposo, pero **cada conmutación** carga y descarga una capacidad:

$$P_{din} = C\, V_{DD}^2\, f$$

Con $C = 20$ pF, $V_{DD} = 5$ V y $f = 1$ MHz: $P = 20 \cdot 10^{-12} \cdot 25 \cdot 10^{6} = 0{,}5$ mW. Doblar la frecuencia dobla la potencia; doblar la tensión la multiplica por cuatro.



Inversor CMOS: con la entrada a 0 conduce el PMOS y la salida va a VDD; con la entrada a 1 conduce el NMOS y va a tierra. Nunca conducen los dos a la vez, por eso casi no consume en reposo.

Salidas especiales

- **Colector abierto / drenador abierto:** la salida solo sabe llevar a 0; para el 1 hace falta una resistencia externa (*pull-up*). Permite conectar varias salidas al mismo cable (AND cableada).
- **Tres estados (*tri-state*):** la salida puede estar en 0, 1 o **alta impedancia** (Z, «desconectada»). Permite que muchos dispositivos compartan un bus: solo uno habilita su salida a la vez.
- **Entradas sin conectar:** en CMOS una entrada al aire **flota** y puede oscilar y consumir. Conéctala siempre a 0 o a 1.

🚧 Qué suele fallar

- Comparar V_{OH} con V_{OL} en vez de V_{OH} con V_{IH} para el margen.
- Dar como fan-out el cociente de un solo nivel en lugar del **mínimo** de los dos.
- Sumar retardos en paralelo: solo se suman los de la **ruta más larga**.
- Creer que CMOS no consume: no consume en reposo, pero sí al conmutar, y crece con la frecuencia.
- Conectar dos salidas totem-pole directamente: si una da 1 y otra 0, hay cortocircuito. Para eso existen colector abierto y tres estados.

∑ Fórmulas clave

- $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
- fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$.
- $P_{din} = C\,V_{DD}^2\,f$.
- Retardo de una ruta: suma de los t_{pd} de sus puertas.

¿Margen de ruido en nivel alto?

$NM_H = V_{OH} - V_{IH}$.

¿Cómo se calcula el fan-out?

El menor de I_{OL}/I_{IL} y I_{OH}/I_{IH}.

¿Para qué sirve el colector abierto?

Permite unir varias salidas en un cable (AND cableada) usando una resistencia pull-up.

¿Qué es el estado Z?

Alta impedancia: la salida queda desconectada. Permite compartir un bus.

¿De qué depende la potencia dinámica de CMOS?

De la capacidad, de la tensión al cuadrado y de la frecuencia.

Margen de ruido

Cuánto ruido tolera una conexión sin que se interprete mal el nivel lógico.

Fan-out

Número máximo de entradas que una salida puede alimentar.

Retardo de propagación

Tiempo que tarda la salida en reaccionar a un cambio de la entrada.

Colector abierto

Salida que solo fuerza el 0; el 1 lo pone una resistencia externa.

Alta impedancia (Z)

Estado de una salida tri-state en el que está eléctricamente desconectada.

Una puerta tiene $V_{OH} = 2{,}7$ V y la siguiente exige $V_{IH} = 2{,}0$ V. El margen de ruido en alto es:

- **A** 0,7 V
- **B** 4,7 V
- **C** 0,3 V
- **D** 2,0 V

$NM_H = V_{OH} - V_{IH} = 0{,}7$ V.

Una salida da $I_{OL} = 16$ mA y $I_{OH} = 0{,}8$ mA; cada entrada pide $I_{IL} = 0{,}8$ mA e $I_{IH} = 40$ μA. El fan-out es:

- **A** 40
- **B** 1
- **C** 16
- **D** 20

$16/0{,}8 = 20$ y $0{,}8\ \text{mA}/40\ \mu\text{A} = 20$. El mínimo es 20.

Una puerta CMOS conmuta a 2 MHz con $C = 10$ pF y $V_{DD} = 3$ V. Su potencia dinámica es:

- **A** 0,18 mW
- **B** 1,8 mW
- **C** 0,9 mW

D 0,06 mW

$P = 10 \cdot 10^{-12} \cdot 9 \cdot 2 \cdot 10^{6} = 1{,}8 \cdot 10^{-4}$ W $= 0{,}18$ mW.

TEORÍA POR TEMAS

Tema 5 · Funciones de la lógica combinacional

Sumadores, comparadores, multiplexores, decodificadores, codificadores y displays de 7 segmentos.

> 💡 **Desde cero**
>
> Los mismos circuitos se necesitan una y otra vez: sumar, comparar, elegir una entrada entre varias, convertir un número en una línea activa. Los fabricantes los venden ya hechos (bloques **MSI**). Diseñar «a nivel de subsistema» es **usar estos bloques como piezas** en lugar de dibujar puertas. Saber qué hace cada pieza y cómo conectarla es la mitad del examen de combinacional.

Sumadores

- **Semisumador** (suma 2 bits): $S = A \oplus B$, $C_{out} = A \cdot B$.
- **Sumador completo** (suma 2 bits y el acarreo de entrada C_{in}): $S = A \oplus B \oplus C_{in}$ y $C_{out} = AB + C_{in}(A \oplus B)$.

Se ha comprobado por código con las 8 combinaciones que da la suma binaria correcta.

Sumador de n bits con acarreo en cascada (*ripple carry*): encadenas n sumadores completos, el C_{out} de uno es el C_{in} del siguiente. Es barato, pero el acarreo tiene que **viajar** por todos: con 4 etapas y 2 puertas de 10 ns por etapa, el retardo es $4 \cdot 2 \cdot 10 = 80$ ns. Los **sumadores con acarreo anticipado** (*carry look-ahead*) calculan los acarreos en paralelo y son más rápidos.

Restar con un sumador: $A - B = A + \overline{B} + 1$ (invertir B y poner $C_{in} = 1$). Con XOR en la entrada B el mismo circuito suma o resta según una señal de control.

Ejemplo T5 — Sumador de 4 bits

✔ verificado

Dibuja cómo se construye un sumador de 4 bits con sumadores completos y calcula su retardo en el peor caso si cada puerta tarda 10 ns y cada sumador completo tiene dos niveles de puertas en el camino del acarreo.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: sumador completo, cascada de acarreo, retardo = suma de retardos de la ruta más larga.

1 Cascada

Los bits $A_3A_2A_1A_0$ y $B_3B_2B_1B_0$ entran por pares en cuatro sumadores completos. C_{in} del primero es 0 y cada C_{out} va al C_{in} siguiente.

2 Ruta crítica

El peor caso es cuando el acarreo nace en el bit 0 y viaja hasta el bit 3: pasa por 4 sumadores.

3 Retardo

$4\ \text{etapas} \times 2\ \text{puertas} \times 10\ \text{ns} = 80\ \text{ns}.$

✅ **Resultado final**

Cuatro sumadores completos en cascada. Retardo del acarreo: **80 ns**.

Comparadores

Un comparador de 1 bit da tres salidas: $A > B = A\overline{B}$, $A = B = \overline{A \oplus B}$ y $A < B = \overline{A}B$. Para n bits se compara **de más a menos significativo**: manda el primer bit que difiere. Los circuitos integrados tienen entradas de cascada para unir varios.

Multiplexores

Un **multiplexor** (MUX) de 2^k a 1 tiene 2^k entradas de datos, k entradas de **selección** y una salida: copia a la salida la entrada que indica el número de selección. Es un «selector de vía».

$$Y = \sum_{i=0}^{2^k-1} m_i(S) \cdot D_i$$

Truco de examen: implementar una función con un MUX. Una función de n variables cabe en un MUX de 2^{n-1} a 1. Las $n-1$ primeras variables van a la selección y la última variable (o su negada, 0 o 1) a cada entrada de datos.

Ejemplo T5 **Implementar una función con un MUX 4:1** ✔ verificado

Implementa $f(A, B, C) = \Sigma m(1, 2, 4, 7)$ con un multiplexor 4:1. Usa A y B como selección.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: para cada valor de (A, B), mira cómo depende f de C en las dos filas correspondientes: da 0, 1, C o $\overline{C}$. Eso es lo que va a la entrada D_i de ese valor.

1 Mira las parejas de filas

$AB = 00$: filas 0 y 1. $f(0) = 0$, $f(1) = 1 \Rightarrow D_0 = C$. $AB = 01$: filas 2 y 3. $f = 1, 0 \Rightarrow D_1 = \overline{C}$. $AB = 10$: filas 4 y 5. $f = 1, 0 \Rightarrow D_2 = \overline{C}$. $AB = 11$: filas 6 y 7. $f = 0, 1 \Rightarrow D_3 = C$.

2 Conecta

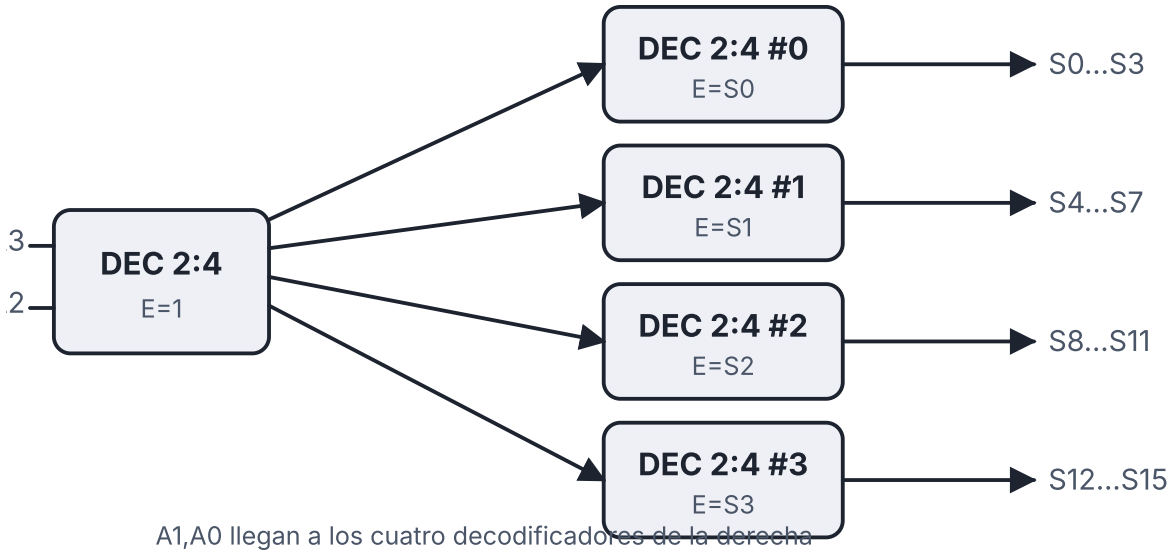
Selección: $S_1 = A$, $S_0 = B$. Entradas: $D_0 = C$, $D_1 = \overline{C}$, $D_2 = \overline{C}$, $D_3 = C$. Solo hace falta un inversor.

> ✅ **Resultado final**
>
> $Y = \text{MUX}(A, B;\ C, \overline{C}, \overline{C}, C)$. Comprobado con las 8 combinaciones.

Decodificadores y codificadores

- **Decodificador** k a 2^k: activa **una** salida según el número de entrada. Cada salida es un **minitérmino** de las entradas. Por eso con un decodificador y una puerta OR puedes implementar cualquier función: juntas las salidas de los minitérminos que valen 1. Tiene entrada de habilitación (*enable*).
- **Codificador:** hace lo inverso: de 2^k líneas, activa el código de la línea activa. El **codificador de prioridad** decide cuando hay varias líneas a 1: gana la de mayor índice. Para las entradas 0110 da 2 y para 1010 da 3.

Ampliar: con decodificadores 2:4 puedes montar uno de 4:16 en árbol: uno de 2:4 habilita a cuatro más.



Un decodificador 4:16 hecho con cinco 2:4. Los dos bits altos eligen cuál de los cuatro de la derecha se habilita; los dos bajos eligen la salida dentro de él.

Display de 7 segmentos

Un display tiene 7 LED (a a g). Un **decodificador BCD a 7 segmentos** recibe 4 bits (cifra de 0 a 9) y da 7 salidas. Las combinaciones 1010 a 1111 no existen en BCD: son **indiferentes**, y se aprovechan para simplificar. En las placas Basys los segmentos son **activos a nivel bajo**: se encienden con 0.

🚧 Qué suele fallar

- Montar el sumador sin poner $C_{in} = 0$ en el primer bit.
- Confundir qué es selección y qué es dato en un MUX.
- Olvidar la habilitación (*enable*) de un decodificador o mezclar su polaridad (activa a nivel bajo).
- En el codificador de prioridad, ignorar que **la entrada de mayor índice gana**.
- En el display, olvidar que en la Basys se enciende con 0.

∑ Fórmulas clave

- $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$.
- $A - B = A + \overline{B} + 1$.
- MUX $2^k : 1$ implementa cualquier función de $k + 1$ variables.
- Salida i de un decodificador $= m_i$ (minitérmino).

¿Ecuación del acarreo de un sumador completo?

$C_{out} = AB + C_{in}(A \oplus B)$.

¿Cómo se resta con un sumador?

$A + \overline{B} + 1$: se invierte B y se pone $C_{in} = 1$.

¿Qué número de selección necesita un MUX de 8 a 1?

3 bits ($2^3 = 8$).

¿Qué es cada salida de un decodificador?

Un minitérmino de las entradas.

¿Qué hace un codificador de prioridad?

Entrega el código de la entrada activa de mayor índice cuando hay varias.

MSI

Integración de media escala: bloques como sumadores, multiplexores o decodificadores.

Multiplexor

Selector que copia a la salida una de varias entradas según un código de selección.

Decodificador

Activa una única salida según el código binario de entrada.

Codificador de prioridad

Da el código de la línea activa de mayor prioridad.

Acarreo anticipado

Técnica que calcula los acarreos en paralelo para acelerar un sumador.

Para implementar una función de 4 variables con un solo multiplexor, ¿cuál es el mínimo?

A Imposible con un solo MUX

B MUX 4:1

C MUX 16:1 y nada más

D MUX 8:1

Con $2^{n-1} = 8$ entradas de datos y 3 variables en selección; la cuarta va a los datos.

Un decodificador 3:8 tiene la entrada 101. ¿Qué salida se activa?

A S_6

B S_1

C S_5

D S_3

$101_2 = 5$. Se activa la salida S_5.

Un sumador de 8 bits con acarreo en cascada tiene un retardo de 15 ns por etapa. El retardo del acarreo es:

A 120 ns

B 23 ns

C 15 ns

D 8 ns

$8 \cdot 15 = 120$ ns: el acarreo atraviesa las 8 etapas.

TEORÍA POR TEMAS

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

La memoria de 1 bit: latch, flip-flop SR, JK, D y T, tablas características y de excitación.

> 💡 **Desde cero**
>
> Un **biestable** es un circuito con dos estados estables que **recuerda un bit**. Un **flip-flop** cambia de estado solo cuando llega un **flanco** del reloj (el instante en que la señal de reloj pasa de 0 a 1, o de 1 a 0). Entre flanco y flanco no escucha a las entradas. Esa disciplina es lo que permite que un circuito con miles de biestables funcione ordenado: todos cambian a la vez, en el mismo instante.

Latch frente a flip-flop

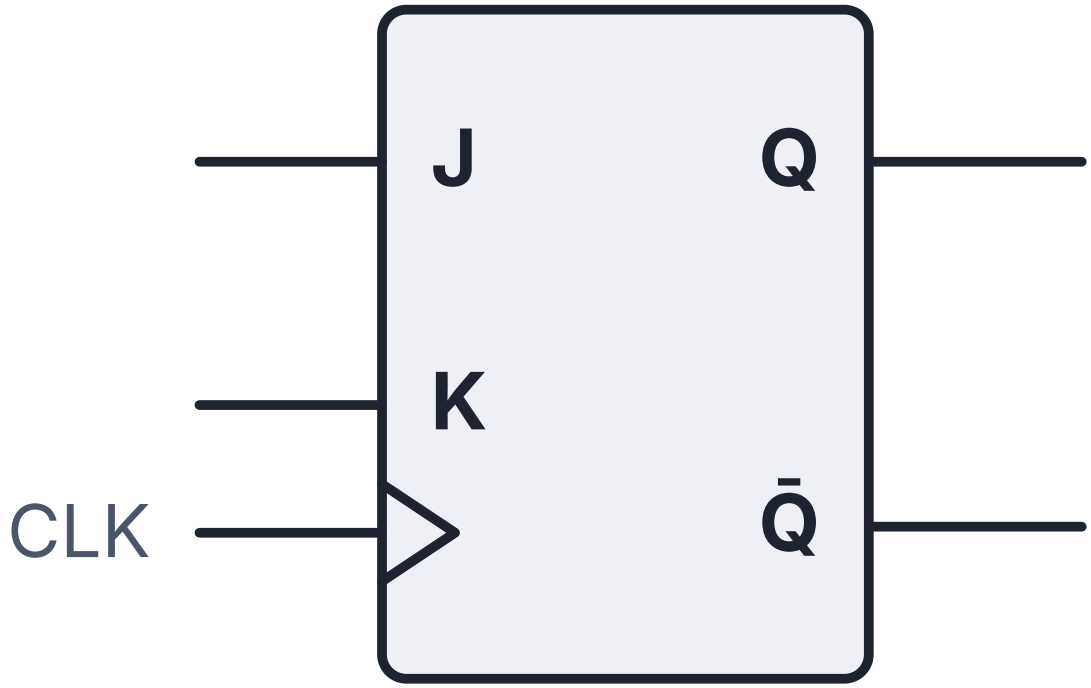
- **Latch** (cerrojo): es **transparente** mientras el reloj (o *enable*) está activo: la salida sigue a la entrada todo ese tiempo.
- **Flip-flop disparado por flanco:** muestrea las entradas **solo en el flanco** y mantiene la salida el resto del ciclo.

Usa flip-flops para diseñar secuenciales síncronos. Los latches dan problemas de temporización.

Los cuatro tipos y sus tablas

Q es el estado actual y Q^+ el estado después del flanco.

Tipo	Ecuación característica	Qué hace
SR	$Q^+ = S + \overline{R}\,Q$ (con $SR = 0$)	$S = 1$ pone a 1; $R = 1$ pone a 0; $S = R = 0$ mantiene; $S = R = 1$ **prohibido**
JK	$Q^+ = J\overline{Q} + \overline{K}\,Q$	Como SR, pero $J = K = 1$ **conmuta**
D	$Q^+ = D$	Copia D en cada flanco
T	$Q^+ = T \oplus Q$	$T = 1$ conmuta; $T = 0$ mantiene



Símbolo del flip-flop JK. El triángulo en la entrada de reloj indica que se dispara por flanco.

Se ha comprobado por código que la tabla del JK, la del D y la del T cumplen estas ecuaciones para todas las combinaciones.

Tabla de excitación: lo que necesitas para diseñar

La tabla característica dice «con estas entradas, ¿qué pasa?». Al **diseñar** preguntas lo contrario: «quiero pasar de Q a Q^+, ¿qué entradas pongo?». Es la **tabla de excitación**:

$Q \to Q^+$	J	K	D	T
$0 \to 0$	0	X	0	0
$0 \to 1$	1	X	1	1
$1 \to 0$	X	1	0	1
$1 \to 1$	X	0	1	0

El JK tiene muchos **indiferentes (X)**: eso suele dar ecuaciones más simples que con D, a costa de dos entradas por biestable.

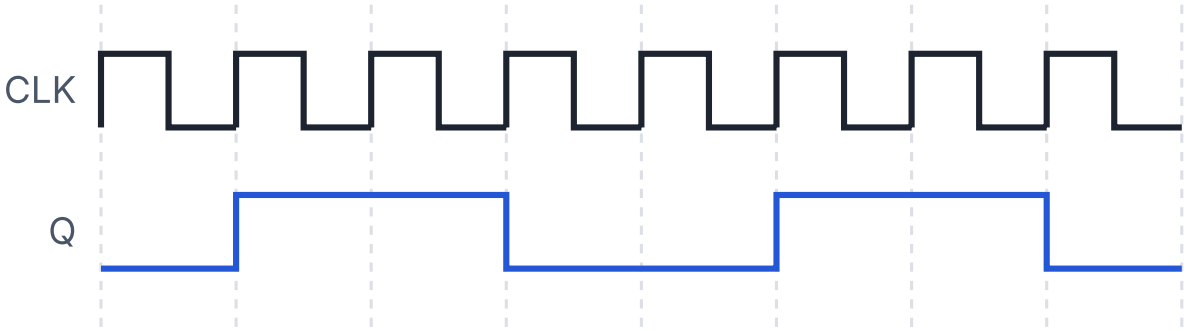
> **💡 Por qué hay una X en el JK**
>
> Para pasar de 0 a 1 puedes poner $J = 1$ y $K = 0$ (poner a 1) **o** $J = 1$ y $K = 1$ (conmutar). El valor de K da igual. Pasa lo mismo en los otros tres casos.



Captura del simulador interactivo. Pruébalo en la versión web.

Biestables como divisores de frecuencia

Un JK con $J = K = 1$ (o un T con $T = 1$, o un D con $D = \overline{Q}$) **conmuta en cada flanco**: la salida cambia cada ciclo, así que su frecuencia es la **mitad** de la del reloj. Encadenar n así da una división por 2^n. Es la base de los contadores.



Salida de un biestable T con T=1: periodo doble

Q cambia solo en los flancos de subida del reloj y completa un ciclo cada dos del reloj.

Temporización

- **Tiempo de preparación** (*setup*, t_{su}): la entrada debe estar estable **antes** del flanco.
- **Tiempo de mantenimiento** (*hold*, t_h): debe seguir estable **después** del flanco.
- **Retardo de salida** (t_{cq}): tarda un poco en reflejarse la salida tras el flanco.

Si no se respetan, el biestable cae en un estado **metaestable** (ni 0 ni 1, durante un tiempo impredecible). Por eso las entradas asíncronas (un pulsador) se sincronizan con dos biestables en serie.

Frecuencia máxima: el periodo debe cubrir el viaje completo entre dos flancos:

$$T_{clk} \geq t_{cq} + t_{pd,\,comb} + t_{su}$$

🚧 Qué suele fallar

- Usar la tabla característica para diseñar (hay que usar la de excitación).
- Escribir 0 en una X: las X son libertad, no valores.
- Confundir latch (transparente) con flip-flop (flanco).
- Olvidar que $S = R = 1$ está prohibido en un SR.
- No distinguir **flanco de subida** de **bajada** al dibujar un cronograma.

∑ Fórmulas clave

- $Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$.
- $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
- Divisor por 2: $f_{out} = f_{clk}/2$.

¿Ecuación característica del JK?

$Q^+ = J\overline{Q} + \overline{K}Q.$

¿Qué hace un JK con J=K=1?

Conmuta en cada flanco.

¿Entradas del JK para pasar de 1 a 0?

$J = X$ (cualquiera) y $K = 1$.

¿Qué es el tiempo de setup?

Tiempo que la entrada debe estar estable antes del flanco de reloj.

¿Qué diferencia hay entre latch y flip-flop?

El latch es transparente mientras está habilitado; el flip-flop muestrea solo en el flanco.

Flanco

Instante en que la señal de reloj pasa de 0 a 1 (subida) o de 1 a 0 (bajada).

Biestable

Circuito de dos estados estables que almacena un bit.

Tabla de excitación

Dice qué entradas del biestable hacen falta para ir de Q a Q^+.

Metaestabilidad

Estado intermedio e impredecible tras violar los tiempos de setup o hold.

Setup / hold

Tiempos de preparación y mantenimiento de las entradas alrededor del flanco.

Un biestable JK tiene $Q = 1$ y recibe $J = 0$, $K = 0$. Tras el flanco:

A $Q = 1$ y luego conmuta

B $Q = 1$ (mantiene)

C $Q = 0$

D Estado prohibido

$J = K = 0$ mantiene. Con $Q^+ = J\overline{Q} + \overline{K}Q = 1$.

Quieres pasar de $Q = 1$ a $Q^+ = 0$ con un biestable T. ¿Qué pones en T?

A $T = X$

B $T = 1$

C $T = 0$

D No es posible

Para cambiar de estado, $T = 1$.

Un biestable T con $T = 1$ recibe un reloj de 8 MHz. La salida tiene frecuencia:

A 2 MHz

B 8 MHz

C 4 MHz

D 16 MHz

Conmuta en cada flanco: la frecuencia se divide por 2.

TEORÍA POR TEMAS

Tema 7 · Memorias y circuitos lógicos programables

ROM, RAM, cómo se calcula su tamaño y cómo una memoria o una PLA implementan funciones.

> 💡 **Desde cero**
>
> Una **memoria** es una tabla: das una **dirección** y te devuelve el **dato** guardado ahí. Con k líneas de dirección hay 2^k posiciones, y cada una guarda m bits. Una memoria $2^k \times m$ guarda $m \cdot 2^k$ bits. Ese mismo hecho (una tabla indexada por la entrada) significa que **una ROM es una tabla de verdad con otra forma**: cualquier función combinacional cabe en una ROM.

Tipos de memoria

Tipo	Se puede escribir	Se pierde al apagar	Uso típico
ROM	No (se programa en fábrica)	No	Código fijo
PROM / EPROM / EEPROM / Flash	Sí, pocas veces y despacio	No	Firmware, pendrive
SRAM	Sí, rápido	Sí	Cachés, registros
DRAM	Sí, necesita refresco	Sí	Memoria principal

Tamaño y organización

$$\text{bits totales} = 2^k \cdot m \qquad k = \log_2(\text{posiciones})$$

- Una memoria de $8\text{K}\times 8$ necesita $k = 13$ líneas de dirección ($2^{13} = 8192$).
- Una de $64\text{K}\times 16$ guarda $64 \cdot 1024 \cdot 16 = 1\,048\,576$ bits.

Ampliar la memoria:

- **Más ancho** (más bits por dato): pon chips en paralelo con las mismas direcciones.
- **Más largo** (más posiciones): pon chips con las líneas altas de dirección pasando por un **decodificador** que habilita un solo chip cada vez.

Ejemplo T7 Ampliar una memoria

✔ verificado

Con chips de $2\text{K}\times 8$ construye una memoria de $8\text{K}\times 16$. ¿Cuántos chips necesitas y cuántas líneas de dirección tiene el conjunto?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: ampliar a lo ancho (bits) y a lo largo (posiciones), y el número de líneas de dirección $k = \log_2 N$.

1 Direcciones

$8\text{K} = 2^{13}$ posiciones: **13** líneas de dirección. Cada chip usa 11 ($2^{11} = 2\text{K}$).

2 Ancho

Cada chip da 8 bits y necesito 16: hacen falta **2 chips** en paralelo.

3 Largo

Cada chip tiene 2K posiciones y necesito 8K: hacen falta $8/2 = 4$ filas de chips. Las 2 líneas altas (A_{12}, A_{11}) entran en un decodificador 2:4 que habilita una fila.

4 Total

$4 \times 2 = 8$ chips.

✅ **Resultado final**

8 chips y **13** líneas de dirección.

La memoria como circuito combinacional

Una ROM de 2^k palabras de m bits **es** un circuito de k entradas y m salidas: cada dirección es una fila de la tabla de verdad. Ejemplo: un **sumador completo** (C_{in}, A, B como dirección) cabe en una ROM de 8×2 cuyo contenido es $0, 1, 1, 2, 1, 2, 2, 3$ (el número de unos de la dirección, en binario).

Lógica programable: PLA, PAL

- **PLA** (*programmable logic array*): un plano **AND programable** genera términos producto y un plano **OR programable** los suma.
- **PAL:** plano AND programable y plano OR **fijo**. Más barata y rápida.
- **Ventaja sobre la ROM:** una ROM genera **todos** los minitérminos (2^n), la PLA solo los **términos producto simplificados** que necesitas. Por eso, si simplificas antes (Karnaugh), cabe mucho más en una PLA.

> 🔭 **Más a fondo: PLD, CPLD y FPGA**
>
> Las **PLD** simples (PAL, GAL) tienen decenas de funciones. Las **CPLD** juntan varias PLD con una red de interconexión. Las **FPGA** (Tema 12) cambian el plano AND-OR por miles de pequeñas tablas (*LUT*) y registros, con interconexión programable: son las más flexibles y las que se usan en el laboratorio.

> 🚧 **Qué suele fallar**
>
> - Calcular las líneas de dirección con el número de bits del dato en vez del número de posiciones.
> - Olvidar que 1K = 1024, no 1000.
> - Al ampliar «a lo largo», conectar todos los chips habilitados a la vez: solo debe estar activo uno.
> - Pensar que la ROM guarda datos «que cambian»: solo se lee.

> ∑ **Fórmulas clave**
>
> - bits totales $= 2^k \cdot m$; $k = \log_2 N$.
> - Chips necesarios $= \dfrac{N_{total}}{N_{chip}} \cdot \dfrac{m_{total}}{m_{chip}}$.
> - Una ROM $2^n \times m$ implementa m funciones de n variables.

¿Cuántos bits tiene una memoria de $2^k \times m$?

$m \cdot 2^k$ bits.

¿Cuántas líneas de dirección tiene una memoria de 16K posiciones?

14 ($2^{14} = 16384$).

¿Qué diferencia hay entre ROM y RAM?

La ROM solo se lee y no se borra al apagar; la RAM se lee y escribe y (volátil) se borra.

¿Qué plano es fijo en una PAL?

El plano OR.

ROM

Memoria de solo lectura: guarda una tabla fija.

RAM

Memoria de lectura y escritura, volátil.

PLA

Matriz lógica programable con plano AND y plano OR programables.

PAL

Matriz con plano AND programable y plano OR fijo.

Volátil

Que pierde su contenido al quitar la alimentación.

¿Cuántas líneas de dirección tiene una memoria de 32K×8?

A 15

B 32

C 5

D 8

$32\text{K} = 2^5 \cdot 2^{10} = 2^{15}$.

¿Cuántos chips de 4K×4 hacen falta para una memoria de 8K×8?

A 2

B 16

C 4

D 8

A lo ancho, $8/4 = 2$; a lo largo, $8\text{K}/4\text{K} = 2$. Total $2 \cdot 2 = 4$.

Una ROM de 8 posiciones y 2 bits puede implementar:

A Un sumador completo (3 entradas, 2 salidas)

B Un contador de 8 bits

C Un biestable

D Cualquier función de 4 entradas

Tiene 3 líneas de dirección (entradas) y 2 bits de dato (salidas).

TEORÍA POR TEMAS

Tema 8 · Contadores

Contadores asíncronos y síncronos, módulo N, carga y puesta a cero, divisores de frecuencia.

> 💡 **Desde cero**
>
> Un **contador** es un circuito secuencial cuyo estado **avanza una cuenta** con cada pulso de reloj: 0, 1, 2, 3… y vuelve a empezar. Se usa para medir tiempo, dividir frecuencias, generar direcciones y controlar secuencias. Un contador de **módulo** N recorre N estados distintos y necesita $n = \lceil \log_2 N \rceil$ biestables.

Contador binario asíncrono (*ripple*)

Es una cadena de biestables T (o JK con $J = K = 1$) donde la salida de cada uno es el **reloj del siguiente**. Solo el primero recibe el reloj externo. Cada etapa divide la frecuencia por 2.

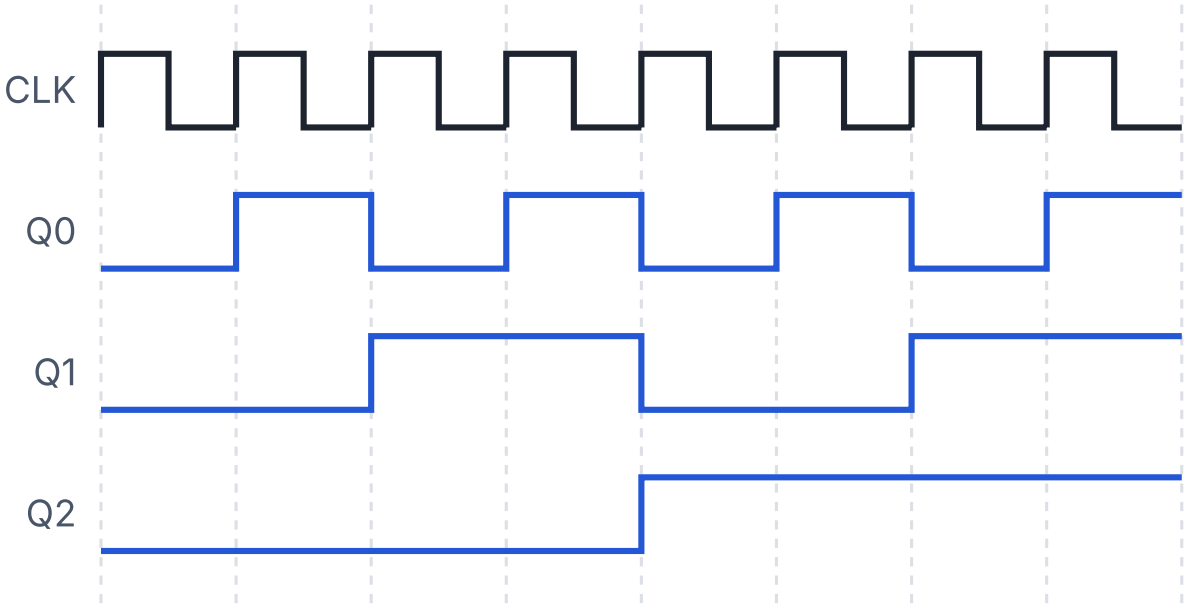
- Ventaja: muy simple.
- Inconveniente: los cambios **se propagan en cadena** (de ahí *ripple*). Un contador de 4 bits con $t_{pd} = 12$ ns por etapa tarda $4 \cdot 12 = 48$ ns en estabilizarse, así que el reloj máximo es unos $1/48\ \text{ns} \approx 20{,}8$ MHz, y mientras se propaga hay valores intermedios erróneos (*glitches*).

Contador síncrono

Todos los biestables comparten el mismo reloj. El biestable i debe conmutar cuando **todos los bits menores valen 1**:

$$T_i = Q_{i-1} \cdot Q_{i-2} \cdots Q_0$$

Es más rápido (no hay propagación en cadena) y no tiene glitches. Se ha comprobado por código que con $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_1 Q_0$ la secuencia de 3 bits es $0, 1, 2, 3, 4, 5, 6, 7, 0, \ldots$



Contador binario de 3 bits (cuenta ascendente)

Q0 divide por 2, Q1 por 4 y Q2 por 8. Cada bit conmuta cuando todos los de menor peso valen 1.

SIMULADOR · DISEÑO DE CONTADORES SÍNCRONOS CON BIESTABLES JK

Secuencia de cuenta (estados en decimal, separados por comas)

0,2,4,6,1,3,5,7

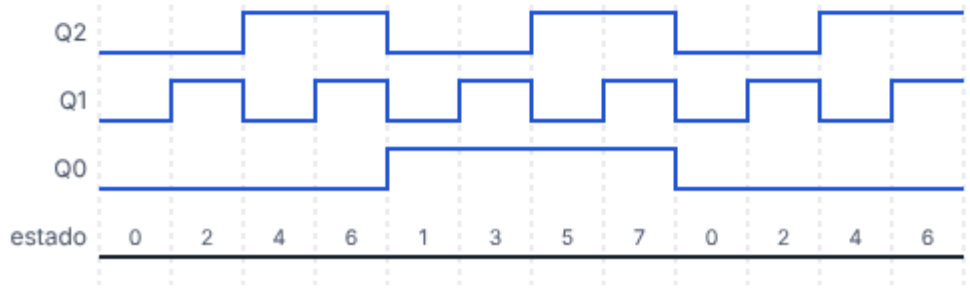
☐ Añadir entrada X (X=0 sube, X=1 baja por la secuencia inversa)

Ciclos de reloj a dibujar

12

```
Estados: 8 → 3 biestables JK (2^3 = 8 ≥ 8)
Estados no usados: 0 (indiferentes en los mapas)

J2 = Q1
K2 = Q1
J1 = 1
K1 = 1
J0 = Q2Q1
K0 = Q2Q1
```



Captura del simulador interactivo. Pruébalo en la versión web.

Contadores de módulo distinto de 2^n

Para contar N estados que no sean potencia de 2, hay tres maneras:

1. **Puesta a cero cuando se alcanza N:** detecta el estado N con una puerta y activa el *reset* (asíncrono: el estado N dura un instante y es un *glitch*; síncrono: el reset actúa en el siguiente flanco).
2. **Carga paralela:** al llegar al último estado, carga el valor inicial.
3. **Diseño directo como autómata** (Tema 10): tabla de transiciones y ecuaciones de entrada de los biestables. Es lo que se pide en examen.

Contador módulo 10 (BCD): $0 \to 1 \to \cdots \to 9 \to 0$. Módulo 12: $0 \to \cdots \to 11 \to 0$ y a 1 MHz la salida más lenta sale a $1\ \text{MHz}/12 \approx 83{,}33$ kHz.

Ejemplo T8 **Contador módulo 12 y su frecuencia de salida** ✔ verificado

Se quiere un contador síncrono de módulo 12 con reloj de 1 MHz. (a) ¿Cuántos biestables hacen falta? (b) ¿Cuál es el último estado? (c) ¿A qué frecuencia se repite el ciclo completo?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: $n = \lceil \log_2 N \rceil$ biestables; un contador de módulo N repite su ciclo cada N pulsos; $f_{out} = f_{clk}/N$.

1 Número de biestables

$2^3 = 8 < 12 \leq 2^4 = 16 \Rightarrow$ **4 biestables**.

2 Último estado

Cuenta $0, 1, \ldots, 11$. El último es $11 = 1011_2$ y el siguiente es 0000. Los estados 12 a 15 **no se usan**.

3 Frecuencia del ciclo

$$f = \frac{1\ \text{MHz}}{12} = 83{,}33\ \text{kHz}.$$

✅ **Resultado final**

(a) 4. (b) 1011_2 (11). (c) $83{,}33$ kHz.

Contadores en circuito integrado

Los contadores comerciales (74×161, 74×163…) incorporan: **carga paralela** (*LOAD*), **puesta a cero** (*CLR*), habilitación de cuenta (*ENP, ENT*), sentido ascendente/descendente (*UP/DOWN*) y una salida de **fin de cuenta** (*RCO/carry*) para **cascadear** varios. Lee siempre en la hoja si cada entrada es **síncrona o asíncrona** y si es **activa a nivel bajo**.

🔭 Más a fondo: contador reversible

Un contador ascendente/descendente con una señal X ($X = 1$ sube, $X = 0$ baja) tiene $T_i = \overline{X} \cdot \overline{Q_{i-1}} \cdots + X \cdot Q_{i-1} \cdots$. Una versión útil es la de los **contadores en código no binario**, como el de secuencia $0, 2, 4, 6, 1, 3, 5, 7$.

🚧 Qué suele fallar

- Contar mal los biestables: para módulo N son $\lceil \log_2 N \rceil$, no N.
- Olvidar que el reset **asíncrono** produce un estado intermedio fugaz.
- En un contador asíncrono, calcular la frecuencia máxima con el retardo de **una** etapa en vez de **todas**.
- Cascadear sin usar la salida de fin de cuenta o la habilitación.
- Confundir *carga síncrona* (espera al flanco) con *carga asíncrona* (inmediata).

∑ Fórmulas clave

- $n = \lceil \log_2 N \rceil$; estados no usados: $2^n - N$.
- $f_{out} = f_{clk} / N$.
- Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$.
- Asíncrono: $t_{total} = n \cdot t_{pd}$.

¿Cuántos biestables necesita un contador de módulo 60?

6 ($2^5 = 32 < 60 \leq 64$).

¿Ecuación de T_2 en un contador síncrono binario?

$T_2 = Q_1 Q_0$.

¿Ventaja del contador síncrono?

No acumula retardos de propagación y no genera glitches.

¿Qué es un glitch?

Un valor erróneo fugaz mientras los biestables de un contador asíncrono cambian en cadena.

¿Frecuencia de salida de un divisor por N?

f_{clk} / N.

Contador módulo N

Contador que recorre N estados y vuelve al inicial.

Contador síncrono

Contador en el que todos los biestables reciben el mismo reloj.

Contador asíncrono

Contador en el que cada biestable usa como reloj la salida del anterior.

Carga paralela

Entrada que fuerza al contador a un valor dado.

Fin de cuenta

Salida que avisa de que el contador llega a su último estado, para encadenar contadores.

¿Cuántos biestables necesita un contador de módulo 100?

A 100

B 10

C 6

D 7

$2^6 = 64 < 100 \leq 2^7 = 128$.

Un contador binario síncrono de 4 bits tiene $Q_3Q_2Q_1Q_0 = 0111$. Tras el siguiente flanco:

A 1111

B 0110

C 1000

D 0000

$T_3 = Q_2Q_1Q_0 = 1, T_2 = Q_1Q_0 = 1, T_1 = Q_0 = 1, T_0 = 1$: todos conmutan.

Un contador asíncrono de 5 bits con $t_{pd} = 10$ ns por etapa: ¿cuánto tarda en estabilizarse como máximo?

A 5 ns

B 32 ns

C 10 ns

D 50 ns

$5 \cdot 10 = 50$ ns.

TEORÍA POR TEMAS

Tema 9 · Registros de desplazamiento

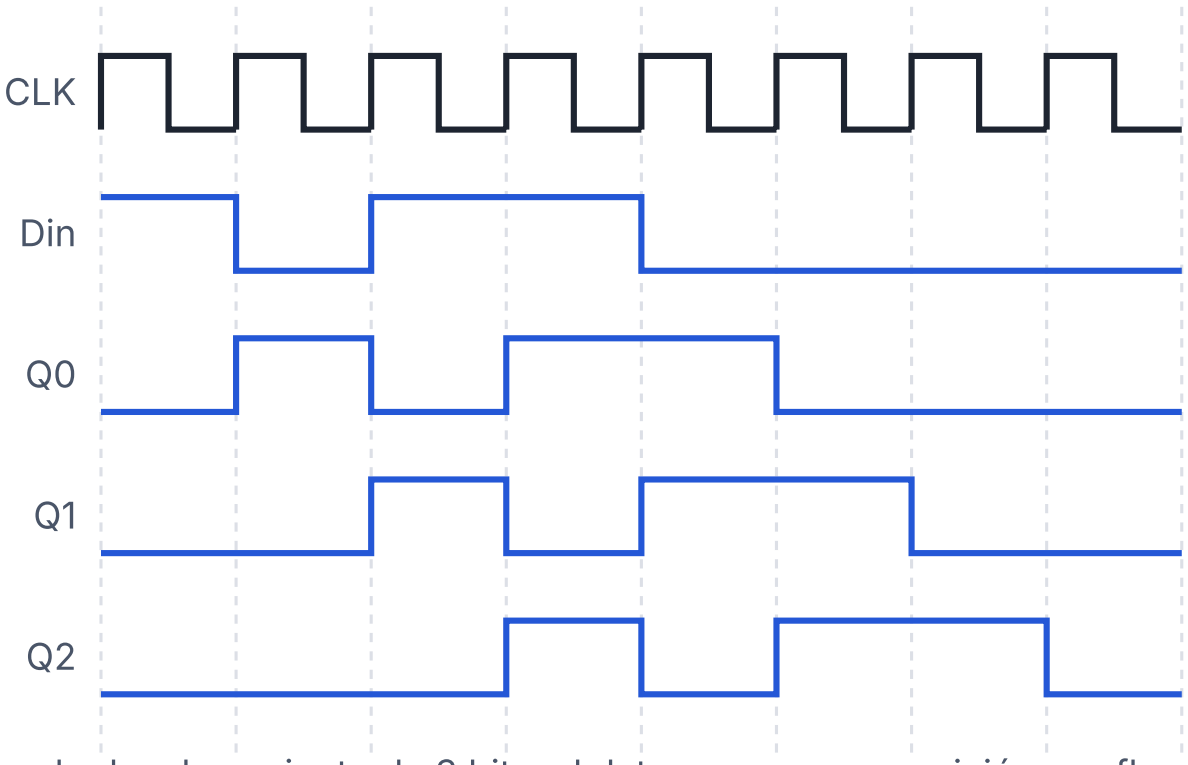
Registros serie/paralelo, contadores en anillo y Johnson, y generadores pseudoaleatorios (LFSR).

> 💡 **Desde cero**
>
> Un **registro** es un grupo de biestables D que guardan una palabra. Un **registro de desplazamiento** además puede **mover** cada bit un lugar a la derecha o a la izquierda en cada flanco, como una cola. Sirve para pasar datos de serie a paralelo (y al revés), para retardar señales, y para construir contadores y generadores de secuencias.

Cómo funciona

Cada biestable toma como entrada la salida del anterior: $D_i = Q_{i-1}$. En cada flanco, todo se mueve una posición y entra un bit nuevo por un extremo.

Si metes en serie los bits $1, 0, 1, 1$ en un registro de 4 bits, tras 4 flancos tiene 1011 en paralelo. Se ha comprobado por código.



Registro de desplazamiento de 3 bits: el dato avanza una posición por flanco

Cada bit tarda un ciclo en pasar de una etapa a la siguiente.

Los cuatro modos

Entrada → Salida	Nombre	Uso
Serie → serie	SISO	Línea de retardo
Serie → paralelo	SIPO	Recibir datos de un puerto serie
Paralelo → serie	PISO	Transmitir una palabra por un solo cable
Paralelo → paralelo	PIPO	Registro de almacenamiento

Los registros universales (74×194) tienen entradas de **modo** que eligen entre cargar en paralelo, desplazar a la izquierda, a la derecha o mantener.

Contadores basados en registros

- **Contador en anillo:** se realimenta la salida del último en la entrada del primero con **un solo 1** circulando: $1000 \to 0100 \to 0010 \to 0001 \to 1000$. Con n biestables da n **estados**. Es un decodificador gratis: cada salida es un estado.
- **Contador Johnson (anillo retorcido):** se realimenta la salida **negada**. Para 4 bits: $0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001$. Da $2n$ **estados** y cada transición cambia un solo bit (como un Gray).

Ejemplo T9 **Secuencias de anillo y de Johnson** ✔ verificado

Un registro de 4 bits parte de 0000 (Johnson) y de 1000 (anillo). Escribe la secuencia y cuántos estados distintos tiene cada uno.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: anillo: $D_3 = Q_0$ (se desplaza a la derecha y el LSB vuelve al MSB). Johnson: $D_3 = \overline{Q_0}$.

1 Anillo

$1000 \to 0100 \to 0010 \to 0001 \to 1000$. **4 estados** para 4 biestables.

2 Johnson

$0000 \to 1000 \to 1100 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 0000$. **8 estados** para 4 biestables.

✅ **Resultado final**

Anillo: 4 estados. Johnson: 8 estados ($2n$).

LFSR: registro con realimentación lineal

Si se realimenta la entrada con un **XOR** de algunas salidas, el registro recorre una secuencia pseudoaleatoria. Con 4 bits y realimentación $Q_3 \oplus Q_2$ se recorren los **15** estados no nulos antes de repetir (periodo $2^4 - 1$). Se usa en comunicaciones, cifrado sencillo y generadores de pruebas. El estado todo ceros es un **atasco**: nunca sale de él.

🚧 **Qué suele fallar**

- Dirección del desplazamiento: dibuja siempre qué bit entra y cuál sale.
- Confundir anillo (n estados) con Johnson ($2n$ estados).
- Olvidar poner el estado inicial (un solo 1 en el anillo): si arranca en 0000 no hay nada que circule.
- En un LFSR, arrancar en todo ceros.

∑ **Fórmulas clave**

- Anillo: n estados; Johnson: $2n$; LFSR máximo: $2^n - 1$.
- Serie → paralelo de n bits: n ciclos de reloj.

¿Cuántos estados tiene un contador Johnson de n bits?

$2n$.

¿Qué entra en el primer biestable de un contador en anillo?

La salida del último.

¿Qué es SIPO?

Registro serie-entrada, paralelo-salida.

¿Periodo máximo de un LFSR de n bits?

$2^n - 1$.

Registro de desplazamiento

Cadena de biestables en la que cada uno toma el valor del anterior en cada flanco.

Contador en anillo

Registro de desplazamiento realimentado con un único 1 que circula.

Contador Johnson

Registro de desplazamiento que realimenta la salida negada; $2n$ estados.

LFSR

Registro de desplazamiento con realimentación lineal (XOR) que genera secuencias pseudoaleatorias.

Un contador Johnson de 5 bits tiene:

A 5 estados

B 10 estados

C 32 estados

D 25 estados

$2n = 10$.

Un registro de desplazamiento de 8 bits recibe un dato serie. ¿Cuántos flancos de reloj hacen falta para cargar una palabra completa?

A 3

B 8

C 16

D 1

Entra un bit por flanco.

TEORÍA POR TEMAS

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

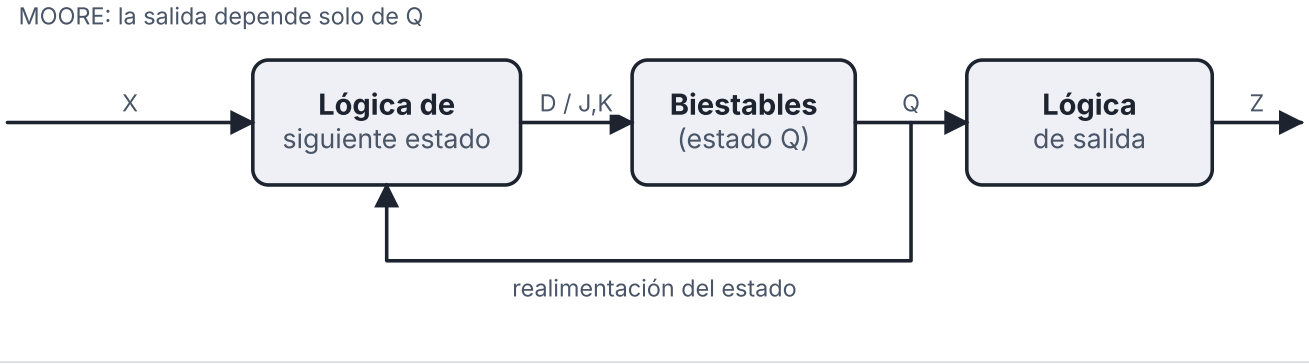
Máquinas de Moore y de Mealy: del enunciado al diagrama de estados, a la tabla y al circuito. El tema que más cae.

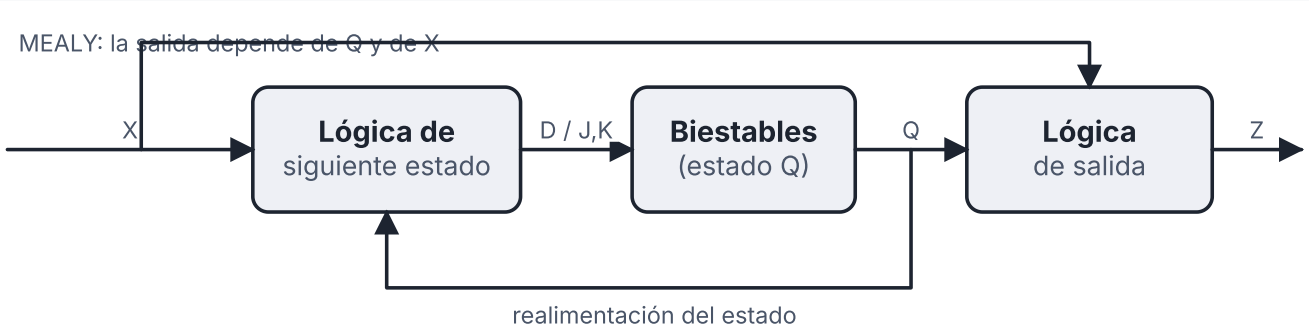
💡 **Desde cero**

Un **autómata** (máquina de estados finitos) es un circuito que **recuerda en qué situación está** (su *estado*) y, según esa situación y las entradas, decide la salida y a qué estado pasa en el siguiente flanco. Un semáforo, una caja fuerte, un ascensor o un detector de secuencias son autómatas. Diseñar uno es siempre el mismo recorrido:

palabras → diagrama de estados → tabla de transiciones → ecuaciones → circuito.

Moore y Mealy





Los dos modelos comparten la parte secuencial (biestables). Se diferencian en de qué depende la salida.

	Moore	Mealy
Salida	$Z = f(Q)$	$Z = f(Q, X)$

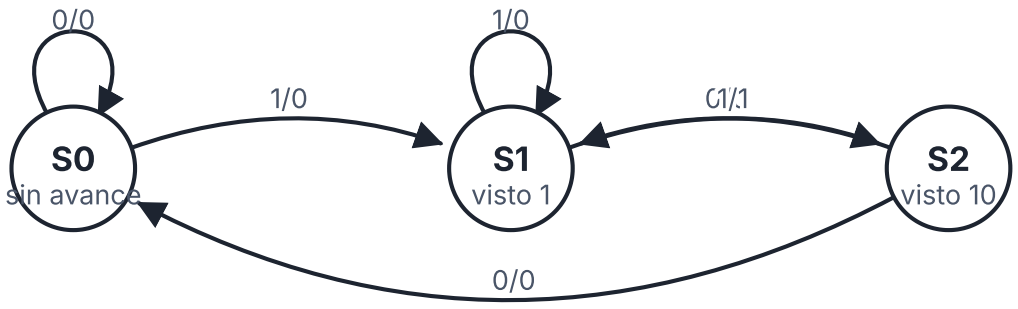
	Moore	Mealy
En el diagrama	Dentro del estado	En la flecha (entrada/salida)
Cambia	Solo en el flanco	En cuanto cambia la entrada
Estados	Suele necesitar más	Suele necesitar menos
Peligro	Sin glitches de entrada	La salida hereda ruidos de la entrada

El método de diseño, paso a paso

1. **Entiende el enunciado:** qué entradas hay, qué salidas, cuándo se activa cada una. Decide Moore o Mealy.
2. **Diagrama de estados:** un estado por cada «situación que hay que recordar». Para cada estado y cada valor de la entrada, una flecha.
3. **Número de biestables:** $n = \lceil \log_2(\text{estados}) \rceil$.
4. **Codificación de estados:** asigna un código binario a cada estado. Afecta al tamaño del circuito.
5. **Tabla de transiciones:** para cada (Q, entradas) escribe Q^+ y la salida.
6. **Tabla de excitación:** con la tabla del tipo de biestable (JK, D, T) escribe qué entradas hacen falta.
7. **Simplifica** con Karnaugh. Los estados sin usar y las entradas imposibles son **indiferentes**.
8. **Dibuja** el circuito y **comprueba** con una secuencia de prueba.

Ejemplo completo: detector de la secuencia 101 (Mealy)

Una entrada serie X y una salida Z. Debe dar $Z = 1$ cuando los **tres últimos** bits recibidos sean $1, 0, 1$ (las secuencias pueden solaparse: 10101 contiene dos).



Detector 101 (Mealy): etiqueta entrada/salida

Tres estados. En S2 (hemos visto 10), un 1 completa la secuencia (Z=1) y además sirve como primer 1 de la siguiente.

Ejemplo T10 **Detector de la secuencia 101 con biestables D** ✔ verificado

Diseña un detector Mealy de la secuencia 101 con solape y biestables D. Comprueba con la entrada $X = 1, 0, 1, 0, 1, 1, 0, 1$.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: Mealy, tabla de transiciones, biestable D ($D = Q^+$), Karnaugh con la combinación $Q_1Q_0 = 11$ sin usar como indiferente.

1 Estados y codificación

S_0: ningún avance. S_1: último bit 1. S_2: últimos bits 10. Son 3 estados: $n = 2$ biestables. Codifico $S_0 = 00$, $S_1 = 01$, $S_2 = 10$. La combinación 11 queda sin usar.

2 Tabla de transiciones

Q_1Q_0	X	$Q_1^+Q_0^+$	Z
00	0	00	0
00	1	01	0
01	0	10	0
01	1	01	0

Q_1Q_0	X	$Q_1^+Q_0^+$	Z
10	0	00	0
10	1	01	1

3 **Ecuaciones**

Con D, $D = Q^+$. $Q_0^+ = 1$ siempre que $X = 1$: $D_0 = X$. $Q_1^+ = 1$ solo en (01, $X = 0$): $D_1 = Q_0\overline{X}$. La salida vale 1 solo en (10, $X = 1$): $Z = Q_1X$.

4 **Comprueba con la secuencia**

Estado inicial 00. $X = 1$: 01, $Z = 0$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$. $X = 1$: 01, $Z = 0$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$.

✅ **Resultado final**

$D_1 = Q_0\overline{X}$, $D_0 = X$, $Z = Q_1X$. Para $X = 1, 0, 1, 0, 1, 1, 0, 1$ la salida es $0, 0, 1, 0, 1, 0, 0, 1$. Verificado contra una especificación independiente con las 2^{10} secuencias de hasta 10 bits.

💡 **Moore equivalente**

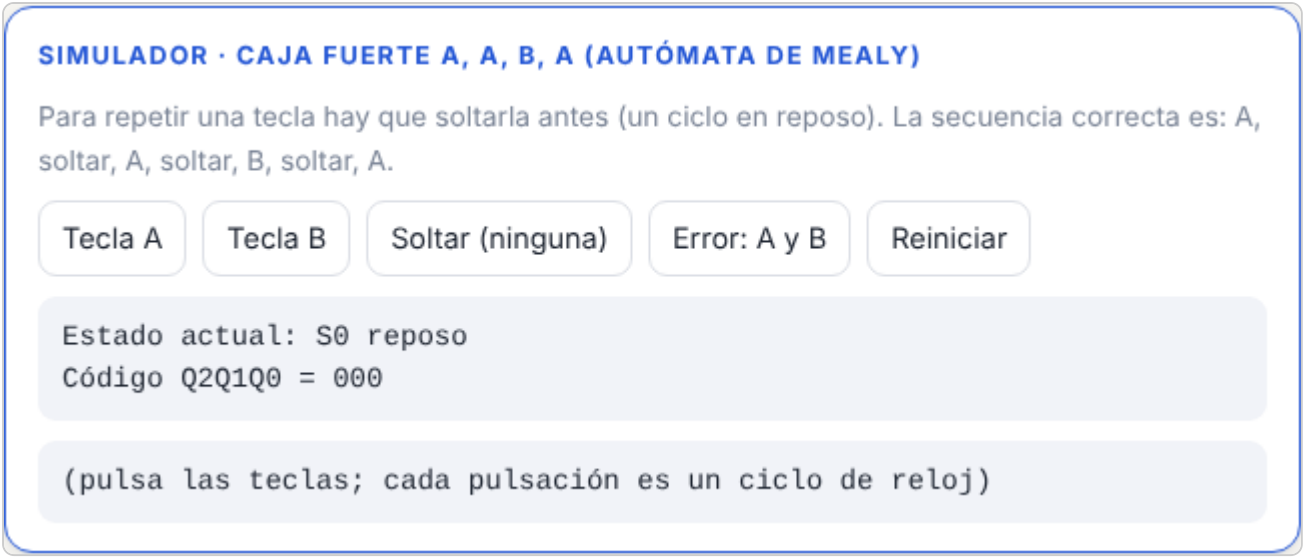
El mismo detector en Moore necesita **4 estados** (hay que añadir un estado «visto 101» donde la salida vale 1), pero la salida solo cambia en el flanco. Se ha verificado por código que es equivalente a la versión Mealy.

Análisis: de un circuito a su comportamiento

Es el camino inverso, y suele ser más corto:

1. Escribe las **ecuaciones de entrada** de los biestables y de la salida a partir del circuito.
2. Construye la **tabla de transiciones** con la tabla característica del biestable.
3. Dibuja el **diagrama de estados**.
4. Con una secuencia de entradas, dibuja el **cronograma**.

Ejemplo de análisis con T: si $T_2 = Q_1 \oplus Q_0$, $T_1 = Q_0$ y $T_0 = \overline{Q_2}$, desde 000 la secuencia de estados es $0, 1, 6, 2, 7, 5, 3, 0$ (período 7). Verificado por código.



Captura del simulador interactivo. Pruébalo en la versión web.

Antirrebote y sincronización de entradas

Un pulsador real **rebota** unos milisegundos al pulsarse. Si lo conectas directamente a un contador, cuenta decenas de pulsaciones. Soluciones: (a) un **filtro**: esperar a que la entrada permanezca estable, por ejemplo, 20 ms; a 50 MHz son $50 \cdot 10^6 \cdot 0{,}02 = 10^6$ ciclos, es decir, un contador de **20 bits**. (b) Pasar la señal por **dos biestables D** para sincronizarla con el reloj y evitar la metaestabilidad.

🔭 Más a fondo: reducción de estados

Dos estados son **equivalentes** si dan las mismas salidas ante todas las secuencias posibles. Fusionarlos reduce biestables y puertas. Se usa la **tabla de implicación** o el método de particiones. Antes de codificar, revisa si hay estados que se pueden unir.

🚧 Qué suele fallar

- Poner la salida de Mealy dentro del estado (o la de Moore en la flecha).
- Olvidar el estado inicial y un modo de forzarlo (reset).
- Contar mal los biestables: con 5 estados hacen falta 3 (no 5).
- Usar la tabla característica en lugar de la de excitación al derivar J, K.
- Olvidar que las combinaciones de código sin usar y las entradas imposibles son **indiferentes**.
- No comprobar el circuito con una secuencia: es la forma más rápida de pillar un error.

∑ Fórmulas clave

- $n = \lceil \log_2(\#\text{estados}) \rceil$.
- Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$.
- D: $D = Q^+$. T: $T = Q \oplus Q^+$. JK: usar tabla de excitación.

¿Diferencia entre Moore y Mealy?

En Moore la salida solo depende del estado; en Mealy, del estado y de la entrada.

¿Cuántos biestables para 9 estados?

4 ($2^3 = 8 < 9 \leq 16$).

¿Entradas D para un estado destino Q^+?

$D = Q^+$.

¿Entrada T para pasar de Q a Q^+?

$T = Q \oplus Q^+$.

¿Qué haces con las combinaciones de código sin usar?

Las marcas como indiferentes (X) en Karnaugh.

Autómata síncrono

Circuito secuencial cuyos biestables cambian todos con el mismo reloj.

Estado

Resumen de la historia del circuito que basta para decidir qué hará.

Diagrama de estados

Grafo con un nodo por estado y una flecha por transición.

Codificación de estados

Asignación de un código binario a cada estado.

Rebote

Oscilaciones breves de un contacto mecánico al abrir o cerrar.

Un autómata tiene 6 estados. ¿Cuántos biestables necesita como mínimo?

A 3

B 6

C 4

D 2

$2^2 = 4 < 6 \leq 2^3 = 8.$

En una máquina de Mealy, la salida:

A Depende solo del estado

B Depende solo de la entrada

C Depende del estado y de la entrada actual

D Es siempre síncrona

Esa es la definición; en Moore depende solo del estado.

Con biestables D, ¿cuál es la entrada D de un biestable que debe pasar de 0 a 1?

A $D = \overline{Q}$

B $D = 0$

C $D = X$

D $D = 1$

En un biestable D, $D = Q^+$.

Un pulsador rebota 10 ms. Con reloj de 100 MHz, ¿cuántos ciclos debe esperar el filtro para ser fiable?

A Al menos 10^6

B 10^9

C 100

D 10^3

$10 \cdot 10^{-3}\ \mathrm{s} \cdot 100 \cdot 10^6\ \mathrm{Hz} = 10^6$ ciclos.

TEORÍA POR TEMAS

Tema 11 · Descripciones funcionales y estructurales

Distintas formas de describir un mismo circuito: ecuaciones, tablas, diagramas, bloques y código.

> 💡 **Desde cero**
>
> Un mismo circuito se puede **describir** de muchas formas. Una **descripción estructural** dice **de qué piezas** está hecho y cómo se conectan (un esquemático). Una **descripción funcional** dice **qué hace** sin decir cómo (una tabla, unas ecuaciones, un diagrama de estados, un programa). Saber pasar de una a otra es lo que permite usar herramientas de diseño: tú describes el comportamiento y la herramienta fabrica la estructura.

Las formas de describir

Descripción	Tipo	Cuándo usarla
Tabla de verdad	Funcional	Pocas entradas
Ecuaciones booleanas	Funcional	Lógica combinacional pequeña
Diagrama de estados / tabla de transiciones	Funcional	Autómatas
Esquemático de puertas	Estructural	Ver el circuito, depurar
Diagrama de bloques	Estructural (alto nivel)	Sistemas grandes
Lenguaje HDL (VHDL)	Ambas	Todo; es lo que usa el laboratorio

Niveles de abstracción

De más bajo a más alto: **transistor → puerta → bloque MSI (sumador, contador…) → sistema**. El diseño **jerárquico** usa piezas ya hechas y probadas. Primero se diseña y simula cada bloque; luego se unen. Es la forma de abordar el proyecto integrador.



Flujo de diseño con herramientas EDA

Flujo de diseño. Si falla un paso, se vuelve a la descripción.

Del comportamiento a la estructura

El flujo de diseño con herramientas EDA:

1. **Descripción** (esquemático, diagrama de estados o VHDL).
2. **Simulación funcional:** comprueba que *hace lo que debe*, sin retardos.
3. **Síntesis:** la herramienta convierte la descripción en puertas y biestables.
4. **Implementación:** coloca y conecta los recursos de la FPGA.
5. **Simulación temporal y prueba en placa.**

🚧 Qué suele fallar

- Confundir «estructural» con «funcional»: estructural = piezas y cables; funcional = comportamiento.
- Saltarse la simulación funcional y depurar directamente en la placa.
- Describir en un bloque todo el sistema en lugar de diseñar por partes.

¿Qué es una descripción estructural?

Dice de qué componentes consta el circuito y cómo se conectan.

¿Qué es una descripción funcional?

Dice qué hace el circuito sin indicar su estructura interna.

¿Qué hace la síntesis?

Convierte la descripción en puertas y biestables.

EDA

Herramientas de diseño electrónico asistido por ordenador.

Síntesis

Transformación de una descripción de alto nivel en una red de puertas y biestables.

Descripción estructural

Descripción por componentes e interconexiones.

Descripción funcional

Descripción por comportamiento.

Un archivo VHDL que describe un sumador con la expresión `s <= a + b;` es una descripción:

- **A** Física
- **B** De transistores
- **C** Estructural
- **D** Funcional (de comportamiento)

Dice qué calcula, no con qué puertas.

¿Qué paso del flujo de diseño comprueba que el circuito hace lo que debe, sin tener en cuenta los retardos?

- **A** La simulación funcional
- **B** La síntesis
- **C** La implementación
- **D** La programación de la placa

La simulación funcional ignora los retardos reales; la temporal los incluye.

TEORÍA POR TEMAS

Tema 12 · Introducción a los dispositivos lógicos programables

Qué es una FPGA, cómo está hecha por dentro (LUT, biestables, interconexión) y cómo se programa.

> 💡 **Desde cero**
>
> Un circuito digital normal tiene su función **fija** al fabricarlo. Un **dispositivo lógico programable** se fabrica «en blanco» y tú decides **después** qué circuito es, cargándole un fichero de configuración. Una **FPGA** (*field-programmable gate array*) es el más flexible: contiene miles de bloques lógicos pequeños y una red de cables que se puede reconectar. Es como una caja de piezas de electrónica digital que se recoloca con un fichero.

Qué hay dentro de una FPGA

- **CLB / slice** (bloque lógico configurable): la unidad básica. Contiene **tablas de consulta** (LUT) y **biestables**.
- **LUT de 4 entradas:** una memoria de $2^4 = 16$ bits. Almacena la tabla de verdad de **cualquier** función de 4 variables ($2^{16} = 65\,536$ funciones posibles).
- **Interconexión programable:** cables y conmutadores que unen los bloques.
- **Bloques de E/S:** adaptan los pines a distintos niveles de tensión.
- **Bloques extra:** RAM en bloque, multiplicadores, gestores de reloj.

Una función de **más de 4 variables** se reparte entre varias LUT conectadas.

¿Por qué FPGA y no un microcontrolador?

	Microcontrolador	FPGA
Ejecuta	Instrucciones, una tras otra	Hardware real, **todo en paralelo**
Velocidad por tarea	Limitada por el programa	Muy alta (cada módulo es un circuito)
Cambiar la función	Cambiar el programa	Reconfigurar el hardware
Facilidad	Alta	Más trabajo de diseño

Flujo de trabajo en el laboratorio

1. Describes el diseño (esquemático, diagrama de estados o VHDL).
2. **Simulas** para comprobar la función.
3. **Sintetizas** y **implementas** (colocar y conectar).
4. Asignas **pines** (fichero de restricciones) a interruptores, LEDs, displays y reloj.
5. Generas el fichero de configuración y **programas la placa**.

La placa **Basys** tiene una FPGA de Xilinx, interruptores, pulsadores, LEDs, displays de 7 segmentos y un reloj de **50 MHz**.

`Ejemplo T12` **Un reloj lento a partir de los 50 MHz de la placa** ✔ verificado

La placa da un reloj de 50 MHz. (a) ¿Hasta cuánto debe contar un divisor para generar 1 Hz y cuántos bits necesita? (b) Para refrescar un display de 4 dígitos con un contador de 18 bits cuyos 2 bits altos eligen el dígito, ¿a qué frecuencia cambia el dígito y a qué frecuencia se refresca cada uno?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: divisor de frecuencia $f_{out} = f_{in}/DIV$; la salida conmuta cada $DIV/2$ ciclos; bits $= \lceil \log_2(\text{cuenta}) \rceil$.

1 **(a) Divisor a 1 Hz**

$DIV = 50 \cdot 10^6/1 = 50\,000\,000$. La salida debe conmutar cada medio periodo: $DIV/2 = 25\,000\,000$ ciclos. El contador cuenta de 0 a $24\,999\,999$. Como $2^{24} = 16{,}8 \cdot 10^6 < 25 \cdot 10^6 \leq 2^{25}$, hacen falta **25 bits**.

2 **(b) Cambio de dígito**

Los dos bits altos de un contador de 18 bits cambian cada 2^{16} ciclos: $50 \cdot 10^6/2^{16} = 762{,}9$ Hz.

3 **(b) Refresco de cada dígito**

Hay 4 dígitos por barrido: $762{,}9/4 = 190{,}7$ Hz por dígito. Por encima de unos 60 Hz el ojo no ve parpadeo.

✅ **Resultado final**

(a) Cuenta hasta $24\,999\,999$ con 25 bits. (b) Cambio de dígito a $762{,}9$ Hz; refresco de cada dígito a $190{,}7$ Hz.

🔭 Más a fondo: CPLD frente a FPGA y memoria de configuración

Las **CPLD** guardan su configuración en memoria no volátil (el circuito está listo al encender) y tienen retardos predecibles. Las **FPGA** de Xilinx usan **SRAM**: pierden la configuración al apagar y la cargan de una memoria externa (o del PC) al encender. Por eso al reiniciar la placa hay que volver a programarla si no usas memoria flash.

🚧 Qué suele fallar

- Olvidar asignar los **pines**: el diseño compila pero no hace nada en la placa.
- Suponer que la placa tiene los displays «activos a 1»: en la Basys se encienden con 0.
- Usar un reloj dividido como si fuera un reloj de la FPGA: es mejor usar **un solo reloj** y una señal de habilitación.
- Pensar que una FPGA «ejecuta» el código VHDL. El código **describe hardware** que existe a la vez.

¿Qué almacena una LUT de 4 entradas?

16 bits: la tabla de verdad de una función de 4 variables.

¿Qué ventaja tiene la FPGA sobre un microcontrolador?

Paralelismo real: cada bloque es hardware que funciona a la vez.

¿Qué reloj trae la placa Basys?

50 MHz.

¿Qué pasa con una FPGA SRAM al apagarse?

Pierde la configuración y hay que volver a cargarla.

FPGA

Dispositivo lógico reprogramable formado por bloques lógicos y una red de interconexión.

LUT

Tabla de consulta: memoria que almacena la tabla de verdad de una función de pocas variables.

CLB

Bloque lógico configurable de una FPGA (LUT y biestables).

Restricciones (UCF)

Fichero que asocia las señales del diseño con los pines físicos de la FPGA.

Síntesis

Conversión de la descripción en puertas y biestables.

Una LUT de 5 entradas almacena:

A 16 bits

B 32 bits

C 5 bits

D 25 bits

$2^5 = 32$ posiciones de un bit.

¿Qué hace falta para que un diseño aparezca en los LEDs de la placa?

A Solo simularlo

B Asignar las señales a los pines de la FPGA

C Cambiar el reloj de la placa

D Nada más que sintetizarlo

Sin las restricciones de pines, la herramienta coloca las señales donde quiere.

TEORÍA POR TEMAS

Tema 13 · Introducción al diseño digital con VHDL

Entidad y arquitectura, descripciones de flujo, de comportamiento y estructurales, procesos y máquinas de estados.

> 💡 **Desde cero**
>
> **VHDL** es un lenguaje para **describir hardware**, no para programar un procesador. Cuando escribes `y <= a and b;` no le dices «calcula» a un ordenador: describes que **existe una puerta AND** entre `a`, `b` e `y`. Dos líneas de VHDL a la vez son dos circuitos funcionando a la vez. Esta idea es la que más cuesta y la que más acaba cayendo en examen.

Las dos piezas de todo diseño

- `entity` (entidad): la **caja negra**. Dice el nombre y los puertos (entradas y salidas).
- `architecture` (arquitectura): lo que hay **dentro** de la caja.

vhdl · Plantilla mínima

```vhdl
 library ieee;
use ieee.std_logic_1164.all;

entity puerta_and is
  port ( a, b : in  std_logic;
         y    : out std_logic );
end puerta_and;

architecture flujo of puerta_and is
begin
  y <= a and b;
end flujo;
```

Tipos que vas a usar: `std_logic` (un bit, con valores `'0'`, `'1'`, `'Z'`, `'X'`...), `std_logic_vector(3 downto 0)` (un bus de 4 bits), `unsigned` / `signed` (para aritmética, en `ieee.numeric_std`) e `integer`.

Tres estilos de descripción

1. **Flujo de datos:** ecuaciones con asignaciones concurrentes (`<=`). Incluye `when ... else` y `with ... select`.

2. **Comportamiento:** dentro de un `process` con `if`, `case`... Se ejecuta en secuencia **en la simulación**, pero describe hardware.
3. **Estructural:** instancias de otros componentes conectadas con `port map`. Es el equivalente a un esquemático.

vhdl · Flujo de datos: multiplexor (p2_mux4.vhd)

```vhdl
 -- Práctica 2: multiplexor 4 a 1 (dos descripciones equivalentes).
library ieee;
use ieee.std_logic_1164.all;

entity p2_mux4 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    sel : in  std_logic_vector(1 downto 0);
    y   : out std_logic
  );
end p2_mux4;

architecture rtl of p2_mux4 is
begin
  with sel select
    y <= d(0) when "00",
         d(1) when "01",
         d(2) when "10",
         d(3) when others;
end rtl;
```

Combinacional y secuencial en VHDL

- **Combinacional:** proceso con **todas** las entradas en la lista de sensibilidad (o asignaciones concurrentes). Si olvidas asignar una salida en alguna rama del `if`, la herramienta infiere un **latch** (memoria no deseada). Regla de oro: **toda salida se asigna en todos los casos**.
- **Secuencial (síncrono):** proceso sensible al reloj con `rising_edge(clk)`. Lo que se asigna ahí se convierte en **biestables**.

vhdl · Secuencial: contador con reset y habilitación (p3_contador.vhd)

```vhdl
 -- Práctica 3: contador síncrono de módulo N con reset asíncrono y habilitación.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p3_contador is
  generic ( N : positive := 10 );
  port (
    clk, rst, en : in  std_logic;
    q            : out std_logic_vector(3 downto 0);
    fin          : out std_logic                      -- vale 1 en el último estado
  );
end p3_contador;
```

```
architecture rtl of p3_contador is
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= (others => '0');
    elsif rising_edge(clk) then
      if en = '1' then
        if cnt = N - 1 then
          cnt <= (others => '0');
        else
          cnt <= cnt + 1;
        end if;
      end if;
    end if;
  end process;

  q   <= std_logic_vector(cnt);
  fin <= '1' when cnt = N - 1 else '0';
end rtl;
```

Señales y variables

- Una **señal** (`signal`) es un cable (o un biestable): la asignación `<=` toma efecto **al terminar el proceso**.
- Una **variable** (`variable`) solo existe dentro de un proceso y se asigna al instante con `:=`.

Si lees una señal que acabas de asignar en el mismo proceso, ves el **valor antiguo**. Es una de las trampas más típicas.

Máquinas de estados en VHDL

La forma recomendada son **dos procesos**: uno secuencial que guarda el estado, otro combinacional con el `case` que calcula el siguiente estado y la salida. Es la traducción directa del diagrama de estados de Tema 10.

vhdl · Máquina de estados: detector 101 (p3_detector101.vhd)

```
 -- Práctica 3: máquina de estados. Detector Mealy de la secuencia 101 con solape.
library ieee;
use ieee.std_logic_1164.all;

entity p3_detector101 is
  port (
    clk, rst, x : in  std_logic;
    z           : out std_logic
  );
end p3_detector101;

architecture rtl of p3_detector101 is
  type estado_t is (S0, S1, S2);
```

```
  signal est, sig : estado_t := S0;
begin
  -- Proceso 1: registro de estado (parte secuencial)
  process (clk, rst)
  begin
    if rst = '1' then
      est <= S0;
    elsif rising_edge(clk) then
      est <= sig;
    end if;
  end process;

  -- Proceso 2: lógica de siguiente estado y de salida (parte combinacional)
  process (est, x)
  begin
    z <= '0';
    case est is
      when S0 =>
        if x = '1' then sig <= S1; else sig <= S0; end if;
      when S1 =>
        if x = '0' then sig <= S2; else sig <= S1; end if;
      when S2 =>
        if x = '1' then sig <= S1; z <= '1'; else sig <= S0; end if;
    end case;
  end process;
end rtl;
```

Ejemplo T13 Leer código VHDL: ¿qué circuito es?

✔ verificado

Dado el contador `p3_contador` con $N = 10$, ¿qué secuencia genera `q` si `rst` se activa en el sexto ciclo? ¿Cuántos biestables se sintetizan y qué hace `fin`?

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: proceso síncrono, reset asíncrono, módulo N, número de bits $= \lceil \log_2 N \rceil$.

1 Cuenta normal

Con `en='1'` incrementa en cada flanco: $0, 1, 2, \ldots, 9$ y al llegar a $N - 1 = 9$ vuelve a 0.

2 Efecto del reset

`rst='1'` pone `cnt` a 0 **al instante** (es asíncrono), sin esperar al flanco. En el ejemplo la cuenta reinicia: $\ldots, 4, 5, 0, 1, 2 \ldots$

3 Recursos

`cnt` tiene 4 bits ($2^3 < 10 \leq 2^4$): **4 biestables**, más la lógica de comparación con 9 y de incremento.

4 La salida `fin`

Es combinacional: `fin='1'` cuando `cnt=9`. Sirve para encadenar otro contador.

✅ Resultado final

Secuencia $0, 1, \ldots, 9, 0, \ldots$; 4 biestables; `fin` vale 1 solo en el estado 9.

🔭 Más a fondo: genéricos y jerarquía

Un `generic` es un parámetro de la entidad (`N`, `DIV`...) que se fija al instanciar el componente. Permite reutilizar el mismo código para un contador de módulo 10 o 60. El **diseño jerárquico** crea módulos pequeños y probados (contador, divisor, display) y los une en un módulo superior con `port map`.

🚧 Qué suele fallar

- Pensar que el código se ejecuta «línea a línea» como en C: las asignaciones concurrentes funcionan **a la vez**.
- No asignar una salida en alguna rama: aparece un **latch** inesperado.
- Mezclar `std_logic_vector` con aritmética sin convertir a `unsigned`.
- Olvidar `rising_edge(clk)` y escribir procesos sensibles a todo.
- Leer una señal justo después de asignarla y esperar el valor nuevo.
- Usar un reloj generado por un divisor como si fuera reloj de la FPGA.

∑ Fórmulas clave

- Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos.
- Bits de un contador: $\lceil \log_2 N \rceil$.
- Una `case` completa en un proceso combinatorio: añade siempre `when others`.

¿Qué es una entity?

La interfaz del módulo: nombre y puertos.

¿Qué es una architecture?

La descripción de lo que hay dentro del módulo.

¿Qué infiere VHDL si no asignas una salida en todas las ramas?

Un latch.

¿Cuándo toma efecto una asignación a una signal?

Al terminar el proceso (no al instante).

¿Qué sentencia detecta un flanco de subida?

`rising_edge(clk)`.

VHDL

Lenguaje de descripción de hardware.

Entity

Parte del código VHDL que define las entradas y salidas de un módulo.

Architecture

Parte del código VHDL que define el funcionamiento interno.

Proceso

Bloque `process` de VHDL que describe comportamiento.

Latch inferido

Memoria no deseada que aparece cuando una señal no se asigna en todos los casos.

En VHDL, `y <= a and b;` describe:

- **A** Una puerta AND siempre presente
- **B** Un proceso secuencial
- **C** Una variable temporal
- **D** Una instrucción que se ejecuta una sola vez

Es una asignación concurrente: hardware que existe de forma permanente.

¿Qué provoca un `if` sin `else` dentro de un proceso combinacional?

A Una puerta NOT

B Un error de sintaxis siempre

C Un latch inferido

D Un contador

La salida mantiene su valor en la rama no cubierta, y eso es memoria.

Un contador módulo 10 en VHDL necesita como mínimo una señal de:

A 5 bits

B 10 bits

C 3 bits

D 4 bits

$\lceil \log_2 10 \rceil = 4$.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 1 · Introducción al diseño con herramientas CAD/EDA

Crear un proyecto en Xilinx ISE, describir un circuito y probarlo en la placa Basys.

💡 Cómo se evalúa

Las prácticas pesan el **20 %**. Se evalúan con memorias e informes y, y, según se decida en el curso, con pruebas prácticas. Hay que **defender** el trabajo: explicar cómo funciona cada diseño y por qué tomaste cada decisión. Es obligatorio asistir (mínimo 80 %) y sacar **al menos un 4,0** en teoría y en prácticas para aprobar. Si usas IA para generar código (permitido en la Práctica 4), tienes que **entenderlo, revisarlo y justificarlo**.

Simulador de las prácticas

Elige una función, cambia los valores y mira el código VHDL equivalente, la salida tipo consola y el cronograma. Los botones «Abrir en el simulador» de cada práctica cargan aquí el ejemplo correspondiente.

SIMULADOR DE PRÁCTICAS · CÓDIGO EQUIVALENTE, SALIDA DE CONSOLA Y CRONOGRAMA

Elige la función

Sumador de 4 bits (Práctica 2)

a (0 – 15)

9

b (0 – 15)

8

cin (0 – 1)

0

Código VHDL equivalente

```
entity sum4 is
  port(a,b: in std_logic_vector(3 downto 0); cin: in std_logic;
       s: out std_logic_vector(3 downto 0); cout: out std_logic);
end sum4;
architecture rtl of sum4 is
  signal t: unsigned(4 downto 0);
begin
  t    <= ('0'&unsigned(a)) + unsigned(b) + ("0000"&cin);
  s    <= std_logic_vector(t(3 downto 0));
  cout <= t(4);
end rtl;
```

Salida de consola

```
a=1001  b=1000  cin=0
s=0001  cout=1   (9 + 8 + 0 = 17)
```

Captura del simulador interactivo. Pruébalo en la versión web.

Objetivo

Aprender el flujo completo con una función trivial: describir, simular, sintetizar, asignar pines y programar la placa.

Pasos

Práctica 1 Crear el proyecto y describir el circuito

Crea un proyecto en Xilinx Foundation/ISE para la FPGA de la placa Basys y describe un circuito que enlace interruptores con LEDs.

🧭 Antes de empezar: qué piden y qué teoría usas

Herramientas: entorno ISE, captura de esquemáticos, diagramas de estados y VHDL (los tres modos de entrada de diseño). Ya conoces `entity` / `architecture` del Tema 13.

1 Nuevo proyecto

En ISE: *File → New Project*. Elige el nombre, la familia de la FPGA y el dispositivo que indica la placa Basys (consulta el manual de tu placa). Indica lenguaje **VHDL** y simulador **ISim**.

2 Nueva fuente

Project → New Source → VHDL Module. Pega el código de abajo (o dibuja el esquemático equivalente con puertas AND, OR, XOR).

3 Comprueba la sintaxis

Haz doble clic en *Check Syntax*. Debe terminar sin errores.

✅ Resultado final

Un módulo `p1_leds` con 8 interruptores, 4 pulsadores y 8 LEDs, listo para simular.

vhdl · p1_leds.vhd

```vhdl
 -- Práctica 1: entradas y salidas de la placa Basys.
-- Cada interruptor enciende el LED de su mismo número.
library ieee;
use ieee.std_logic_1164.all;

entity p1_leds is
  port (
    sw  : in  std_logic_vector(7 downto 0);   -- interruptores
    btn : in  std_logic_vector(3 downto 0);   -- pulsadores
    led : out std_logic_vector(7 downto 0)    -- LEDs
```

```
  );
end p1_leds;

architecture flujo of p1_leds is
begin
  -- Los 4 LEDs bajos copian los interruptores; los 4 altos muestran un AND y un OR de ejemplo.
  led(3 downto 0) <= sw(3 downto 0);
  led(4) <= sw(0) and sw(1);          -- AND
  led(5) <= sw(0) or  sw(1);          -- OR
  led(6) <= sw(0) xor sw(1);          -- XOR
  led(7) <= btn(0);                   -- el pulsador 0 enciende el LED 7
end flujo;
```

Práctica 1 Simular, asignar pines y programar

Simula el módulo, asigna las señales a los pines de la placa y programa la FPGA.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> Flujo: simulación funcional → síntesis → implementación → asignación de pines (UCF) → fichero de configuración → programación.

1 Simulación

Crea un *test bench* o usa el simulador de ISE: pon `sw=00000011`. Espera `led=00110011` (AND=1, OR=1, XOR=0).

2 Asignación de pines

Crea un fichero de restricciones (UCF) que una cada señal con su pin físico. Ejemplo (**comprueba los pines con el manual de tu placa**):

3 Síntesis, implementación y bitstream

Synthesize → *Implement Design* → *Generate Programming File*. Si hay avisos de «latch» o de señales sin pin, léelos.

4 Programar

Con la placa conectada, ejecuta iMPACT y carga el fichero `.bit`. Mueve los interruptores y comprueba los LEDs.

✅ **Resultado final**

Los LEDs siguen los interruptores. Si no se enciende ninguno, lo primero que hay que mirar es la asignación de pines y que el fichero se haya cargado.

text · p1_basys.ucf

```
 # Restricciones de ejemplo. COMPRUEBA los pines con el manual de TU placa Basys antes de usarlos.
NET "sw<0>"  LOC = "P11";
NET "sw<1>"  LOC = "L3";
NET "sw<2>"  LOC = "K3";
NET "sw<3>"  LOC = "B4";
NET "led<0>" LOC = "M5";
NET "led<1>" LOC = "M11";
NET "led<2>" LOC = "P7";
NET "led<3>" LOC = "P6";
# (resto de pines igual: un NET por señal)
```

Salida esperada

text · Resultado del modelo Python (salida esperada en la placa)

```
 sw        btn0  led
00000000  0     00000000
00000001  0     01100001
00000011  0     00110011
00000010  1     11100010
00001111  1     10111111
```

Qué escribir en la memoria

- Un esquema del circuito y su tabla de verdad.
- Una captura de la simulación con $sw = 00000011$ y la explicación del valor de cada LED.
- Una frase sobre qué hace cada paso del flujo (síntesis, implementación, programación).
- Cuál fue el error más típico que cometiste (por ejemplo, pines mal asignados) y cómo lo resolviste.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 2 · Diseño e implementación de sistemas combinacionales

Sumador, comparador, multiplexor y display de 7 segmentos, mostrados en LEDs y displays.

Objetivo

Diseñar con esquemático y con VHDL los bloques combinacionales del Tema 5 y verlos funcionar en LEDs y displays.

Práctica 2 Sumador de 4 bits

✔ verificado

Describe un sumador de 4 bits con acarreo de entrada y salida. Muestra la suma en 4 LEDs y el acarreo en un quinto.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: sumador completo y en cascada (T5); aritmética con `unsigned` (T13).

1 Qué se pide

Entradas `a`, `b` (4 bits) y `cin`; salidas `s` (4 bits) y `cout`.

2 Idea

Para no perder el acarreo, se hace la suma en **5 bits**: se amplían `a` y `b` con un `'0'` por la izquierda. El bit más significativo es `cout`.

3 Código

Ver abajo.

✅ **Resultado final**

$s = (a + b + cin) \bmod 16$ y $cout = \lfloor (a + b + cin)/16 \rfloor$. Comprobado con las 512 combinaciones del modelo.

vhdl · p2_sum4.vhd

```vhdl
 -- Práctica 2: sumador de 4 bits con acarreo de entrada y de salida.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p2_sum4 is
  port (
    a, b : in  std_logic_vector(3 downto 0);
    cin  : in  std_logic;
    s    : out std_logic_vector(3 downto 0);
    cout : out std_logic
  );
end p2_sum4;

architecture rtl of p2_sum4 is
  signal t : unsigned(4 downto 0);        -- un bit más para el acarreo
begin
  t    <= ('0' & unsigned(a)) + ('0' & unsigned(b)) + ("0000" & cin);
  s    <= std_logic_vector(t(3 downto 0));
  cout <= t(4);
end rtl;
```

Práctica 2 Comparador y multiplexor

✔ verificado

Diseña un comparador de 4 bits con salidas $A > B$, $A = B$, $A < B$ y un multiplexor 4:1.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: comparador y multiplexor (T5); `when ... else` y `with ... select` (T13).

1 Comparador

Con `unsigned` el comparador es una línea por salida. **Exactamente una** de las tres salidas vale 1: es una buena comprobación.

2 Multiplexor

Un `with sel select` enumera las cuatro posibilidades de `sel`; la última usa `others` para cubrir los valores no binarios de `std_logic`.

✅ Resultado final

Comparador: una y solo una salida a 1. Mux: $y = d_{sel}$.

vhdl · p2_cmp4.vhd

```vhdl
-- Práctica 2: comparador de 4 bits.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p2_cmp4 is
  port (
    a, b       : in  std_logic_vector(3 downto 0);
    gt, eq, lt : out std_logic
  );
end p2_cmp4;

architecture rtl of p2_cmp4 is
begin
  gt <= '1' when unsigned(a) >  unsigned(b) else '0';
  eq <= '1' when unsigned(a) =  unsigned(b) else '0';
  lt <= '1' when unsigned(a) <  unsigned(b) else '0';
end rtl;
```

vhdl · p2_mux4.vhd

```vhdl
-- Práctica 2: multiplexor 4 a 1 (dos descripciones equivalentes).
library ieee;
use ieee.std_logic_1164.all;

entity p2_mux4 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    sel : in  std_logic_vector(1 downto 0);
    y   : out std_logic
  );
end p2_mux4;

architecture rtl of p2_mux4 is
begin
  with sel select
    y <= d(0) when "00",
         d(1) when "01",
         d(2) when "10",
         d(3) when others;
end rtl;
```

Práctica 2 Display de 7 segmentos

Muestra en un display de 7 segmentos de la Basys una cifra BCD recibida por los interruptores.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: decodificador BCD a 7 segmentos con indiferentes en 1010–1111 (T5); en la Basys los segmentos son **activos a nivel bajo**.

1 Tabla

Para cada cifra, qué segmentos (a a g) deben encenderse. En lógica positiva el 8 enciende los siete; en la Basys hay que **invertir**.

2 Código

Un `with d select` con las 10 cifras, y `"1111111"` (apagado) para el resto.

3 Activa un dígito

La Basys tiene 4 dígitos con ánodo común: para ver una cifra hay que **activar el ánodo** de ese dígito (a nivel bajo).

✅ Resultado final

Con `d=0111` el display muestra un 7: segmentos a, b, c a 0 (encendidos), el resto a 1.

vhdl · p2_seg7.vhd

```vhdl
 -- Práctica 2: decodificador BCD a 7 segmentos para la Basys (segmentos activos a nivel bajo).
-- seg = (a, b, c, d, e, f, g). Un 0 enciende el segmento.
library ieee;
use ieee.std_logic_1164.all;

entity p2_seg7 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    seg : out std_logic_vector(6 downto 0)
  );
end p2_seg7;

architecture rtl of p2_seg7 is
begin
  with d select
    seg <= "0000001" when "0000",  -- 0
           "1001111" when "0001",  -- 1
           "0010010" when "0010",  -- 2
           "0000110" when "0011",  -- 3
           "1001100" when "0100",  -- 4
           "0100100" when "0101",  -- 5
           "0100000" when "0110",  -- 6
           "0001111" when "0111",  -- 7
           "0000000" when "1000",  -- 8
           "0000100" when "1001",  -- 9
           "1111111" when others;  -- apagado (1010 a 1111 no son BCD)
end rtl;
```

Salida esperada

text · Resultado del modelo Python

```
 --- sumador de 4 bits ---
 9 +  8 + 0 = s 0001 ( 1), cout 1
15 +  1 + 0 = s 0000 ( 0), cout 1
 7 +  8 + 1 = s 0000 ( 0), cout 1
15 + 15 + 1 = s 1111 (15), cout 1
--- comparador ---
a= 5 b= 9 -> gt,eq,lt = (0, 0, 1)
a= 9 b= 9 -> gt,eq,lt = (0, 1, 0)
a=12 b= 3 -> gt,eq,lt = (1, 0, 0)
--- mux 4:1 (d=0110) ---
sel=00 -> y=0
sel=01 -> y=1
sel=10 -> y=1
sel=11 -> y=0
--- display (activo a 0, abcdefg) ---
 0 -> 0000001
 1 -> 1001111
 7 -> 0001111
 9 -> 0000100
12 -> 1111111
Comprobaciones exhaustivas: OK
```

Qué escribir en la memoria

- La tabla de verdad del decodificador y las ecuaciones simplificadas (con indiferentes).
- Una comparación del esquemático y del VHDL: cuántos recursos usa cada uno.
- Capturas de la simulación del sumador con un caso con acarreo ($15 + 1$).
- Por qué en el display se usa lógica negativa.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 3 · Diseño de sistemas secuenciales

Contadores, divisor de frecuencia, antirrebote y máquinas de estados con reloj.

Objetivo

Diseñar sistemas sincronizados con reloj: contadores, registros y divisores de frecuencia, máquinas de estados y tratamiento del rebote de los pulsadores (T8, T9, T10).

Práctica 3 **Contador módulo N** ✔ verificado

Diseña un contador síncrono de módulo N configurable, con reset asíncrono, habilitación y una salida de fin de cuenta.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: contador de módulo N, biestables $\lceil \log_2 N \rceil$ (T8); proceso síncrono con reset asíncrono (T13).

1 Estructura

Un único proceso con `clk` y `rst` en la lista de sensibilidad. El reset va **antes** que el flanco.

2 Módulo N

Si `cnt = N-1` vuelve a 0; si no, incrementa. El `generic N` hace el diseño reutilizable.

3 Fin de cuenta

`fin` vale 1 cuando `cnt = N-1` : sirve para encadenar contadores.

✅ **Resultado final**

Secuencia $0, 1, \ldots, N-1, 0, \ldots$. Para $N = 10$: 4 biestables.

vhdl · p3_contador.vhd

```vhdl
-- Práctica 3: contador síncrono de módulo N con reset asíncrono y habilitación.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p3_contador is
  generic ( N : positive := 10 );
  port (
    clk, rst, en : in  std_logic;
    q            : out std_logic_vector(3 downto 0);
    fin          : out std_logic                       -- vale 1 en el último estado
  );
end p3_contador;

architecture rtl of p3_contador is
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= (others => '0');
    elsif rising_edge(clk) then
      if en = '1' then
        if cnt = N - 1 then
          cnt <= (others => '0');
        else
          cnt <= cnt + 1;
        end if;
      end if;
    end if;
  end process;

  q   <= std_logic_vector(cnt);
  fin <= '1' when cnt = N - 1 else '0';
end rtl;
```

Práctica 3 Divisor de frecuencia

✔ verificado

Genera una señal de 1 Hz a partir del reloj de 50 MHz de la placa y úsala para hacer parpadear un LED.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: contador como divisor (T8); $DIV = f_{in}/f_{out}$.

1 Cálculo

$DIV = 50\,000\,000$. La salida conmuta cada $DIV/2 = 25\,000\,000$ ciclos: se cuenta de 0 a $24\,999\,999$ (25 bits).

2 Código

Contador con comparación y una señal `s` que se invierte al llegar al final.

3 Prueba

En simulación no esperes 25 millones de ciclos: baja `DIV` (por ejemplo, a 10) en el *test bench* y comprueba la forma de onda.

✅ Resultado final

LED parpadeando a 1 Hz (medio segundo encendido y medio apagado).

vhdl · p3_divisor.vhd

```vhdl
 -- Práctica 3: divisor de frecuencia. Con reloj de 50 MHz y DIV = 50_000_000 da una salida de 1 Hz.
library ieee;
use ieee.std_logic_1164.all;

entity p3_divisor is
  generic ( DIV : positive := 50_000_000 );     -- relación f_in / f_out (número par)
  port (
    clk, rst : in  std_logic;
    clk_out  : out std_logic
  );
end p3_divisor;

architecture rtl of p3_divisor is
  signal cnt : integer range 0 to DIV/2 - 1 := 0;
  signal s   : std_logic := '0';
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= 0; s <= '0';
    elsif rising_edge(clk) then
      if cnt = DIV/2 - 1 then
        cnt <= 0;
        s   <= not s;                           -- conmuta cada medio periodo
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
  clk_out <= s;
end rtl;
```

Práctica 3 Antirrebote de un pulsador

✔ verificado

Un pulsador mecánico rebota unos milisegundos. Diseña un circuito que entregue un nivel limpio.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: sincronizar la entrada con 2 biestables para evitar la metaestabilidad (T6) y aceptarla solo si permanece estable un tiempo (T10).

1 Sincroniza

`s1` y `s2` son dos biestables D en serie: la entrada asíncrona se alinea con el reloj.

2 Cuenta de estabilidad

Si la entrada cambia, reinicia el contador. Si llega a `LIM` sin cambiar, acepta el valor.

3 Calcula LIM

20 ms a 50 MHz son 10^6 ciclos: contador de 20 bits.

✅ Resultado final

Un solo pulso limpio por pulsación, aunque la señal original rebote.

vhdl · p3_antirrebote.vhd

```vhdl
 -- Práctica 3: antirrebote. La salida sigue a la entrada solo si esta permanece estable LIM ciclos.
-- Con reloj de 50 MHz y LIM = 1_000_000 el filtro espera 20 ms.
library ieee;
use ieee.std_logic_1164.all;

entity p3_antirrebote is
  generic ( LIM : positive := 1_000_000 );
  port (
    clk, btn : in  std_logic;
    limpio   : out std_logic
  );
end p3_antirrebote;

architecture rtl of p3_antirrebote is
  signal s1, s2 : std_logic := '0';          -- sincronizador de dos biestables
  signal prev   : std_logic := '0';
  signal cnt    : integer range 0 to LIM := 0;
  signal sal    : std_logic := '0';
```

```
begin
  process (clk)
  begin
    if rising_edge(clk) then
      s1 <= btn;  s2 <= s1;                  -- evita la metaestabilidad
      if s2 /= prev then                     -- la entrada ha cambiado: reinicia la cuenta
        prev <= s2;
        cnt  <= 0;
      elsif cnt = LIM then
        sal  <= prev;                        -- estable el tiempo suficiente: se acepta
      else
        cnt  <= cnt + 1;
      end if;
    end if;
  end process;
  limpio <= sal;
end rtl;
```

Práctica 3 ## Máquina de estados: detector de secuencia

✔ verificado

Implementa en VHDL el detector Mealy de la secuencia 101 del Tema 10.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: diagrama de estados de Mealy y su traducción a dos procesos.

1 Tipo enumerado

`type estado_t is (S0, S1, S2);` deja que la herramienta elija la codificación.

2 Dos procesos

El primero guarda el estado en el flanco. El segundo, combinacional, calcula `sig` y `z` con un `case`.

3 Evita latches

Asigna `z <= '0'` al principio del proceso combinacional: así `z` se asigna en **todos** los casos.

✅ Resultado final

Para $x = 1, 0, 1, 0, 1, 1, 0, 1$ da $z = 0, 0, 1, 0, 1, 0, 0, 1$.

vhdl · p3_detector101.vhd

```vhdl
 -- Práctica 3: máquina de estados. Detector Mealy de la secuencia 101 con solape.
library ieee;
use ieee.std_logic_1164.all;

entity p3_detector101 is
  port (
    clk, rst, x : in  std_logic;
    z           : out std_logic
  );
end p3_detector101;

architecture rtl of p3_detector101 is
  type estado_t is (S0, S1, S2);
  signal est, sig : estado_t := S0;
begin
  -- Proceso 1: registro de estado (parte secuencial)
  process (clk, rst)
  begin
    if rst = '1' then
      est <= S0;
    elsif rising_edge(clk) then
      est <= sig;
    end if;
  end process;

  -- Proceso 2: lógica de siguiente estado y de salida (parte combinacional)
  process (est, x)
  begin
    z <= '0';
    case est is
      when S0 =>
        if x = '1' then sig <= S1; else sig <= S0; end if;
      when S1 =>
        if x = '0' then sig <= S2; else sig <= S1; end if;
      when S2 =>
        if x = '1' then sig <= S1; z <= '1'; else sig <= S0; end if;
    end case;
  end process;
end rtl;
```

Salida esperada

text · Resultado del modelo Python

```
 contador mod 10: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2]
contador mod 12: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 0, 1]
divisor 50 MHz -> 1 Hz: DIV=50000000, cuenta hasta 24999999, 25 bits
divisor 50 MHz -> 1 kHz: DIV=50000, cuenta hasta 24999, 15 bits
entrada : 001010111111111010000000000
salida  : 000000000000011111111110000
detector 101: x = [1, 0, 1, 0, 1, 1, 0, 1]  z = [0, 0, 1, 0, 1, 0, 0, 1]
Comprobaciones: OK
```

Qué escribir en la memoria

- El diagrama de estados del detector y la tabla de transiciones.
- El cálculo de DIV y del número de bits del contador del divisor.
- Una captura de la señal del pulsador **con rebotes** y la salida del filtro.
- Una discusión de por qué el reset asíncrono puede dar problemas y cuándo conviene el síncrono.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 4 · Desarrollo de sistemas digitales mediante VHDL

Un sistema completo en VHDL: se permite usar IA, pero hay que entender, revisar y justificar el código.

⚠️ Sobre el uso de IA

Se permite usar IA como apoyo, pero el código debe ser **comprendido, revisado y justificado por ti**. En la defensa pueden pedirte que expliques una línea, que encuentres un error o que lo modifiques. La forma segura de prepararte: pide el código, **simúlalo**, tradúcelo a un diagrama y **busca los fallos típicos** (latches, reset, reloj dividido, señales no asignadas).

Objetivo

Construir un sistema completo combinando bloques: el ejemplo de la guía muestra cuatro cifras en el display multiplexado de la Basys.

Práctica 4 **Display de 4 dígitos multiplexado** ✔ verificado

Muestra cuatro cifras BCD en el display de 4 dígitos de la Basys. Calcula la frecuencia de refresco.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: multiplexación en el tiempo, contador de refresco, decodificador de 7 segmentos activo a nivel bajo, múltiples dígitos con ánodo común.

1 Idea

Los 4 dígitos comparten los 7 segmentos. Solo uno está activo cada vez. Se alternan tan rápido que el ojo los ve todos encendidos.

2 Contador de refresco

Un contador libre de 18 bits. Sus dos bits altos eligen el dígito: cada dígito dura 2^{16} ciclos.

3 Selección

Un `with sel select` elige la cifra y activa el ánodo correspondiente (activo a 0).

4 Frecuencias

Cambio de dígito: $50\,\text{MHz}/2^{16} = 762{,}9$ Hz. Refresco de cada dígito: $762{,}9/4 = 190{,}7$ Hz.

✅ Resultado final

Cuatro cifras estables, sin parpadeo visible, con refresco de $190{,}7$ Hz por dígito.

vhdl · p4_display4.vhd

```vhdl
 -- Práctica 4: manejo multiplexado de un display de 4 dígitos (Basys). Muestra 4 cifras BCD.
-- Se enciende un dígito cada vez. Con reloj de 50 MHz se cambia de dígito a 762,9 Hz (50e6/2^16)
-- y cada dígito se refresca a 190,7 Hz: el ojo los ve todos encendidos y sin parpadeo.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p4_display4 is
  port (
    clk    : in  std_logic;                       -- 50 MHz
    d3, d2, d1, d0 : in std_logic_vector(3 downto 0);
    seg    : out std_logic_vector(6 downto 0);    -- activos a nivel bajo
    an     : out std_logic_vector(3 downto 0)     -- ánodos, activos a nivel bajo
  );
end p4_display4;

architecture rtl of p4_display4 is
  signal cnt : unsigned(17 downto 0) := (others => '0');   -- 2^16 ciclos por dígito
  signal sel : std_logic_vector(1 downto 0);
  signal dig : std_logic_vector(3 downto 0);
begin
  process (clk)
  begin
    if rising_edge(clk) then cnt <= cnt + 1; end if;
  end process;
  sel <= std_logic_vector(cnt(17 downto 16));

  with sel select dig <= d0 when "00", d1 when "01", d2 when "10", d3 when others;
  with sel select an  <= "1110" when "00", "1101" when "01", "1011" when "10", "0111" when others;

  with dig select
    seg <= "0000001" when "0000", "1001111" when "0001", "0010010" when "0010",
           "0000110" when "0011", "1001100" when "0100", "0100100" when "0101",
           "0100000" when "0110", "0001111" when "0111", "0000000" when "1000",
           "0000100" when "1001", "1111111" when others;
end rtl;
```

text · Resultado del modelo Python

```
 cambio de dígito: 762.9 Hz; refresco de cada dígito: 190.7 Hz
barrido para mostrar 1 2 3 4:
  an=1110  cifra=4
  an=1101  cifra=3
  an=1011  cifra=2
  an=0111  cifra=1
Comprobaciones: OK
```

Qué escribir en la memoria

- Las prompts que usaste y qué cambiaste del código generado, y por qué.
- Un diagrama de bloques jerárquico del sistema.
- Los cálculos de frecuencias y tiempos.
- Una lista de **tres errores** que detectaste al revisar el código.

PRÁCTICAS (20 % DE LA NOTA)

Prácticas 5 a 7 · Proyecto integrador

Especificación, implementación e integración de un sistema distribuido que monitoriza una incubadora neonatal.

En qué consisten

Es un **proyecto colaborativo** basado en la monitorización de una incubadora neonatal (Dräger Caleo) usada como **entorno de simulación**.

Práctica	Qué se hace
P5	Especificación y diseño del proyecto integrador
P6	Cada grupo implementa y depura el **módulo asignado** del sistema distribuido, con pruebas individuales y documentación técnica
P7	Integración de todos los módulos, validación y **defensa pública** de la solución

> ⚠️ **Qué no sé de tu proyecto**
>
> El programa solo describe el proyecto en general. El módulo concreto que te asignen y los datos exactos de la incubadora se fijan en el curso. El ejemplo de abajo (una alarma de temperatura con histéresis) **no es tu módulo asignado**: está para mostrar cómo se diseña, se prueba y se documenta un módulo típico de un sistema de monitorización.

Método para cada módulo

1. **Especifica** entradas, salidas, rangos y qué debe hacer, con una tabla.
2. **Diseña** en bloques: qué es combinacional, qué es secuencial.
3. **Escribe** el código (VHDL) y **simula** con casos límite.
4. **Prueba** el módulo aislado en la placa.
5. **Documenta**: interfaz, funcionamiento, pruebas y resultados.
6. **Integra**: conecta con los demás módulos y comprueba temporización y niveles.

Ejemplo: alarma de temperatura con histéresis

Un sensor da la temperatura en décimas de grado. Se quiere una alarma de alta ($\geq 37{,}5$ °C) y una de baja ($< 36{,}0$ °C). Sin histéresis, una medida que oscile alrededor del umbral haría **parpadear** la alarma. Con histéresis, la alarma se **desactiva** solo cuando la medida baja del umbral menos un margen (0,5 °C).

Prácticas 5-7 **Alarma con histéresis**

Diseña un módulo `p6_alarma` que reciba `temp` (10 bits, décimas de grado) y active `al_alta` si $T \geq 375$ y `al_baja` si $T < 360$, con histéresis de 5 décimas.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: comparadores (T5), biestable que recuerda el estado de la alarma (T6), proceso síncrono (T13).

1 Qué hay que recordar

La alarma depende del **pasado** (si ya estaba activa o no) y no solo de la entrada: es un circuito **secuencial** con un biestable por alarma.

2 Reglas

Alta: se activa con $T \geq T_{ALTA}$ y se desactiva con $T < T_{ALTA} - H$. Baja: se activa con $T < T_{BAJA}$ y se desactiva con $T \geq T_{BAJA} + H$. Entre ambos valores **mantiene** el estado.

3 Código

Ver abajo.

4 Casos límite que hay que probar

$T = 374$ (no activa), 375 (activa), 373 (sigue activa por la histéresis), 369 (se desactiva), y los equivalentes para la baja.

✅ **Resultado final**

Para las medidas $365, 370, 374, 375, 376, 373, 371, 369, \ldots$ la alarma de alta se activa en 375

y se desactiva en 369.

vhdl · p6_alarma_incubadora.vhd

```vhdl
 -- Prácticas 5 a 7 (ejemplo de módulo): alarma de temperatura con histéresis para un sistema de
monitorización.
-- Entrada: temperatura en décimas de grado (0 a 1023). Salidas: alarma de alta y de baja.
-- La histéresis evita que la alarma parpadee cuando la medida oscila alrededor del umbral.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p6_alarma is
  generic (
    T_ALTA  : natural := 375;    -- 37,5 °C: se activa la alarma de alta
    T_BAJA  : natural := 360;    -- 36,0 °C: se activa la alarma de baja
    HIST    : natural := 5       -- 0,5 °C de histéresis
  );
  port (
    clk, rst : in  std_logic;
    temp     : in  std_logic_vector(9 downto 0);
    al_alta  : out std_logic;
    al_baja  : out std_logic
  );
end p6_alarma;

architecture rtl of p6_alarma is
  signal t : unsigned(9 downto 0);
  signal a, b : std_logic := '0';
begin
  t <= unsigned(temp);
  process (clk, rst)
  begin
    if rst = '1' then
      a <= '0'; b <= '0';
    elsif rising_edge(clk) then
      if    t >= T_ALTA        then a <= '1';
      elsif t <  T_ALTA - HIST then a <= '0';
      end if;
      if    t <  T_BAJA        then b <= '1';
      elsif t >= T_BAJA + HIST then b <= '0';
      end if;
    end if;
  end process;
  al_alta <= a;
  al_baja <= b;
end rtl;
```

text · Resultado del modelo Python

```
 temp(0,1 °C)  al_alta  al_baja
    365        0        0
    370        0        0
    374        0        0
    375        1        0
    376        1        0
```

```
   373        1        0
   371        1        0
   369        0        0
   366        0        0
   362        0        0
   359        0        1
   361        0        1
   364        0        1
   366        0        0
Comprobaciones: OK
```

Qué debe contener la documentación y la defensa

- Una **tabla de interfaz** (señales, ancho, sentido, significado).
- Un **diagrama de bloques** del módulo y de cómo encaja en el sistema.
- La **estrategia de pruebas** (casos normales y límite) y los resultados.
- La **justificación** de cada decisión de diseño (por qué histéresis, por qué ese ancho de bits).
- Si usaste IA: qué pediste, qué cambiaste y cómo comprobaste que es correcto.

🚧 Qué suele fallar

- Integrar sin haber probado cada módulo por separado.
- Interfaces mal acordadas (anchos, polaridad activo-alto/bajo, qué unidad lleva cada dato).
- No documentar las suposiciones.
- Defender código que no entiendes.

EJERCICIOS RESUELTOS

Exámenes resueltos

Tres entradas: empieza por aquí, practica por tema o busca un examen concreto. Todos los problemas están resueltos paso a paso y verificados.

Antes de empezar

Cada problema tiene el **enunciado completo**, una caja «Antes de empezar: qué piden y qué teoría usas» y los pasos numerados. Los pasos están plegados: intenta resolver el problema tú primero y abre un paso solo cuando te atasques. El resultado final también está plegado.

Tipo de problema	Problemas	Peso	Aparece en
Bloques MSI y displays	6 de 11		3 de 3 exámenes
Autómatas / síntesis secuencial	5 de 11		3 de 3 exámenes
Aritmética y códigos (BCD, C2)	4 de 11		3 de 3 exámenes
Combinacional (puertas, Karnaugh)	3 de 11		3 de 3 exámenes
Contadores y biestables	2 de 11		1 de 3 exámenes

Recuento propio sobre los 11 problemas de los 3 exámenes resueltos aquí (un problema puede tener varias etiquetas). Son pocos exámenes: úsalo como orientación, no como regla.

⚠️ **Procedencia**

Los exámenes son de la misma asignatura en otra titulación de la misma escuela (véase examen). Al final de esta página se dice qué problemas no están incluidos y por qué.

1 · Empieza por aquí

Los exámenes completos más recientes y más representativos. Haz uno entero en 2 horas, sin mirar la solución.

Febrero 2025 (convocatoria del 23/01/2025)

4 problemas: aritmética con display, decodificador, expendedora (Mealy) y noria

Febrero 2023 (convocatoria del 22/01/2023)

Problemas 1, 2 y 4: sensores con signo, toldo y caja fuerte. El problema 3 (contador de la

(Moore).
4 problemas

figura) no se resuelve aquí.
3 problemas

Diciembre (convocatoria sin año en el enunciado)
4 problemas: recinto con comparadores, media de cuatro números BCD, contador JK+NAND y alarma de ventanas.
4 problemas

2 · Practicar por tema

Elige el tipo de problema que más te cuesta. Verás las listas de problemas de exámenes y de las hojas, con su origen.

Combinacional (puertas, Karnaugh) (13)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Circuito de control de un toldo · Feb 2023 · P2
- P2 · Decodificador 4:16 con decodificadores 2:4 · Feb 2025 · P2
- K1 · Mayoría de tres entradas · Hoja Karnaugh · K1
- K2 · Divisor de dos números de 2 bits · Hoja Karnaugh · K2
- K4 · Multiplicador de 2 × 2 bits · Hoja Karnaugh · K4
- K5 · Cuatro máquinas en serie: estado de la producción · Hoja Karnaugh · K5
- K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos) · Hoja Karnaugh · K6
- K7 · Decodificador BCD a 7 segmentos · Hoja Karnaugh · K7
- K8 · Comparador de dos números de 2 bits · Hoja Karnaugh · K8
- K9 · Máquina de monedas con cambio · Hoja Karnaugh · K9
- K10 · Nota de 0 a 10 en un display de barras · Hoja Karnaugh · K10
- K17 · Binario de 4 bits a dos dígitos BCD · Hoja Karnaugh · K17

Bloques MSI y displays (10)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Media de cuatro números de dos cifras en BCD · Diciembre · P2
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- P1 · Dos sensores en C2: diferencia y media · Feb 2023 · P1
- P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display · Feb 2025 · P1

- P2 · Decodificador 4:16 con decodificadores 2:4 · Feb 2025 · P2
- K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos) · Hoja Karnaugh · K6
- K7 · Decodificador BCD a 7 segmentos · Hoja Karnaugh · K7
- K8 · Comparador de dos números de 2 bits · Hoja Karnaugh · K8
- K10 · Nota de 0 a 10 en un display de barras · Hoja Karnaugh · K10

Aritmética y códigos (BCD, C2) (8)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Media de cuatro números de dos cifras en BCD · Diciembre · P2
- P1 · Dos sensores en C2: diferencia y media · Feb 2023 · P1
- P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display · Feb 2025 · P1
- A9 · Sumador serie · Hoja autómatas · A9
- K2 · Divisor de dos números de 2 bits · Hoja Karnaugh · K2
- K4 · Multiplicador de 2 × 2 bits · Hoja Karnaugh · K4
- K17 · Binario de 4 bits a dos dígitos BCD · Hoja Karnaugh · K17

Autómatas / síntesis secuencial (13)

- P3 · Contador síncrono reversible con JK y NAND · Diciembre · P3
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- P4 · Caja fuerte: secuencia A, A, B, A (Mealy con JK) · Feb 2023 · P4
- P3 · Expendedora de tabaco (Mealy, biestables T y JK) · Feb 2025 · P3
- P4 · Noria con dos motores (Moore, biestables JK) · Feb 2025 · P4
- A1 · Tres o más unos seguidos · Hoja autómatas · A1
- A2 · Tres o más ceros seguidos · Hoja autómatas · A2
- A3 · Detector de la secuencia 1011 (con solapamiento) · Hoja autómatas · A3
- A5a · Detector de cuatro unos (1111) · Hoja autómatas · A5a
- A5b · Detector de la secuencia 11011 · Hoja autómatas · A5b
- A7 · Detector de 1010 o de 110 · Hoja autómatas · A7
- A9 · Sumador serie · Hoja autómatas · A9
- A11 · Contador reversible módulo 4 · Hoja autómatas · A11

Contadores y biestables (10)

- P3 · Contador síncrono reversible con JK y NAND · Diciembre · P3
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- A1 · Tres o más unos seguidos · Hoja autómatas · A1
- A2 · Tres o más ceros seguidos · Hoja autómatas · A2

- A3 · Detector de la secuencia 1011 (con solapamiento) · Hoja autómatas · A3
- A5a · Detector de cuatro unos (1111) · Hoja autómatas · A5a
- A5b · Detector de la secuencia 11011 · Hoja autómatas · A5b
- A7 · Detector de 1010 o de 110 · Hoja autómatas · A7
- A9 · Sumador serie · Hoja autómatas · A9
- A11 · Contador reversible módulo 4 · Hoja autómatas · A11

3 · Buscar un examen

Filtra por tipo de problema o escribe una palabra. Los exámenes están plegados y ordenados por curso.

Sin año

Diciembre (convocatoria sin año en el enunciado)

4 problemas: recinto con comparadores, media de cuatro números BCD, contador JK+NAND y alarma de ventanas.

Diciembre · P1

P1 · Recinto con dos puertas (comparadores y sumadores)

✔ verificado

Para entrar en un recinto hay que pasar por dos puertas, P1 y P2. Para entrar por P1 hay que introducir un dígito BCD mayor o igual a 6 y para pasar por la puerta P2 hay que introducir un número que esté tres unidades por arriba o por debajo de la mitad del número introducido en la puerta P1. Por ejemplo, si en la puerta P1 introducimos el 8, entramos por P1 y deberemos introducir en P2 un número entre el 1 $[(8/2) - 3]$ y el 7 $[(8/2) + 3]$. Diseñar el circuito lógico correspondiente.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un circuito combinacional con dos entradas BCD (P_1, P_2) y dos salidas «abre P1» y «abre P2». **Teoría:** comparadores y sumadores (T5), dividir entre 2 como desplazamiento (T2), C2 para el -3 (T2).

1 **Abrir P1**

$P_1 \geq 6$ para $P_1 \in \{0, \ldots, 9\}$ es $P_1 > 5$. Se usa un comparador de 4 bits con $B = 0101$ y la salida $A > B$.

2 La mitad de P_1

Dividir entre 2 es **desplazar a la derecha**: se toman los bits $P_1(3{:}1)$ y se completan con un 0 en el MSB. El bit menos significativo se pierde (mitad entera): $9/2 \rightarrow 4$.

3 Los límites $h+3$ y $h-3$

Con $h = P_1/2$ (3 bits, ampliado a 4): un sumador de 4 bits con la constante 0011 da $h+3$. Otro con la constante 1101 (que es -3 en C2 de 4 bits) da $h-3$. Como $P_1 \geq 6$ implica $h \geq 3$, el resultado $h-3$ nunca es negativo y se puede comparar **sin signo**. Se ha comprobado con las 40 combinaciones válidas de (P_1, P_2).

4 Abrir P2

Dos comparadores de 4 bits: $P_2 \leq h+3$ (salidas $A<B$ o $A=B$ unidas con una OR) y $P_2 \geq h-3$ ($A>B$ o $A=B$). «Abre P2» es la AND de ambos con «abre P1».

5 Tabla de valores permitidos

P_1	$h = P_1/2$	P_2 permitido
6	3	0 a 6
7	3	0 a 6
8	4	1 a 7
9	4	1 a 7

El caso $P_1 = 8$ coincide con el ejemplo del enunciado (1 a 7).

✅ Resultado final

Abre P1 $= (P_1 > 0101)$; Abre P2 $=$ Abre P1 $\cdot (P_2 \leq h+3) \cdot (P_2 \geq h-3)$ con $h = P_1(3{:}1)$. 2 comparadores de 4 bits para P2, 1 para P1 y 2 sumadores de 4 bits.

Diciembre · P2 — P2 · Media de cuatro números de dos cifras en BCD

✔ verificado

Realice el diseño de un circuito que haga la media de cuatro números de dos cifras dado en BCD.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar los cuatro números de BCD a binario, sumarlos, dividir entre 4 y presentar el resultado. **Teoría:** BCD ↔ binario (T2), sumadores en cascada (T5), dividir entre 2^k como desplazamiento.

1 De BCD a binario

Un número de dos cifras $d_1 d_0$ vale $10d_1 + d_0 = 8d_1 + 2d_1 + d_0$. Multiplicar por 8 y por 2 son **desplazamientos** (cableado), así que basta un sumador: $(d_1 \ll 3) + (d_1 \ll 1) + d_0$. Cada número sale en binario de 7 bits (máximo $99 < 128$).

2 Sumar los cuatro

La suma máxima es $4 \cdot 99 = 396$. Como $2^8 = 256 < 396 < 512 = 2^9$, hacen falta **9 bits**. Se suman con tres sumadores (o dos niveles: $A + B$, $C + D$, y la suma de ambos).

3 Dividir entre 4

Dividir entre 4 es **descartar los dos bits menos significativos** de la suma de 9 bits. El resultado tiene 7 bits ($396/4 = 99$). Es la mitad entera de la mitad entera, calculada **una sola vez**: $\lfloor (A + B + C + D)/4 \rfloor$.

4 Presentar en BCD

Para mostrar la media en dos displays hay que volver de binario a BCD (decenas y unidades). Se hace con comparadores y restas de 10, o con una ROM de 128 posiciones (ver T7).

🛠️ Ojo: la cascada de medias pierde precisión

Una solución típica hace $((A + B)/2 + (C + D)/2)/2$ con dos desplazamientos. **No es equivalente**: cada desplazamiento descarta un bit que el siguiente necesitaba. Con $A = 0, B = 1, C = 1, D = 2$ la media entera correcta es $\lfloor 4/4 \rfloor = 1$, pero la cascada da $(0 + 1) \gg 1 = 0$. Sobre las $199 \times 199 = 39\,601$ combinaciones de $(A + B,\ C + D)$, la cascada falla en $4\,900$ (un 12.4 %), con error máximo de 1 unidad. Para una media exacta: suma primero **todo** y desplaza **una vez**.

✅ Resultado final

BCD→binario con $8d_1 + 2d_1 + d_0$; suma de los cuatro en 9 bits; media = suma$(8{:}2)$ (7 bits).

La versión en cascada de desplazamientos es incorrecta en 12.4 % de los casos.

Diciembre · P3

P3 · Contador síncrono reversible con JK y NAND

✔ verificado

Utilizando biestables J-K y puertas NAND, diseñar un contador síncrono que posea una entrada de control X y que cumpla: (a) si $X = 0$, el contador funciona en modo ascendente con la secuencia $0 \to 2 \to 4 \to 6 \to 1 \to 3 \to 5 \to 7\ [\to 0 \ldots]$; (b) si $X = 1$, funciona en modo descendente, siendo la secuencia de conteo $0 \to 7 \to 5 \to 3 \to 1 \to 6 \to 4 \to 2\ [\to 0 \ldots]$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador de 3 bits con una secuencia no binaria y dos sentidos, con JK y solo NAND. **Teoría:** diseño de contadores como autómata (T8, T10); tabla de excitación del JK (T6); NAND universal (T3).

1 Estados y entradas

8 estados (000 a 111), 3 biestables, una entrada X. Cada estado tiene dos sucesores: el siguiente en la lista (si $X = 0$) o el anterior (si $X = 1$). Es la misma secuencia recorrida en uno u otro sentido. La descendente enunciada es justo la ascendente al revés: $0, 7, 5, 3, 1, 6, 4, 2$ ✓.

2 Tabla de transiciones

$Q_2Q_1Q_0$	$X = 0$	$X = 1$
000	010	111
001	011	110
010	100	000
011	101	001
100	110	010
101	111	011
110	001	100
111	000	101

3 Ecuaciones con la tabla de excitación del JK

Se rellena J y K de cada biestable según $Q \to Q^+$ y se simplifica con Karnaugh de 4 variables (Q_2, Q_1, Q_0, X):

$$J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X} \qquad J_1 = K_1 = 1$$

$$J_0 = K_0 = \overline{Q_2 Q_1}X + Q_2 Q_1 \overline{X}$$

Intuición: Q_1 conmuta en cada ciclo (en ambas secuencias alterna 0,1,0,1...), por eso $J_1 = K_1 = 1$. Q_2 conmuta cuando $Q_1 \neq X$.

4 Pasarlo a NAND

Cada suma de productos se implementa en **dos niveles de NAND** (NAND-NAND). Para $J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X}$: $J_2 = \mathrm{NAND}\big(\mathrm{NAND}(\overline{Q_1}, X),\ \mathrm{NAND}(Q_1, \overline{X})\big)$. Las variables negadas salen de la salida $\overline{Q}$ de los biestables y de una NAND con las dos entradas unidas para $\overline{X}$. $J_1 = K_1$ se conectan a nivel alto.

5 Comprobación

Las ecuaciones minimizadas, simuladas a nivel de puerta, reproducen la tabla en los 16 casos (Q, X) y generan las secuencias ascendente [0, 2, 4, 6, 1, 3, 5, 7] y descendente [0, 7, 5, 3, 1, 6, 4, 2].

✅ Resultado final

$J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X}$; $J_1 = K_1 = 1$; $J_0 = K_0 = \overline{Q_2 Q_1}X + Q_2 Q_1 \overline{X}$. Todo con NAND en dos niveles.

Diciembre · P4

P4 · Alarma de ventanas con barrido secuencial

✔ verificado

Un edificio de cuatro plantas tiene cuatro ventanas por planta. En cada ventana se coloca un sensor que detecta la apertura de la misma (0 cerrada, 1 abierta). Realice el diseño de un sistema de alarma que funcione así: dos displays (planta y número de ventana vigilada); un LED junto a los displays que se enciende si la ventana indicada está abierta; barrido secuencial cíclico, desde planta 1-ventana 1 hasta planta 4-ventana 4, de frecuencia 1 o 2 segundos seleccionable con un interruptor; en el momento en que se encuentre una ventana abierta el sistema se detendrá hasta que la ventana en cuestión se cierre.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador que recorra las 16 ventanas, un selector que mire el sensor de la ventana actual y que **congele** el contador si está abierta. **Teoría:** contador binario (T8), multiplexor 16:1 (T5), sumador para mostrar «1 a 4» en lugar de «0 a 3», decodificador BCD a 7 segmentos (T5).

1 Contador de ventanas

Un contador binario de 4 bits $Q_3Q_2Q_1Q_0$. Los dos bits altos Q_3Q_2 son la **planta** (0 a 3) y los dos bajos Q_1Q_0 la **ventana** (0 a 3). Cuenta 16 estados y vuelve a 0: barrido cíclico.

2 Mirar el sensor de la ventana actual

Un multiplexor 16:1 con los 16 sensores en las entradas de datos y $Q_3Q_2Q_1Q_0$ en la selección da la señal $ABIERTA$ del sensor vigilado. Esa misma señal enciende el LED.

3 Detener el barrido

El contador **solo avanza si** $ABIERTA = 0$: se usa $ABIERTA$ negada como habilitación del contador (o se hace AND de la señal de reloj con $\overline{ABIERTA}$). Mientras la ventana siga abierta, la selección no cambia y el LED sigue encendido. En cuanto se cierra, el barrido continúa. Mejor usar una **habilitación** que cortar el reloj: así no se generan pulsos cortos.

4 Frecuencia seleccionable

Se parte de un reloj de 1 Hz (periodo 1 s). Un biestable T con $T = 1$ lo divide por 2 (periodo 2 s). Un multiplexor 2:1 controlado por el interruptor elige uno de los dos relojes.

5 Displays

Planta $= Q_3Q_2 + 1$ y ventana $= Q_1Q_0 + 1$ (valores de 1 a 4): un sumador de 2 bits con la constante 01 para cada uno, y un decodificador BCD a 7 segmentos para cada display.

6 Comprobación

Con la ventana 5 (planta 2, ventana 2) abierta durante 10 ciclos, el contador se queda en ella los 10 ciclos y luego pasa a la siguiente. Los valores mostrados están siempre entre 1 y 4.

✅ **Resultado final**

Contador de 4 bits (planta Q_3Q_2, ventana Q_1Q_0) + MUX 16:1 + habilitación $\overline{ABIERTA}$ + selector de reloj (1 Hz o 0,5 Hz) + dos sumadores «+1» y dos decodificadores 7 segmentos.

2024-25

Febrero 2025 (convocatoria del 23/01/2025)

4 problemas: aritmética con display, decodificador, expendedora (Mealy) y noria (Moore).

Feb 2025 · P1 **P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display**

✔ verificado

Realice el diseño de un circuito que realice la función $(A \cdot B) + C$, siendo A, B y C números de 2 bits de valor más 1 de signo, es decir, si el bit de signo es 1 el número es negativo y está representado en complemento a 2. El resultado deberá visualizarse en los displays de 7 segmentos que consideres necesarios.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un circuito combinacional con entradas de 3 bits en C2 (-4 a 3) que multiplique, sume y muestre el resultado con signo en decimal. **Teoría que usas:** C2 y su rango (T2), sumadores y multiplicadores (T5), BCD y display de 7 segmentos (T5). Idea clave: multiplicar **valores absolutos** (más fácil: sin signo) y corregir el signo al final.

1 Rango del resultado (decide cuántos bits y cuántos displays)

$A, B \in [-4, 3]$, luego $A \cdot B \in [-12, 16]$ y $A \cdot B + C \in [-16, 19]$. El módulo llega a 19: **dos dígitos** (decenas 0 o 1, y unidades) más un indicador de signo. Cabe en 6 bits en C2 ($[-32, 31]$). Rango comprobado por código.

2 Valor absoluto de A y de B

Con s el bit de signo: $|X| = X \oplus s$ (XOR de cada bit con s) **más** s (acarreo de entrada de un sumador). Es el «invertir y sumar 1» del C2, que solo se aplica si es negativo. Resultado de 3 bits sin signo (el $-4 = 100$ da $|-4| = 4 = 100$).

3 Producto sin signo

Un multiplicador 3×3 de AND y sumadores da $P = |A| \cdot |B| \leq 4 \cdot 4 = 16$: **5 bits**.

4 Signo del producto y paso a C2

El producto es negativo si los signos son distintos: $s_P = s_A \oplus s_B$. Se amplía P a 6 bits y se convierte a C2 con el mismo truco XOR + s_P como acarreo.

5 Sumar C

Se extiende el signo de C a 6 bits (se repite su MSB) y se suma con un sumador de 6 bits: $R = P_{C2} + C$.

6 Presentar con signo y en decimal

El signo de R es su bit 5: va a un segmento «−» (o a un LED). El módulo $M = |R|$ se obtiene otra vez con XOR + s_R. Para pasar a BCD: **decenas** $= 1$ si $M \geq 10$ (comparador); **unidades** $= M - 10$ si hay decenas, es decir $M + 6$ quedándose con 4 bits. Dos decodificadores BCD a 7 segmentos (T5).

7 Comprobación

El diseño se ha simulado con las $8^3 = 512$ combinaciones y siempre coincide con $A \cdot B + C$ en signo, decenas y unidades.

✅ **Resultado final**

Valor absoluto (XOR + sumador) → multiplicador 3×3 → signo $s_A \oplus s_B$ y paso a C2 en 6 bits → suma de C extendido → signo y módulo → BCD (decenas $= M \geq 10$, unidades $= M + 6$ módulo 16) → 2 displays y un segmento de signo. Rango $[-16, 19]$.

Feb 2025 · P2

P2 · Decodificador 4:16 con decodificadores 2:4

✔ verificado

Realizar un decodificador de 4 entradas y 16 salidas a partir de decodificadores de 2 entradas y 4 salidas (con entrada de habilitación E activa a 1; si $E = 0$ todas las salidas valen 0).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: construir un 4:16 con piezas 2:4. **Teoría:** cada salida de un decodificador es un minitérmino; la entrada E permite **habilitar** un decodificador sin tocar los demás (T5).

1 Cuántas piezas hacen falta

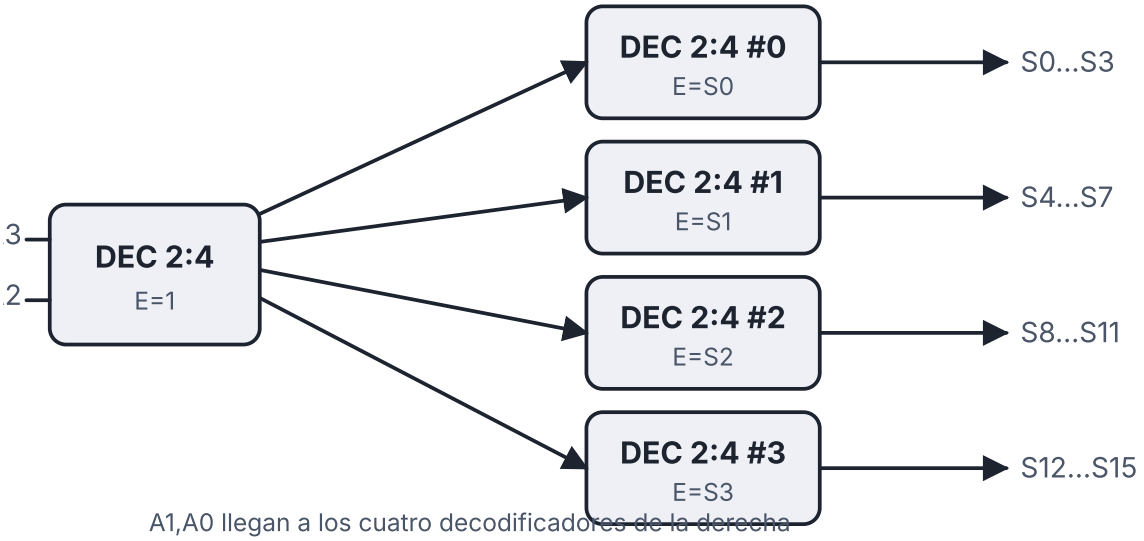
Son 16 salidas y cada 2:4 da 4: $16/4 = 4$ decodificadores para las salidas, más **1** para decidir cuál de ellos trabaja: 5 en total.

2 Reparto de entradas

Los bits altos $A_3 A_2$ entran en el decodificador de arriba (con $E = 1$ siempre). Sus cuatro salidas $S_0 \ldots S_3$ van a la entrada E de cada uno de los cuatro decodificadores de abajo. Los bits bajos $A_1 A_0$ entran en **los cuatro** a la vez.

3 Por qué funciona

Para $A = 1011_2 = 11$: arriba $A_3 A_2 = 10$ activa S_2, que habilita al tercer decodificador; este recibe $A_1 A_0 = 11$ y activa su salida 3. La salida activa es $4 \cdot 2 + 3 = 11$ ✓.



Esquema de la solución.

✅ **Resultado final**

Cinco decodificadores 2:4. Salida $4k + j$ activa $\iff$ $A_3A_2 = k$ y $A_1A_0 = j$. Verificado para las 16 entradas.

Feb 2025 · P3

P3 · Expendedora de tabaco (Mealy, biestables T y JK)

✔ verificado

Una máquina expendedora de tabaco está controlada por un circuito secuencial. Recibe como entradas el código de dos bits de las monedas que se introducen (M_1M_0: 00 no hay moneda, 01 moneda de 50 cts, 10 moneda de 1 €, 11 error) y activa dos señales: si se debe entregar el tabaco (T) y la cantidad de cambio a reintegrar (C_0: 00 no cambio y no tabaco; 01 no cambio y da tabaco; 10 no procede; 11 da cambio y da tabaco). El usuario introduce monedas hasta alcanzar 2 €, que es el precio. Al alcanzar o superar el importe, la máquina sirve el tabaco y devuelve el cambio. Diseña el circuito (Mealy o Moore) indicando: número y codificación de estados, diagrama de estados, tabla de transiciones y salidas con un biestable T para el bit más significativo del estado y JK para el resto, e implementación con mapas de Karnaugh.

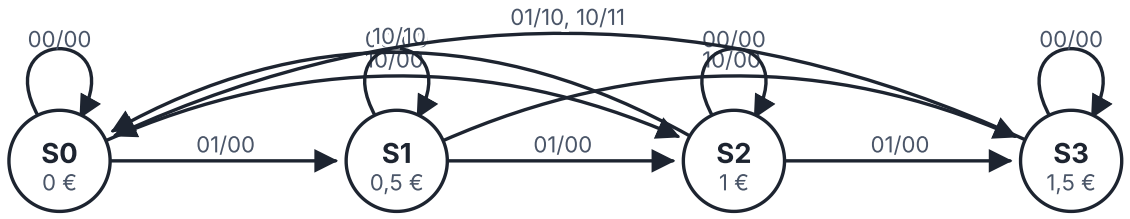
🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que acumule dinero y dispare T (y C_0) al llegar a 2 €. **Teoría:** método de diseño de autómatas (T10): estados → diagrama → tabla → tabla de excitación de T y JK (T6) → Karnaugh (T3).

> ⚠️ **Ojo con el enunciado**
>
> El ejemplo del enunciado habla de «monedas de 25», pero la Tabla 1 solo tiene monedas de **50 cts y 1 €**. Aquí se resuelve con esas dos monedas: el cambio posible es siempre una moneda de 50 cts (si el total llega a 2,5 €). El código 11 de entrada (error) se trata como «sin moneda».

1 Estados

Lo único que hay que recordar es **cuánto dinero hay**: 0 €, 0,5 €, 1 € o 1,5 € (al llegar a 2 € se sirve y se vuelve a 0). **4 estados**, 2 biestables: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$ (el código es el dinero en medios euros). Es Mealy: la salida T, C_0 depende del estado **y de la moneda** que cae.



Etiquetas: M1M0 / T C0

Al llegar a 2 € (T=1) se vuelve a S0. Con 1,5 € y moneda de 1 € se pasa a 2,5 € y se devuelve 50 cts (C0=1).

2 Tabla de transiciones

Q_1Q_0	M_1M_0	$Q_1^+Q_0^+$	T	C_0
00	00/11	00	0	0
00	01	01	0	0
00	10	10	0	0
01	00/11	01	0	0
01	01	10	0	0

Q_1Q_0	M_1M_0	$Q_1^+Q_0^+$	T	C_0
01	10	11	0	0
10	00/11	10	0	0
10	01	11	0	0
10	10	00	1	0
11	00/11	11	0	0
11	01	00	1	0
11	10	00	1	1

3 Tabla de excitación

Para el biestable T del bit alto: $T_1 = Q_1 \oplus Q_1^+$ (1 si cambia). Para el JK del bit bajo se usa la tabla de excitación de T6: $0 \to 0$: $J = 0, K = X$; $0 \to 1$: $J = 1, K = X$; $1 \to 0$: $J = X, K = 1$; $1 \to 1$: $J = X, K = 0$.

4 Ecuaciones (Karnaugh de 4 variables Q_1, Q_0, M_1, M_0)

$$T_1 = M_1\overline{M_0} + Q_0\overline{M_1}M_0$$

$$J_0 = \overline{M_1}M_0 \qquad K_0 = \overline{M_1}M_0 + Q_1M_1\overline{M_0}$$

$$T = Q_1M_1\overline{M_0} + Q_1Q_0\overline{M_1}M_0 \qquad C_0 = Q_1Q_0M_1\overline{M_0}$$

Se simplifica cada una con un mapa de Karnaugh de 4 variables, con las filas $M_1M_0 = 11$ tratadas como moneda inexistente.

5 Comprobación con el ejemplo

Tres monedas: 50 cts, 50 cts, 1 €. Estados: $S_1 \to S_2 \to S_0$; en la última, $T = 1$, $C_0 = 0$ (2 € justos, sin cambio). **Cuatro monedas: 50, 50, 50, 1 €.** $S_1 \to S_2 \to S_3 \to S_0$ y en la última, $T = 1$ y $C_0 = 1$ (2,5 € ⇒ 50 cts de cambio).

✅ **Resultado final**

4 estados (dinero acumulado), 2 biestables. $T_1 = M_1\overline{M_0} + Q_0\overline{M_1}M_0$; $J_0 = \overline{M_1}M_0$, $K_0 = \overline{M_1}M_0 + Q_1M_1\overline{M_0}$; $T = Q_1M_1\overline{M_0} + Q_1Q_0\overline{M_1}M_0$; $C_0 = Q_1Q_0M_1\overline{M_0}$. Verificado con todas las secuencias de hasta 7 monedas contra una especificación aritmética independiente.

Feb 2025 · P4

P4 · Noria con dos motores (Moore, biestables JK)

✔ verificado

El mecanismo de una noria de un parque de atracciones dispone de dos motores que permiten que la noria gire en un sentido o en el inverso. Estos dos motores se gobiernan por medio de dos señales (M_A y M_B) de tal manera que cuando una de ellas se pone a «1» se pone en funcionamiento el motor correspondiente. El empleado de controlar la atracción dispone de dos pulsadores (A y B), de manera que al pulsar uno de ellos se acciona el motor correspondiente. Si se acciona dos veces seguidas el mismo pulsador, la noria se detiene. Además, en el puesto de control central se dispone de otro pulsador (P) que en caso de ser accionado detiene la noria. (Los pulsadores están normalmente a nivel bajo, pasando a nivel alto cuando se accionan.) Diseña el autómata.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que recuerde qué motor está en marcha y si el último pulsador fue el mismo. **Teoría:** Moore (salida solo del estado) y diseño con JK (T10, T6). Hay que distinguir «sigo con el dedo en el botón» de «he vuelto a pulsar».

⚠️ **Suposiciones que hay que declarar**

- A y B no se pulsan a la vez ($AB = 11$ no ocurre: indiferente).
- P tiene prioridad sobre todo: si $P = 1$ se va a parada.
- «Dos veces seguidas el mismo pulsador» = soltar y volver a pulsar el mismo. Mantener pulsado **no** cuenta como dos pulsaciones.
- Pulsar el **otro** botón cambia el sentido sin pararse.

1 Estados (6) y codificación

Estado	Código $Q_2Q_1Q_0$	Significado	M_A	M_B
S_0	000	Parada	0	0

Estado	Código $Q_2Q_1Q_0$	Significado	M_A	M_B
S_1	001	Gira A, botón A aún pulsado	1	0
S_2	010	Gira A, botón soltado (otra pulsación de A parará)	1	0
S_3	011	Gira B, botón B aún pulsado	0	1
S_4	100	Gira B, botón soltado	0	1
S_5	101	Parada, esperando a soltar el botón	0	0

Los códigos 110 y 111 no se usan (indiferentes). Necesitas 3 biestables: $2^2 < 6 \leq 2^3$.

2 Transiciones (entradas A, B, P)

- S_0: $P \to S_0$; $A \to S_1$; $B \to S_3$; nada $\to S_0$.
- S_1: $P \to S_0$; $A \to S_1$ (sigue pulsado); $B \to S_3$; nada $\to S_2$.
- S_2: $P \to S_0$; $A \to S_5$ (segunda pulsación de A: para); $B \to S_3$; nada $\to S_2$.
- S_3: $P \to S_0$; $B \to S_3$; $A \to S_1$; nada $\to S_4$.
- S_4: $P \to S_0$; $B \to S_5$ (para); $A \to S_1$; nada $\to S_4$.
- S_5: si nada y $P = 0 \to S_0$; si no, se queda en S_5 (evita que mantener el botón rearranque).

3 Ecuaciones de entrada de los JK (Karnaugh de 6 variables)

$$J_2 = Q_1\overline{Q_0}A\overline{P} + Q_1Q_0\overline{ABP} \qquad K_2 = \overline{Q_0}P + \overline{Q_0}A + Q_0\overline{ABP}$$

$$J_1 = \overline{Q_2}B\overline{P} + \overline{Q_2}Q_0\overline{AP} \qquad K_1 = P + A + Q_0\overline{B}$$

$$J_0 = B\overline{P} + A\overline{P} \qquad K_0 = \overline{ABP} + \overline{Q_2}P$$

Cada una sale de aplicar la tabla de excitación del JK a las 6 variables Q_2, Q_1, Q_0, A, B, P con indiferentes en los estados 6 y 7 y en $AB = 11$.

4 Salidas (Moore)

$$M_A = \overline{Q_2 Q_1} Q_0 + Q_1 \overline{Q_0} \qquad M_B = Q_1 Q_0 + Q_2 \overline{Q_0}$$

Dependen solo de Q: $M_A = 1$ en S_1, S_2 y $M_B = 1$ en S_3, S_4.

5 Comprobación con escenarios

Pulsación larga de A: gira A y sigue. A, soltar, A: para. A, soltar, B: cambia a giro B. B y luego P: para. A, A (para), A de nuevo: arranca otra vez. Todos coinciden con el circuito de puertas.

✅ **Resultado final**

6 estados, 3 JK, Moore. $M_A = \overline{Q_2 Q_1} Q_0 + Q_1 \overline{Q_0}$, $M_B = Q_1 Q_0 + Q_2 \overline{Q_0}$. Ecuaciones de entrada: ver pasos. Comprobado con las ecuaciones minimizadas simuladas a nivel de puerta frente a la tabla.

2022-23

Febrero 2023 (convocatoria del 22/01/2023)

Problemas 1, 2 y 4: sensores con signo, toldo y caja fuerte. El problema 3 (contador de la figura) no se resuelve aquí.

Feb 2023 · P1

P1 · Dos sensores en C2: diferencia y media

✔ verificado

Un dispositivo industrial utiliza dos sensores para la medida. Estos sensores son idénticos y están midiendo la misma variable, aunque su medida puede diferir. Las dos medidas se entregan a un sistema digital codificadas cada una con 4 bits en complemento a 2 (A_3A_0 para el sensor 1 y B_3B_0 para el sensor 2; A_3 y B_3 son los bits de signo). Para visualizar la medida se dispone de dos displays de 7 segmentos. En uno de ellos se visualizará la diferencia de valores y en el otro la media de las dos medidas. El signo se representará mediante un led. Diseñar mediante bloques combinacionales estándares (multiplexores, decodificadores, sumadores, comparadores...) el esquema del circuito combinacional.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: restar y promediar dos números en C2, mostrar el módulo en un display y el signo en un LED. **Teoría:** C2, extensión de signo y desbordamiento (T2); sumador/restador y valor absoluto (T5).

1 Rango de la diferencia

Cada medida va de -8 a 7. La diferencia $A - B$ va de -15 a 15: **5 bits** en C2. Para no desbordar, primero se **extiende el signo** de A y B a 5 bits (se repite el MSB).

2 Restar con un sumador

$A - B = A + \overline{B} + 1$: un sumador de 5 bits, con B negada y $C_{in} = 1$. El bit 4 del resultado es el **signo**: va al LED.

3 Módulo de la diferencia

$|D| = D \oplus s$ más s como C_{in} de un segundo sumador. El módulo va de 0 a 15 (4 bits).

4 Media

Se suman $A + B$ en 5 bits (rango -16 a 14) y se divide entre 2 con un **desplazamiento aritmético** (se descarta el LSB y se repite el signo). La media va de -8 a 7. El signo (otro LED o el mismo) es el MSB del desplazado.

5 Display

Un módulo de 0 a 15 en un solo display solo se puede mostrar en **hexadecimal** (0–F). Si se quiere decimal hace falta un segundo dígito. Para cada uno: decodificador a 7 segmentos.

⚠️ Redondeo de la media

El desplazamiento aritmético da $\lfloor (A + B)/2 \rfloor$ (redondea hacia $-\infty$: $-1/2 \to -1$). Truncar hacia cero daría 0. Ambas son válidas, pero **hay que decir cuál** se usa. Coinciden salvo en 72 de las 256 combinaciones (suma impar y negativa).

✅ Resultado final

Diferencia: sumador de 5 bits con B invertida y $C_{in} = 1$; signo = bit 4; módulo = XOR + sumador. Media: $A + B$ en 5 bits desplazado 1 a la derecha. Verificado con las 256 combinaciones.

Feb 2023 · P2

P2 · Circuito de control de un toldo

✔ verificado

Se desea realizar un circuito de control para el toldo de una terraza. El toldo tiene la función tanto de dar sombra como de proteger del viento y de la lluvia. Entradas: S (hay sol), L (llueve), V (hay mucho viento), F (hace frío en el interior de la casa). Salida BT: si $BT = 1$ el toldo debe estar extendido (bajado), si $BT = 0$ recogido. Características: (1) siempre que llueva se extiende el toldo para evitar que moje la terraza (no se considera posible que simultáneamente llueva y haga sol); (2) si hace viento se debe extender el toldo, salvo si el día está soleado y hace frío en la casa: entonces se recoge para que el sol caliente la casa; (3) si no hace viento ni llueve, solo se baja el toldo en los días de sol y cuando haga calor en el interior. Se pide: 1. Tabla de verdad de $BT(S, L, V, F)$. 2. Expresión reducida en suma de productos y en producto de sumas. 3. Esquema en puertas. 4. Variante con un interruptor manual/automático: en automático funciona como se ha descrito; en manual, un pulsador da la orden de recoger y otro de bajar, y el circuito debe mantener la orden tras soltarlos; si se pulsan ambos a la vez, el comportamiento debe ser predecible.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar un texto a tabla de verdad, simplificar (SOP y POS) y añadir memoria en el modo manual. **Teoría:** tabla de verdad y Karnaugh con indiferentes (T3); biestables (T6).

1 Traducir el texto a reglas

- Llueve ($L = 1$) $\Rightarrow$ $BT = 1$.
- Viento ($V = 1$) $\Rightarrow$ $BT = 1$, **salvo** $S = 1$ y $F = 1$ ($BT = 0$).
- Sin lluvia ni viento: $BT = 1$ solo si $S = 1$ y $F = 0$.
- $S = L = 1$ **no ocurre**: son 4 combinaciones indiferentes.

2 Tabla de verdad (orden S, L, V, F)

$BT = 1$ en las filas $2, 3, 4, 5, 6, 7, 8, 10$ (con $S = L = 1$, filas 12 a 15, indiferentes). En el resto, $BT = 0$.

SL\VF

	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 ×	13 ×	15 ×	14 ×
10	8 **1**	9	11	10 **1**

BT con indiferentes

Mapa de Karnaugh con las filas $S = L = 1$ como indiferentes.

3 Suma de productos

Tres grupos: L (cuatro casillas con indiferentes), $\overline{S}V$ y $S\overline{F}$:

$$BT = \overline{S}V + L + S\overline{F}$$

4 Producto de sumas

Se agrupan los ceros y se niega: $BT = (S + L + V)(\overline{S} + \overline{F})$. Comprobación: ambas coinciden en las 12 filas posibles.

5 Modo manual con memoria

En manual hay que **recordar** la orden tras soltar los pulsadores: un biestable Q. Sea U «recoger» y D «bajar». $Q^+ = \overline{U}\,(D + Q)$: U tiene prioridad (si se pulsan ambos, se recoge: comportamiento predecible y **seguro**), D pone $Q = 1$ y se mantiene hasta que se pulse U. La salida es $BT = M \cdot Q + \overline{M} \cdot BT_{auto}$ (un multiplexor 2:1 controlado por M).

✅ **Resultado final**

$BT = \overline{S}V + L + S\overline{F} = (S + L + V)(\overline{S} + \overline{F})$. Manual: biestable con $Q^+ = \overline{U}(D + Q)$ y un MUX para elegir manual o automático. Verificado con las 12 combinaciones posibles.

Feb 2023 · P4

P4 · Caja fuerte: secuencia A, A, B, A (Mealy con JK)

✔ verificado

El sistema de apertura de una caja fuerte está compuesto por dos teclas A y B, un circuito secuencial a diseñar y un temporizador que mantiene la caja fuerte abierta durante 5 minutos cuando recibe un nivel lógico 1 desde el circuito secuencial. Este temporizador vuelve a cerrar la caja fuerte pasado dicho tiempo. Mientras no se pulse ninguna tecla no se genera ningún nivel lógico de entrada al circuito secuencial. Para abrir la caja fuerte, la combinación secreta es: pulsar dos veces seguidas la tecla A, a continuación pulsar una vez la tecla B, y finalmente pulsar una vez la tecla A. Si se hace de esta manera, el circuito secuencial dará una salida a nivel lógico 1 que actuará sobre el temporizador. Si en cualquier momento se introdujera un error al pulsar la secuencia secreta, en el siguiente ciclo de reloj todos los biestables se pondrán a cero (el sistema pasará al estado inicial) y la secuencia debe volver a introducirse desde el principio. 1. Dibujar el diagrama de estados, explicando claramente en qué consiste cada estado. 2. Implementar el circuito secuencial usando biestables JK y las puertas necesarias.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un detector de la secuencia A-A-B-A, con las pulsaciones separadas, que reinicia ante cualquier error. **Teoría:** diseño de autómatas (T10), tabla de excitación del JK (T6), Karnaugh de 5 variables.

⚠️ **Suposiciones que hay que declarar**

El enunciado no fija cómo llegan las teclas. Se supone **dos líneas**: $a = 1$ si se pulsa A y $b = 1$ si se pulsa B ($ab = 00$: ninguna tecla; $ab = 11$ no ocurre: indiferente). «Pulsar dos veces seguidas A» = pulsar, soltar y pulsar de nuevo A (dos pulsaciones). Mantener la tecla varios ciclos cuenta como **una** pulsación. Después de abrir se espera a soltar A antes de volver a empezar.

1 Estados (8)

Estado	Significado
S_0	Reposo, esperando la primera A
S_1	A nº1 pulsada (se mantiene)
S_2	A nº1 soltada, espera A nº2
S_3	A nº2 pulsada (se mantiene)
S_4	A nº2 soltada, espera B
S_5	B pulsada (se mantiene)
S_6	B soltada, espera A nº3
S_7	A final pulsada: caja abierta, espera soltar

Son 8 estados: **3 biestables**, código $Q_2Q_1Q_0$ igual al número del estado.

2 Transiciones

- «Esperando una tecla» (S_2, S_4, S_6): sin tecla se queda; con la tecla correcta avanza; con la incorrecta, vuelve a S_0.
- «Tecla pulsada» (S_1, S_3, S_5): si se mantiene, se queda; si se suelta, avanza; si se pulsa otra, error y vuelve a S_0.
- S_0: con A va a S_1; con B va a S_0 (error).
- S_6 con A: pasa a S_7 y $Z = 1$.
- S_7: mientras A siga pulsada se queda; si no, vuelve a S_0.

3 Ecuaciones (Karnaugh de 5 variables Q_2, Q_1, Q_0, a, b con $ab = 11$ indiferente)

$$J_2 = Q_1Q_0\overline{a}\overline{b} \qquad K_2 = \overline{Q_1}a + Q_1b + Q_1Q_0\overline{a}$$

$$J_1 = Q_0\overline{a}\overline{b} \qquad K_1 = b + Q_0\overline{a}$$

$$J_0 = \overline{Q_2}a + Q_1a + Q_2\overline{Q_1}b \qquad K_0 = \overline{Q_2}\overline{a} + Q_1\overline{a} + Q_2\overline{Q_1}b$$

$$Z = Q_2 Q_1 \overline{Q_0} a$$

4 **Comprobación**

Las ecuaciones, simuladas a nivel de puerta, reproducen el diagrama en todas las transiciones y generan $Z = 1$ solo con la secuencia correcta. Se han comparado con una especificación independiente en las 9840 secuencias de entrada de hasta 8 ciclos: sin discrepancias. Si en la secuencia hay un error, no se abre; si A se mantiene dos ciclos, cuenta como una sola pulsación.

✅ **Resultado final**

8 estados, 3 JK. $Z = Q_2 Q_1 \overline{Q_0} a$. $J_2 = Q_1 Q_0 \overline{a}\,\overline{b}$, $K_2 = \overline{Q_1} a + Q_1 b + Q_1 Q_0 \overline{a}$, $J_1 = Q_0 \overline{a}\,\overline{b}$, $K_1 = b + Q_0 \overline{a}$, $J_0 = \overline{Q_2} a + Q_1 a + Q_2 \overline{Q_1} b$, $K_0 = \overline{Q_2}\,\overline{a} + Q_1 \overline{a} + Q_2 \overline{Q_1}\,\overline{b}$ (en total 14 términos y 35 literales).

Qué no se ha incluido

⚠️ **Problemas que no están resueltos aquí**

- **Febrero de 2023, problema 3** (contador de la figura): el esquema no se ha podido interpretar con seguridad y no se inventa.
- **Invernadero** (diciembre de 2024) y **cronograma con biestables T**: no se ha incluido un enunciado completo y fiable, así que no se reproducen. La teoría para resolverlos está en T3 y T6.
- Los exámenes de 2014 y 2015 no se han incluido por la misma razón.

La teoría para resolver los que faltan está en los temas indicados en cada caja.

EJERCICIOS RESUELTOS

Hojas de problemas resueltas

Diseño combinacional con Karnaugh y autómatas síncronos con biestables JK: 18 problemas resueltos paso a paso.

Dos hojas, en el mismo formato que los exámenes: enunciado, caja «Antes de empezar», pasos numerados plegados y resultado. Todas las ecuaciones se han comprobado con simulación a nivel de puerta contra una especificación independiente: las de **Karnaugh** con las tablas de verdad completas y las de **autómatas** con todas las secuencias de entrada hasta 12 bits.

⚠️ **Enunciados**

Los enunciados de las hojas están planteados en esta guía con sus condiciones explícitas (por ejemplo, qué hace cada indicador o cómo se codifican los estados). Si tu enunciado cambia alguna condición, el método es el mismo y solo cambia la tabla.

Hoja de problemas · Diseño combinacional con Karnaugh

10 problemas: mayoría, divisor, multiplicador, producción en serie, barco, decodificadores a 7 segmentos, comparador, máquina de monedas y binario a BCD.

`Hoja Karnaugh · K1` **K1 · Mayoría de tres entradas** ✔ verificado

Diseña un circuito con tres entradas S_2, S_1, S_0 cuya salida S vale 1 cuando **al menos dos** de las entradas valen 1 (circuito de mayoría, por ejemplo para un votador de tres sensores).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tabla de verdad, ecuación mínima y esquema. **Teoría:** minterms, mapa de Karnaugh de 3 variables y grupos de 2 y 4 casillas (T3).

1 **Tabla de verdad**

$S = 1$ en las combinaciones con dos o tres unos: $011, 101, 110, 111$, es decir $S = \sum m(3, 5, 6, 7)$.

2 **Mapa de Karnaugh**

S2\S1S0

	00	01	11	10
0	0	1	3 **1**	2
1	4	5 **1**	7 **1**	6 **1**

Mayoría de tres

Tres grupos de dos casillas. Cada par de entradas forma un producto.

Los tres grupos son S_1S_0, S_2S_0 y S_2S_1.

3 Ecuación y puertas

$S = S_1S_0 + S_2S_0 + S_2S_1$: tres AND de dos entradas y una OR de tres. En NAND-NAND: tres NAND de dos entradas y una NAND de tres.

✅ Resultado final

$S = S_1S_0 + S_2S_0 + S_2S_1$ (verificada con las 8 combinaciones).

`Hoja Karnaugh · K2` **K2 · Divisor de dos números de 2 bits** ✔ verificado

Diseña un circuito combinacional que divida el número $A = A_1A_0$ entre $B = B_1B_0$ (ambos de 2 bits, sin signo). Salidas: cociente C_1C_0 y resto R_1R_0. El caso $B = 0$ no puede darse (indiferente).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cuatro funciones de cuatro variables con indiferentes. **Teoría:** Karnaugh de 4 variables con indiferentes (T3).

1 Qué hace el circuito

Es la división entera: $A = B \cdot C + R$ con $0 \le R < B$. Como $A \le 3$, el cociente cabe en 2 bits. Ejemplos: $3/2 \to C = 1, R = 1$; $3/1 \to C = 3, R = 0$; $2/3 \to C = 0, R = 2$.

2 Un mapa por salida

Las cuatro filas con $B_1 B_0 = 00$ son indiferentes (4 casillas «X» en cada mapa). El mapa de C_1:

A1A0\B1B0

	00	01	11	10
00	0 ×	1	3	2
01	4 ×	5	7	6
11	12 ×	13 **1**	15	14
10	8 ×	9 **1**	11	10

C1 (cociente, bit alto)

$C_1 = 1$ solo si $A \ge 2B$ con cociente ≥ 2: $A = 2$ o 3 y $B = 1$.

3 Ecuaciones

$$C_1 = A_1 \overline{B_1} \qquad C_0 = A_0 \overline{B_1} + A_1 \overline{B_0} + A_1 A_0$$

$$R_1 = A_1\overline{A_0}B_1B_0 \qquad R_0 = A_0\overline{B_0} + \overline{A_1}A_0B_1$$

✅ **Resultado final**

Las cuatro ecuaciones anteriores. Comprobadas contra la división entera en las 12 combinaciones válidas.

Hoja Karnaugh · K4

K4 · Multiplicador de 2 × 2 bits

✔ verificado

Diseña un multiplicador combinacional de dos números de 2 bits $A = A_1A_0$ y $B = B_1B_0$. El producto tiene 4 bits $S_3S_2S_1S_0$.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: cuatro funciones de cuatro variables, sin indiferentes. **Teoría:** Karnaugh de 4 variables (T3); producto binario como suma de productos parciales (T2).

1 Rango del producto

$3 \times 3 = 9$ cabe en 4 bits. Todas las combinaciones son válidas.

2 Mapas

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4	5	7 **1**	6 **1**
11	12	13 **1**	15	14 **1**
10	8	9 **1**	11 **1**	10

S1 del producto

S_1 es la función más complicada: cuatro grupos de dos casillas.

3 Ecuaciones

$$S_3 = A_1A_0B_1B_0 \qquad S_2 = A_1\overline{A_0}B_1 + A_1B_1\overline{B_0}$$

$$S_1 = \overline{A_1}A_0B_1 + A_0B_1\overline{B_0} + A_1\overline{A_0}B_0 + A_1\overline{B_1}B_0 \qquad S_0 = A_0B_0$$

$S_0 = A_0B_0$ es la AND de los bits menos significativos. $S_3 = 1$ solo con $3 \times 3 = 9$.

✅ **Resultado final**

Las cuatro ecuaciones anteriores, verificadas con las 16 combinaciones. Alternativa sin Karnaugh: cuatro AND de partida y dos semisumadores.

Hoja Karnaugh · K5

K5 · Cuatro máquinas en serie: estado de la producción

✔ verificado

Una línea de producción tiene cuatro máquinas $S_3S_2S_1S_0$ (1 = funcionando). Un panel tiene cuatro indicadores: **TA** (las cuatro funcionan), **MA** (al menos tres funcionan), **I** (funcionan exactamente dos) y **MR** (funciona una o ninguna). Diseña el circuito que los activa. Este es el reparto que se usa aquí.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cuatro funciones de cuatro variables según el **número de unos**. **Teoría:** Karnaugh de 4 variables y grupos de 2, 4 y 8 casillas (T3).

1 Contar los unos

Sea n el número de máquinas que funcionan. TA: $n = 4$. MA: $n \geq 3$. I: $n = 2$. MR: $n \leq 1$. Los indicadores MA y TA se solapan (con $n = 4$ ambos valen 1) pero I y MR no.

2 Ecuaciones

$$TA = S_3S_2S_1S_0$$

$$MA = S_2S_1S_0 + S_3S_1S_0 + S_3S_2S_0 + S_3S_2S_1$$

$$MR = \overline{S_3S_2S_1} + \overline{S_3S_2S_0} + \overline{S_3S_1S_0} + \overline{S_2S_1S_0}$$

$$I = \overline{S_3}S_2S_1S_0 + \overline{S_3}S_2\overline{S_1}S_0 + \overline{S_3}S_2S_1\overline{S_0} + S_3\overline{S_2S_1}S_0 + S_3\overline{S_2}S_1\overline{S_0} + S_3S_2\overline{S_1S_0}$$

I necesita seis productos de cuatro variables: los seis unos del mapa son todas las casillas aisladas, ninguna se puede agrupar con otra. Es el caso típico donde **no** simplifica el mapa.

⚠️ I (exactamente 2) no se simplifica

Los unos de «exactamente dos» están en las casillas de peso 2: cada una tiene como vecinas casillas de peso 1 o 3, que valen 0. Por eso ningún grupo mayor que 1 es posible. Tras Karnaugh, la ecuación sigue siendo la suma de seis minterms.

✅ Resultado final

TA, MA, MR e I con las ecuaciones anteriores. Verificadas con las 16 combinaciones.

`Hoja Karnaugh · K6`

K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos)

✔ verificado

Una brújula electrónica da cuatro señales N, S, E, O (1 = el barco apunta en esa dirección). Se muestra en un display de 7 segmentos: N enciende a; S enciende d; E enciende b, c; O enciende e, f; y las direcciones intermedias combinan las dos (NE: a, b; NO: a, f; SE: c, d; SO: d, e). No pueden valer 1 a la vez N y S, ni E y O. Si todas valen 0, no se encuentra nada (indiferente).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: siete funciones (a–g) con muchos indiferentes. **Teoría:** indiferentes en Karnaugh (T3), displays de 7 segmentos (T5).

1 Indiferentes

Las combinaciones con $N = S = 1$ o $E = O = 1$ y la combinación 0000 son indiferentes: 8 de las 16.

2 Ecuaciones

$$a = N \qquad b = \overline{S}E \qquad c = \overline{N}E \qquad d = S$$

$$e = \overline{N}O \qquad f = \overline{S}O \qquad g = 0$$

Con tantos indiferentes cada segmento queda con **un solo literal** o dos: $a = N$ (el segmento de arriba se enciende siempre que hay norte), $d = S$.

💡 **Por qué sale tan simple**

Una combinación indiferente se usa como 1 si ayuda a hacer un grupo mayor. Aquí las prohibidas ($N = S = 1$, $E = O = 1$) permiten agrupar casillas lejanas y el segmento g no se enciende nunca.

✅ **Resultado final**

$a = N;\ b = \overline{S}E;\ c = \overline{N}E;\ d = S;\ e = \overline{N}O;\ f = \overline{S}O;\ g = 0.$

`Hoja Karnaugh · K7`

K7 · Decodificador BCD a 7 segmentos

✔ verificado

Diseña el decodificador de un dígito BCD $ABCD$ (A es el bit de más peso) a las siete señales $a, \ldots, g$ de un display de ánodo/cátodo común activas a 1. Las combinaciones 10 a 15 no existen (indiferentes). Los dígitos se dibujan con la forma habitual: el 6 lleva el segmento a, el 7 solo a, b, c y el 9 lleva el segmento d.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: siete funciones de cuatro variables con 6 indiferentes. **Teoría:** Karnaugh con indiferentes (T3), display de 7 segmentos (T5).

1 Tabla de segmentos

Dígito	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1

Dígito	a	b	c	d	e	f	g
9	1	1	1	1	0	1	1

2 Un mapa por segmento



Segmento e

El segmento e solo se enciende en 0, 2, 6 y 8. Con los indiferentes se agrupa en dos grupos.

3 Ecuaciones

$$a = \overline{BD} + C + BD + A \qquad b = \overline{B} + \overline{CD} + CD \qquad c = \overline{C} + D + B$$

$$d = \overline{BD} + \overline{B}C + C\overline{D} + B\overline{C}D + A$$

$$e = \overline{BD} + C\overline{D} \qquad f = \overline{CD} + B\overline{C} + B\overline{D} + A \qquad g = \overline{B}C + C\overline{D} + B\overline{C} + A$$

✅ **Resultado final**

Las siete ecuaciones anteriores, comprobadas con los 10 dígitos válidos (los indiferentes no se comprueban).

`Hoja Karnaugh · K8` **K8 · Comparador de dos números de 2 bits** ✔ verificado

Diseña un comparador de $A = A_1A_0$ y $B = B_1B_0$ con tres salidas: GT ($A > B$), EQ ($A = B$) y LT ($A < B$).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tres funciones de cuatro variables. **Teoría:** comparadores y Karnaugh (T3, T5).

1 **Mapas**

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7	6
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11	10

GT: A > B

Seis casillas en escalera: tres grupos de dos.

EQ vale 1 en la diagonal: cuatro casillas que no se pueden agrupar. LT es simétrica a GT.

2 Ecuaciones

$$GT = A_0\overline{B_1}\,\overline{B_0} + A_1\overline{B_1} + A_1A_0\overline{B_0}$$

$$LT = \overline{A_1}\,\overline{A_0}B_0 + \overline{A_1}B_1 + \overline{A_0}B_1B_0$$

$$EQ = \overline{A_1}\,\overline{A_0}\,\overline{B_1}\,\overline{B_0} + \overline{A_1}A_0\overline{B_1}B_0 + A_1\overline{A_0}B_1\overline{B_0} + A_1A_0B_1B_0$$

💡 **Atajo para EQ**

$EQ = \overline{(A_1 \oplus B_1) + (A_0 \oplus B_0)}$: dos XOR y una NOR. Es lo mismo que la suma de cuatro minterms, con menos puertas. También se obtiene como $EQ = \overline{GT + LT}$.

✅ **Resultado final**

Las tres ecuaciones anteriores (con EQ en minterms o con XOR), verificadas con las 16 combinaciones.

`Hoja Karnaugh · K9` **K9 · Máquina de monedas con cambio** ✔ verificado

Una máquina vende un producto de 40 céntimos. Admite hasta dos monedas de 25 (E_{25A}, E_{25B}), una de 10 (E_{10}) y una de 5 (E_5): cada entrada vale 1 si esa moneda se ha introducido. La salida P entrega el producto si la suma llega a 40 o más. El cambio se devuelve con dos dispensadores de 10 (C_{10A}, C_{10B}) y uno de 5 (C_5).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: cuatro funciones; hay que calcular el cambio $= \text{suma} - 40$ y repartirlo en monedas de 10 y 5. **Teoría:** Karnaugh (T3), tabla de verdad a partir de un texto.

1 Tabla de casos relevantes

La suma máxima es 65 céntimos. El producto se entrega con suma ≥ 40.

Monedas	Suma	P	Cambio	Dispensadores
25 + 25	50	1	10	C_{10A}
25 + 25 + 10	60	1	20	C_{10A}, C_{10B}
25 + 25 + 5	55	1	15	C_{10A}, C_5
25 + 25 + 10 + 5	65	1	25	C_{10A}, C_{10B}, C_5
25 + 10 + 5	40	1	0	ninguno
25 + 10	35	0	0	ninguno

Con una sola moneda de 25 y sin la de 10 y la de 5 juntas no se llega a 40.

2 Ecuaciones

$$P = E_{25B}E_{10}E_5 + E_{25A}E_{10}E_5 + E_{25A}E_{25B}$$

$$C_{10A} = E_{25A}E_{25B} \qquad C_{10B} = E_{25A}E_{25B}E_{10} \qquad C_5 = E_{25A}E_{25B}E_5$$

El cambio solo existe cuando se han metido **las dos** de 25 (50 céntimos): por eso las tres salidas contienen $E_{25A}E_{25B}$.

✅ Resultado final

Las cuatro ecuaciones anteriores, verificadas con las 16 combinaciones (suma máxima 65 céntimos).

`Hoja Karnaugh · K10`

K10 · Nota de 0 a 10 en un display de barras

✔ verificado

Una nota de 0 a 10 llega en binario ($N_3N_2N_1N_0$). Se visualiza en un display de 7 segmentos como una **barra que crece**: de 0 a 4 solo se enciende g (un guion: suspenso); 5 enciende a; 6, a, b; 7 y 8, a, b, c; 9, a, b, c, d; y 10 enciende $a, \ldots, f$. Las combinaciones 11 a 15 son indiferentes.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: siete funciones con 5 indiferentes. **Teoría:** Karnaugh con indiferentes (T3).

1 Ecuaciones

$$a = N_2N_0 + N_2N_1 + N_3 \qquad b = N_2N_1 + N_3 \qquad c = N_2N_1N_0 + N_3$$

$$d = N_3N_0 + N_3N_1 \qquad e = N_3N_1 \qquad f = N_3N_1 \qquad g = \overline{N_3N_2} + \overline{N_3N_1N_0}$$

Fíjate en que e y f valen lo mismo: solo se encienden con la nota 10.

✅ Resultado final

Las siete ecuaciones anteriores, comprobadas para las notas 0 a 10.

Hoja Karnaugh · K17

K17 · Binario de 4 bits a dos dígitos BCD

✔ verificado

Un número binario $E_3E_2E_1E_0$ (0 a 15) se convierte a **dos dígitos BCD**: decenas D (un solo bit, vale 0 o 1) y unidades $U_3U_2U_1U_0$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cinco funciones de cuatro variables. **Teoría:** BCD y binario (T2), Karnaugh (T3).

1 Idea

Para $E \leq 9$ el BCD coincide con el binario. Para $10 \leq E \leq 15$ las decenas valen 1 y las unidades son $E - 10$ (0 a 5). Por ejemplo, $13 = 0\,1101 \rightarrow D = 1$, unidades $= 0011$.

2 Ecuaciones

$$D = E_3E_1 + E_3E_2 \qquad U_0 = E_0$$

$$U_3 = E_3\overline{E_2E_1} \qquad U_2 = \overline{E_3}E_2 + E_2E_1 \qquad U_1 = \overline{E_3}E_1 + E_3E_2\overline{E_1}$$

$U_0 = E_0$: restar 10 no cambia la paridad.

✅ Resultado final

Las cinco ecuaciones anteriores, verificadas con las 16 combinaciones.

Hoja de problemas · Autómatas síncronos con biestables JK

8 problemas: tres o más unos o ceros, detectores de secuencia con solapamiento, sumador serie y contador reversible.

Hoja autómatas · A1

A1 · Tres o más unos seguidos

✔ verificado

Diseña un autómata síncrono (Mealy, biestables JK) con una entrada serie E y una salida S que valga 1 mientras se hayan recibido **tres o más unos consecutivos** (desde el tercer uno y mientras sigan llegando unos). Un cero reinicia la cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: diagrama de estados, codificación, tablas de excitación y ecuaciones J-K y de salida. **Teoría:** diseño de autómatas síncronos (T10), excitación del JK (T6).

1 Estados

S_0: ningún uno seguido. S_1: un uno. S_2: dos o más unos. Tres estados necesitan dos biestables: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$ (Q_1Q_0); el 11 es indiferente.

2 Transiciones (entrada E / salida S)

Estado	$E = 0$	$E = 1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_0/0$	$S_2/1$

En S_2 con $E = 1$ ya van tres o más unos: salida 1 y se queda.

3 Excitación y ecuaciones

Con la tabla de excitación del JK y Karnaugh con el estado 11 como indiferente:

$$J_1 = Q_0E \qquad K_1 = \overline{E}$$

$$J_0 = \overline{Q_1}E \qquad K_0 = 1$$

$$S = Q_1E$$

4 Comprobación

Simulado a nivel de puerta contra la definición («lleva tres o más unos seguidos») para **todas** las secuencias de hasta 12 bits.

✅ Resultado final

$J_1 = Q_0E$, $K_1 = \overline{E}$, $J_0 = \overline{Q_1}E$, $K_0 = 1$, $S = Q_1E$ (Mealy).

`Hoja autómatas · A2` **A2 · Tres o más ceros seguidos** ✔ verificado

El mismo problema que A1, pero con **tres o más ceros consecutivos**. Un uno reinicia la cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: el mismo diseño con la entrada negada. **Teoría:** T10, T6.

1 Cambio respecto de A1

Es A1 con E y $\overline{E}$ intercambiados: los mismos tres estados cuentan ceros en vez de unos.

2 Ecuaciones

$$J_1 = Q_0\overline{E} \qquad K_1 = E$$

$$J_0 = \overline{Q_1E} \qquad K_0 = 1$$

$$S = Q_1\overline{E}$$

✅ Resultado final

$J_1 = Q_0\overline{E}$, $K_1 = E$, $J_0 = \overline{Q_1E}$, $K_0 = 1$, $S = Q_1\overline{E}$. Verificado con todas las secuencias de hasta 12 bits.

Hoja autómatas · A3

A3 · Detector de la secuencia 1011 (con solapamiento)

✔ verificado

Diseña un detector de la secuencia **1011** para una entrada serie E. La salida S vale 1 en el ciclo en que llega el último bit de la secuencia. Las secuencias pueden **solaparse** (en 1011011 se detectan dos).

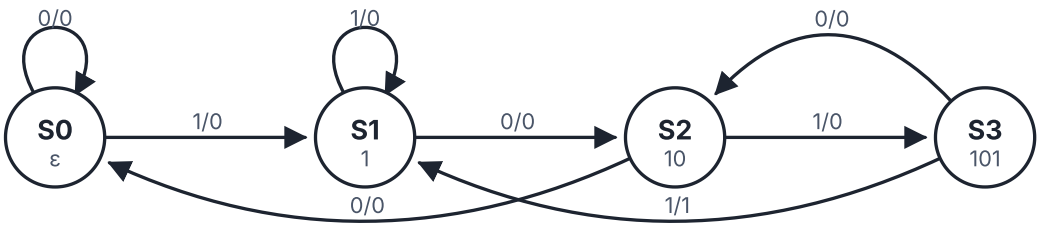
🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: autómata de Mealy con 4 estados y 2 biestables JK. **Teoría:** detectores de secuencia (T10).

1 Estados

Cada estado recuerda **el mayor prefijo de 1011** que es sufijo de lo recibido. S_0: ninguno. S_1: «1». S_2: «10». S_3: «101». Codificación $S_0 = 00,\ S_1 = 01,\ S_2 = 10,\ S_3 = 11$.

2 Diagrama



Detector de 1011 con solapamiento. Etiquetas: E / S

Desde S_3 con un 1 se detecta la secuencia y el último 1 pasa a ser el primero de la siguiente (S_1). Con un 0 se queda «10» (S_2).

3 Ecuaciones

$$J_1 = Q_0\overline{E} \qquad K_1 = \overline{Q_0 E} + Q_0 E$$

$$J_0 = E \qquad K_0 = \overline{E}$$

$$S = Q_1 Q_0 E$$

⚠️ **El solapamiento**

Sin solapamiento, S_3 con $E = 1$ volvería a S_0. Con solapamiento vuelve a S_1. Es el error más común en este tipo de problemas: **mira siempre qué parte del final de la secuencia vale como principio de la siguiente**.

✅ **Resultado final**

$J_1 = Q_0\overline{E}$, $K_1 = \overline{Q_0 E} + Q_0 E$, $J_0 = E$, $K_0 = \overline{E}$, $S = Q_1 Q_0 E$ (Mealy). Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A5a`

A5a · Detector de cuatro unos (1111)

✔ verificado

Diseña un detector de la secuencia **1111** con solapamiento (la salida vale 1 cada vez que los cuatro últimos bits recibidos son unos).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: Mealy con 4 estados. **Teoría:** T10.

1 Estados

Los estados cuentan los unos seguidos: S_0 (0), S_1 (1), S_2 (2), S_3 (3 o más). Con $E = 1$ en S_3 se activa la salida y se **permanece** en S_3 (solapamiento). Con $E = 0$ siempre se vuelve a S_0.

2 Ecuaciones

$$J_1 = Q_0 E \qquad K_1 = \overline{E}$$

$$J_0 = E \qquad K_0 = \overline{Q_1} + \overline{E}$$

$$S = Q_1 Q_0 E$$

✅ **Resultado final**

$J_1 = Q_0 E$, $K_1 = \overline{E}$, $J_0 = E$, $K_0 = \overline{Q_1} + \overline{E}$, $S = Q_1 Q_0 E$. Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A5b` **A5b · Detector de la secuencia 11011** ✔ verificado

Diseña un detector de la secuencia **11011** con solapamiento.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: Mealy con 5 estados, es decir 3 biestables. **Teoría:** T10.

1 Estados

Prefijos de 11011: S_0 (ninguno), S_1 («1»), S_2 («11»), S_3 («110»), S_4 («1101»). Cinco estados necesitan 3 biestables (000 a 100); los otros tres códigos son indiferentes.

2 Transiciones

Estado	$E = 0$	$E = 1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_3/0$	$S_2/0$
S_3	$S_0/0$	$S_4/0$
S_4	$S_0/0$	$S_2/1$

En S_2 con $E = 1$ (tres unos seguidos) se sigue en «11»: se queda en S_2. En S_4 con $E = 1$ se detecta la secuencia y los dos últimos unos valen como «11»: se pasa a S_2.

3 Ecuaciones

$$J_2 = Q_1 Q_0 E \qquad K_2 = 1$$

$$J_1 = Q_0 E + Q_2 E \qquad K_1 = Q_0$$

$$J_0 = \overline{Q_2 Q_1} E + Q_1 \overline{E} \qquad K_0 = 1$$

$$S = Q_2 E$$

✅ **Resultado final**

Las siete ecuaciones anteriores. Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A7` **A7 · Detector de 1010 o de 110** ✔ verificado

Diseña un autómata con una entrada serie E cuya salida vale 1 cuando los últimos bits recibidos son **1010 o 110** (con solapamiento).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que detecta **dos** secuencias a la vez. **Teoría:** T10.

1 Estados

Cada estado es el mayor sufijo de lo recibido que es prefijo de alguna de las dos secuencias. Los prefijos útiles son: ε, «1», «10», «101», «11». Son 5 estados ($S_\varepsilon, S_1, S_{10}, S_{11}, S_{101}$), 3 biestables. Codificación: $S_\varepsilon = 000$, $S_1 = 001$, $S_{10} = 010$, $S_{11} = 011$, $S_{101} = 100$.

2 Transiciones

Estado	$E = 0$	$E = 1$
S_ε	$S_\varepsilon / 0$	$S_1 / 0$
S_1	$S_{10} / 0$	$S_{11} / 0$
S_{10}	$S_\varepsilon / 0$	$S_{101} / 0$

Estado	$E=0$	$E=1$
S_{11}	$S_{10}/1$	$S_{11}/0$
S_{101}	$S_{10}/1$	$S_{11}/0$

Con 110 el último 0 deja «10» (vale como inicio de 1010). Con 1010 también queda «10».

3 Ecuaciones

$$J_2 = Q_1\overline{Q_0}E \qquad K_2 = 1$$

$$J_1 = Q_0 + Q_2 \qquad K_1 = \overline{Q_0}$$

$$J_0 = \overline{Q_1}E \qquad K_0 = \overline{E}$$

$$S = Q_1Q_0\overline{E} + Q_2\overline{E}$$

✅ Resultado final

Las siete ecuaciones anteriores. Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A9` **A9 · Sumador serie** ✔ verificado

Diseña un **sumador serie**: dos números llegan bit a bit, empezando por el menos significativo, por las entradas E_1 y E_0. En cada ciclo la salida S da el bit correspondiente de la suma. Un biestable JK guarda el acarreo.

🧭 Antes de empezar: qué piden y qué teoría usas

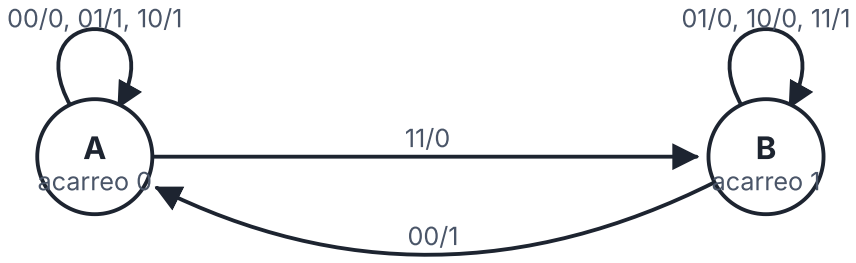
Qué piden: autómata de Mealy de dos estados (acarreo 0 o 1). **Teoría:** sumador completo (T5), autómatas (T10).

1 Estados

Dos estados: A (acarreo 0, $Q_0 = 0$) y B (acarreo 1, $Q_0 = 1$). Un solo biestable.

2 Transiciones

Con $t = E_1 + E_0 + \text{acarreo}$: salida $= t \bmod 2$ y nuevo acarreo $= \lfloor t/2 \rfloor$.



Sumador serie. Etiquetas: E_1E_0 / S

El estado recuerda el acarreo; la salida es el bit de suma.

3 Ecuaciones

$$J_0 = E_1 E_0 \qquad K_0 = \overline{E_1 E_0}$$

$$S = \overline{Q_0 E_1} E_0 + \overline{Q_0} E_1 \overline{E_0} + Q_0 \overline{E_1 E_0} + Q_0 E_1 E_0$$

El acarreo se activa solo si las dos entradas valen 1 ($J_0 = E_1 E_0$) y se desactiva solo si las dos valen 0 ($K_0 = \overline{E_1}\,\overline{E_0}$). En el resto de casos se mantiene. La salida es la **XOR de tres bits**: $S = Q_0 \oplus E_1 \oplus E_0$.

✅ Resultado final

$J_0 = E_1 E_0$, $K_0 = \overline{E_1 E_0}$, $S = Q_0 \oplus E_1 \oplus E_0$. Probado con 400 sumas aleatorias de hasta 10 bits.

Hoja autómatas · A11

A11 · Contador reversible módulo 4

✔ verificado

Diseña un contador módulo 4 **reversible** (autómata de Moore). La entrada E elige el sentido: $E = 1$ cuenta hacia arriba $(0, 1, 2, 3, 0 \ldots)$ y $E = 0$ hacia abajo $(0, 3, 2, 1, 0 \ldots)$. La salida es el valor de la cuenta en binario de dos bits.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador de dos biestables JK con entrada de sentido. **Teoría:** contadores (T8), autómatas de Moore (T10).

1 Estados y salida

Cuatro estados $S_0, \ldots, S_3$ con codificación igual al valor $(00, 01, 10, 11)$. Como es Moore, **la salida depende solo del estado**: $S_1 S_0 = Q_1 Q_0$.

2 Ecuaciones

Para contar: Q_0 cambia siempre; Q_1 cambia cuando $Q_0 = 1$ si sube y cuando $Q_0 = 0$ si baja.

$$J_0 = K_0 = 1 \qquad J_1 = K_1 = \overline{Q_0 E} + Q_0 E$$

$J_1 = K_1 = \overline{Q_0 \oplus E}$: se activa cuando $Q_0 = E$ (con $E = 1$ sube y $Q_0 = 1$ acarrea; con $E = 0$ baja y $Q_0 = 0$ pide préstamo).

3 Comprobación

Con $E = 1, 1, 1, 1, 1, 0, 0, 0, 0$ desde el estado 0 las salidas, tras cada flanco de reloj, son [1, 2, 3, 0, 1, 0, 3, 2, 1].

✅ Resultado final

$J_0 = K_0 = 1$; $J_1 = K_1 = \overline{Q_0 E} + Q_0 E$; salidas $S_1 = Q_1$, $S_0 = Q_0$ (Moore).

EJERCICIOS RESUELTOS

Simulacros de examen

Dos exámenes de cuatro problemas con las soluciones ocultas. Cronométrate 2 horas.

Cómo usarlos

1. Prepara papel, calculadora y el formulario de una página (formulario). Nada más.
2. Pon un cronómetro de **2 horas** por simulacro. Son cuatro problemas de 2,5 puntos, como en los exámenes reales (examen).
3. Resuelve **sin abrir** las soluciones. Los desplegables están cerrados a propósito.
4. Corrige con la regla de los exámenes: se valoran la **simplicidad** y la **claridad**. Un cronograma sin explicación no cuenta.
5. Anota qué tipo de problema has fallado y vuelve a su tema en la guía (plan).

> 💡 **Todo está comprobado**
>
> Cada solución se ha simulado a nivel de puerta con todas las combinaciones posibles (o con todas las secuencias de entrada de hasta 8 a 12 bits, en los autómatas). Los problemas son originales de esta guía, pero del mismo tipo que los de examen.

Simulacro 1

Problema 1 · Código Gray a binario (2,5 puntos)

Un sensor de posición angular entrega su lectura en **código Gray de 3 bits** $G_2G_1G_0$. Diseña el circuito combinacional que la convierte a binario natural $B_2B_1B_0$: tabla de verdad, ecuaciones simplificadas y esquema con puertas.

Ver la solución del problema 1

Teoría: código Gray (solo cambia un bit entre valores consecutivos) y Karnaugh o XOR (T2, T3).

La secuencia Gray de 0 a 7 es ["000", "001", "011", "010", "110", "111", "101", "100"]. Se invierte la tabla: para cada valor Gray se busca el binario.

$G_2G_1G_0$	000	001	011	010	110	111	101	100
$B_2B_1B_0$	000	001	010	011	100	101	110	111

Ecuaciones simplificadas por Karnaugh:

$$B_2 = G_2 \qquad B_1 = \overline{G_2}G_1 + G_2\overline{G_1}$$

$$B_0 = \overline{G_2G_1}G_0 + \overline{G_2}G_1\overline{G_0} + G_2\overline{G_1G_0} + G_2G_1G_0$$

B_1 es una XOR y B_0 es la XOR de tres entradas. Con puertas XOR queda:

$$B_2 = G_2 \qquad B_1 = G_2 \oplus G_1 \qquad B_0 = G_2 \oplus G_1 \oplus G_0 = B_1 \oplus G_0$$

Son **dos XOR en cascada**: la forma más simple. Cada bit binario es la XOR de todos los bits Gray de igual o mayor peso.

Problema 2 · Sumador BCD de un dígito (2,5 puntos)

Diseña con bloques estándar un sumador de dos dígitos BCD (A y B, cada uno de 0 a 9) con acarreo de entrada. La salida debe ser un dígito BCD y un acarreo de salida. Explica cómo se detecta que hace falta corregir y por qué se suma 6.

Ver la solución del problema 2

Teoría: BCD y aritmética en BCD (T2), sumador de 4 bits (T5).

1. Un sumador de 4 bits suma $A + B + C_{in}$ en binario. El máximo es $9 + 9 + 1 = 19$, que cabe en 5 bits (acarreo K y suma $S_3S_2S_1S_0$).
2. Si la suma vale ≤ 9 ya es un dígito BCD. Si vale de 10 a 19, el resultado BCD correcto es **«1» y el dígito** $S - 10$. Pero el binario ha contado 16 por cada acarreo, no 10. Se **suma 6** (la diferencia entre 16 y 10) y se descartan los 16.
3. Hace falta corregir cuando la suma es mayor que 9: $K = 1$ (suma ≥ 16) o $S_3S_2 = 1$ o $S_3S_1 = 1$ (suma de 10 a 15):

$$\text{Corregir} = K + S_3S_2 + S_3S_1$$

1. Un segundo sumador de 4 bits suma $0\,(\text{Corregir})(\text{Corregir})0$, es decir 0110 o 0000, a $S_3S_2S_1S_0$. Su resultado es el dígito BCD. El acarreo de salida es «Corregir».

Ejemplos: $7 + 8 = 15 = 01111_2$; hay que corregir; $1111 + 0110 = 10101$: dígito 0101 y acarreo 1, es decir **15**. $9 + 9 + 1 = 19 = 10011_2$: el binario da $K = 1$ y 0011; $0011 + 0110 = 1001$ con acarreo 1: **19**. $4 + 3 = 0111$: no se corrige.

Se ha comprobado con las 200 combinaciones de (A, B, C_{in}).

Problema 3 · Contador síncrono 0, 2, 5, 7 (2,5 puntos)

Diseña con biestables JK un contador síncrono de 3 bits que repita la secuencia **0, 2, 5, 7, 0, 2, 5, 7...** (en decimal). Los estados 1, 3, 4 y 6 no se usan. Da las ecuaciones J y K de cada biestable y una señal F que valga 1 en el estado 7. Comenta qué pasa si el circuito arranca en un estado no usado.

Ver la solución del problema 3

Teoría: diseño de contadores con JK (T8, T6).

Tabla de transición (los estados no usados son indiferentes):

Estado actual $Q_2Q_1Q_0$	Siguiente
000	010
010	101
101	111
111	000

Con la tabla de excitación del JK y Karnaugh con indiferentes:

$$J_2 = K_2 = Q_1 \qquad J_1 = K_1 = 1 \qquad J_0 = K_0 = Q_1$$

$$F = Q_1 Q_0$$

Todos los biestables trabajan en modo **toggle** ($J = K$): Q_1 conmuta siempre, Q_2 y Q_0 conmutan cuando $Q_1 = 1$. Se ha simulado a nivel de puerta y repite $0, 2, 5, 7, 0, \ldots$

> ⚠️ **El circuito no se recupera solo**
>
> Los estados no usados forman un **ciclo cerrado**: $1 \to 3 \to 4 \to 6 \to 1$. Si al encender el circuito los biestables arrancan en uno de ellos, el contador **nunca entra** en la secuencia buena. Solución: poner un **reset** asíncrono (clear) que lleve a 000 al arrancar, o elegir las indiferentes para que los estados no usados desemboquen en la secuencia (autoarranque).

Problema 4 · Detector de la secuencia 0110 (2,5 puntos)

Diseña con biestables JK un autómata (Mealy) con una entrada serie E y una salida S que valga 1 cuando los últimos cuatro bits recibidos sean **0110**. Puede haber solapamiento. Da el diagrama de estados, la tabla de transición y las ecuaciones.

Ver la solución del problema 4

Teoría: autómatas síncronos (T10).

Estados: el mayor prefijo de 0110 que es sufijo de lo recibido. S_0: ninguno. S_1: «0». S_2: «01». S_3: «011». Codificación $00, 01, 10, 11$.

Estado	$E = 0$	$E = 1$
S_0	$S_1/0$	$S_0/0$
S_1	$S_1/0$	$S_2/0$
S_2	$S_1/0$	$S_3/0$
S_3	$S_1/1$	$S_0/0$

Con un 0 siempre se pasa a «0» (S_1), salvo en S_3 donde ese 0 completa 0110 (salida 1) y vale como inicio de la siguiente (S_1).

$$J_1 = Q_0 E \qquad K_1 = \overline{E} + Q_0$$

$$J_0 = \overline{E} + Q_1 \qquad K_0 = E$$

$$S = Q_1 Q_0 \overline{E}$$

Verificado con todas las secuencias de hasta 12 bits.

Simulacro 2

Problema 1 · Codificador de prioridad (2,5 puntos)

Diseña un codificador de prioridad de cuatro entradas $I_3 I_2 I_1 I_0$ (I_3 es la más prioritaria). Salidas: $A_1 A_0$ (número de la entrada activa más prioritaria) y V, que indica si hay alguna entrada activa. Da las ecuaciones simplificadas.

Ver la solución del problema 1

Teoría: codificadores de prioridad (T5), Karnaugh (T3).

Entradas activas	A_1A_0	V
ninguna	00	0
solo I_0	00	1
I_1 (y quizá I_0)	01	1
I_2 (y quizá menores)	10	1
I_3 (y cualquier otra)	11	1

Con Karnaugh sobre las 16 combinaciones:

$$A_1 = I_2 + I_3 \qquad A_0 = \overline{I_2} I_1 + I_3 \qquad V = I_0 + I_1 + I_2 + I_3$$

V es una OR de las cuatro entradas. Fíjate en que A_0 **no** es «$I_3 + I_1$»: si $I_2 = 1$ y $I_1 = 1$, gana I_2 y A_0 debe ser 0. Por eso aparece $\overline{I_2} I_1$.

Problema 2 · Multiplicar por 5 (2,5 puntos)

Un número X de 4 bits sin signo debe multiplicarse por 5. Diseña el circuito con un solo sumador y sin multiplicador. ¿Cuántos bits tiene el resultado?

Ver la solución del problema 2

Teoría: numeración y desplazamientos (T2), sumadores (T5).

$5X = 4X + X$. Multiplicar por 4 es **desplazar dos posiciones a la izquierda** (cableado, sin puertas): $X \ll 2$ es $X_3X_2X_1X_0\,00$ (6 bits). Un sumador de 7 bits suma $X \ll 2$ y X (ampliados con ceros).

El máximo es $5 \cdot 15 = 75$. Como $2^6 = 64 < 75 < 128 = 2^7$, hacen falta **7 bits**. Se ha comprobado con los 16 valores de X.

Problema 3 · Temporización con un reloj de 50 MHz (2,5 puntos)

Una placa tiene un reloj de 50 MHz. (a) Obtén una señal de 1 Hz con ciclo de trabajo del 50 %: ¿hasta qué valor cuenta el contador y cuántos bits tiene? (b) Un display de 4 dígitos multiplexado se refresca con un contador de 17 bits cuyos dos bits altos eligen el dígito. ¿Cuál es la frecuencia de barrido y la de refresco de cada dígito? ¿Se verá parpadear? (c) ¿Cuántos bits necesita un contador que mide 10 ms (antirrebote) con ese reloj?

Ver la solución del problema 3

Teoría: divisores de frecuencia, multiplexado de displays y antirrebote (T8, T12).

(a) Una señal de 1 Hz con ciclo del 50 % cambia cada **medio segundo**: $25\,000\,000$ ciclos de reloj. El contador cuenta de 0 a 24999999 y entonces invierte un biestable de salida. Como $2^{24} < 25\,000\,000 \leq 2^{25}$, necesita **25 bits**.

(b) El contador de 17 bits desborda cada 2^{17} ciclos: $50\,000\,000/2^{17} \approx 381.5$ Hz. Los dos bits altos recorren cuatro dígitos, así que cada dígito se enciende ≈ 95.4 Hz. Es más que 60 Hz: **no se ve parpadear**.

(c) 10 ms $= 0{,}01 \cdot 50\,000\,000 = 500\,000$ ciclos. Como $2^{18} = 262\,144 < 500\,000 \leq 524\,288 = 2^{19}$, hacen falta **19 bits**.

Problema 4 · Máquina de 30 céntimos con cambio (2,5 puntos)

Una máquina vende un producto de 30 céntimos y acepta monedas de 10 y de 20 céntimos, una por ciclo. Las entradas son $M_{20}M_{10}$: 00 (nada), 01 (moneda de 10), 10 (moneda de 20); 11 no se da. Cuando el total llega a 30 o más, la salida Z entrega el producto y se vuelve a empezar. Si el total es 40, la salida C devuelve 10 céntimos de cambio. Diseña el autómata de Mealy con JK.

Ver la solución del problema 4

Teoría: autómatas de Mealy (T10).

Estados: S_0 (0 céntimos), S_1 (10), S_2 (20). Codificación $00, 01, 10$; el 11 es indiferente, y la entrada 11 también.

Estado	$M_{20}M_{10} = 00$	01 (10 cts)	10 (20 cts)
S_0	$S_0/ZC = 00$	$S_1/00$	$S_2/00$
S_1	$S_1/00$	$S_2/00$	$S_0/Z = 1$ (30)
S_2	$S_2/00$	$S_0/Z = 1$ (30)	$S_0/Z = 1, C = 1$ (40)

$$J_1 = \overline{Q_0}M_{20} + Q_0M_{10} \qquad K_1 = M_{10} + M_{20}$$

$$J_0 = \overline{Q_1}M_{10} \qquad K_0 = M_{10} + M_{20}$$

$$Z = Q_0M_{20} + Q_1M_{10} + Q_1M_{20} \qquad C = Q_1M_{20}$$

Verificado con todas las secuencias de monedas de hasta 8 ciclos.

DEMOSTRACIONES

Demostraciones

Por qué son ciertas las reglas que usas: De Morgan, consenso, C2, sumador, universalidad de NAND, JK, contadores, Johnson y LFSR.

Aquí se demuestran, paso a paso, las reglas que en los temas se usan sin discutir. En el examen no suelen pedir la demostración entera, pero **entender el porqué** te permite reconstruir la regla si la olvidas. Todos los enunciados se han comprobado además por código con todos los casos posibles.

Demostración D1

D1 · Leyes de De Morgan

✔ verificado

Demuestra que $\overline{A+B} = \overline{A} \cdot \overline{B}$ y que $\overline{A \cdot B} = \overline{A} + \overline{B}$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: probar una igualdad entre funciones lógicas. **Idea:** dos funciones son iguales si valen lo mismo para **todas** las combinaciones de entradas (inducción perfecta).

1 Tabla de verdad

A	B	$\overline{A+B}$	$\overline{A}\,\overline{B}$	$\overline{AB}$	$\overline{A}+\overline{B}$
0	0	1	1	1	1
0	1	0	0	1	1
1	0	0	0	1	1
1	1	0	0	0	0

Las columnas 3 y 4 coinciden; las 5 y 6 también. Con dos variables hay solo 4 filas: la prueba es completa.

2 La idea en palabras

$\overline{A+B}$ vale 1 solo si **ninguna** de las dos vale 1, es decir, si $A = 0$ **y** $B = 0$. $\overline{AB}$ vale 0 solo si las dos valen 1, es decir, vale 1 si **alguna** vale 0.

3 Generalización

Se aplica a n variables: $\overline{A+B+C}=\overline{A}\,\overline{B}\,\overline{C}$. La regla práctica: **cambia + por · (y al revés) y niega cada término**.

✅ **Resultado final**

Demostrado por inducción perfecta (y comprobado también con tres variables).

Demostración D2 · D2 · Adyacencia, absorción y consenso

✔ verificado

Demuestra: (a) $AB+A\overline{B}=A$; (b) $A+AB=A$; (c) $AB+\overline{A}C+BC=AB+\overline{A}C$.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tres identidades que justifican los grupos de Karnaugh. **Idea:** sacar factor común y usar $X+\overline{X}=1$.

1 (a) Adyacencia

$AB+A\overline{B}=A(B+\overline{B})=A\cdot 1=A$. Es la regla que hace que **dos casillas vecinas** del mapa se simplifiquen: la variable que cambia desaparece.

2 (b) Absorción

$A+AB=A(1+B)=A\cdot 1=A$. Si $A=1$ la suma ya vale 1, y si $A=0$ el término AB también vale 0.

3 (c) Consenso

Se multiplica el término BC por $(A+\overline{A})=1$:

$$BC=BC(A+\overline{A})=ABC+\overline{A}BC.$$

Entonces $AB+\overline{A}C+ABC+\overline{A}BC$. Pero ABC está absorbido por AB y $\overline{A}BC$ por $\overline{A}C$ (absorción). Queda $AB+\overline{A}C$.

⚠️ Para qué sirve el consenso

En un circuito con **aleas** (fallos momentáneos) a veces se **añade** el término de consenso a propósito. En el diseño con mínimos se **quita**.

✅ Resultado final

Las tres identidades se demuestran con factor común y $X + \overline{X} = 1$.

Demostración D3

D3 · Por qué invertir y sumar 1 da el opuesto en C2

✔ verificado

Demuestra que en complemento a 2 de n bits el opuesto de N es $\overline{N} + 1$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar la regla de «invertir y sumar 1». **Idea:** el C2 de N se define como $2^n - N$. Sumar N y su C2 da 2^n, que en n bits es 0.

1 Un número más su inverso

Si inviertes todos los bits de N, cada posición suma 1 con la de N: $N + \overline{N} = 11\ldots1 = 2^n - 1$.

2 Despejar el opuesto

$$-N \equiv 2^n - N = (2^n - 1 - N) + 1 = \overline{N} + 1.$$

Eso es exactamente «invertir y sumar 1». En n bits, 2^n se descarta (es el acarreo final), así que $N + (\overline{N} + 1) = 0$.

3 Ejemplo

Con 4 bits, $N = 0101 = 5$: $\overline{N} = 1010$, $+1 = 1011$. Comprobación: $0101 + 1011 = 1\,0000$; el acarreo se descarta y queda 0000. Entonces $1011 = -5$.

✅ Resultado final

$-N = \overline{N} + 1$ porque $N + \overline{N} = 2^n - 1$. Comprobado con todos los números de 3, 4, 5 y 8 bits.

Demostración D4

D4 · Por qué restar es sumar el inverso y un 1

✔ verificado

Demuestra que $A - B = A + \overline{B} + 1$ en un sumador de n bits (módulo 2^n).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar que un sumador sirve también como restador. **Teoría:** D3.

1 Aplicar D3

Por D3, $-B = \overline{B} + 1$ módulo 2^n.

2 Sumar

$A - B = A + (-B) = A + \overline{B} + 1$. El «+1» se introduce como **acarreo de entrada** $C_{in} = 1$, sin gastar otro sumador.

✅ Resultado final

Un sumador con B invertida y $C_{in} = 1$ calcula $A - B$. Comprobado con las 2^8 combinaciones de 4 bits y con 8 bits.

Demostración D5

D5 · Ecuaciones del sumador completo

✔ verificado

Demuestra que un sumador completo tiene $S = A \oplus B \oplus C_{in}$ y $C_{out} = AB + C_{in}(A \oplus B)$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar de «suma de tres bits» a ecuaciones lógicas. **Idea:** la suma $A + B + C_{in}$ va de 0 a 3 y se escribe $C_{out}S$ en binario.

1 Tabla

$A+B+C_{in}$	0	1	2	3
$C_{out}S$	00	01	10	11

S vale 1 cuando el número de unos es **impar**: es la XOR de tres entradas. C_{out} vale 1 cuando hay **dos o más** unos: es la mayoría de tres.

2 **Mayoría de tres**

La mayoría de tres es $AB + AC_{in} + BC_{in}$ (ver K1 en las hojas). Se agrupa: $AB + C_{in}(A + B)$. Y como AB ya cubre el caso $A = B = 1$, se puede cambiar $A + B$ por $A \oplus B$: $C_{out} = AB + C_{in}(A \oplus B)$. También es válida $AB + C_{in}(A + B)$.

✅ **Resultado final**

$S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$. Comprobadas las 8 combinaciones.

Demostración D6

D6 · NAND y NOR son puertas universales

✔ verificado

Demuestra que con solo NAND se puede construir NOT, AND y OR (y por tanto cualquier función). Haz lo mismo con NOR.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: construir las tres puertas básicas a partir de una sola. **Teoría:** De Morgan (D1).

1 **NOT**

$\text{NAND}(A, A) = \overline{A \cdot A} = \overline{A}$. Se unen las dos entradas.

2 **AND**

$\text{NAND}(A, B) = \overline{AB}$. Se niega con una NAND con las entradas unidas: $\overline{\overline{AB}} = AB$.

3 **OR**

Por De Morgan, $A + B = \overline{\overline{A} \cdot \overline{B}} = \text{NAND}(\overline{A}, \overline{B})$: se niegan A y B con NAND y luego otra

NAND.

4 NOR

Por dualidad: $\mathrm{NOR}(A,A)=\overline{A}$; $A+B=\overline{\mathrm{NOR}(A,B)}$; $AB=\mathrm{NOR}(\overline{A},\overline{B})$.

✅ Resultado final

Con NAND (o con NOR) se obtiene NOT, AND y OR, y con ellas cualquier función. Comprobado con todas las combinaciones.

Demostración D7

D7 · Ecuación característica del JK y tabla de excitación

✔ verificado

Demuestra que el JK cumple $Q^+=J\overline{Q}+\overline{K}Q$ y deduce la tabla de excitación.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar del comportamiento del JK a una ecuación y de ahí al diseño. **Teoría:** T6.

1 La ecuación

El JK **pone a 1** (J) cuando $Q=0$ y **mantiene** el 1 si $K=0$ cuando $Q=1$:

- $Q=0$: sube solo si $J=1$. Término $J\overline{Q}$.
- $Q=1$: se queda en 1 solo si $K=0$. Término $\overline{K}Q$.

Sumando: $Q^+=J\overline{Q}+\overline{K}Q$.

2 Tabla de excitación

Se despejan J,K para cada transición $Q\to Q^+$:

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1

Q	Q^+	J	K
1	1	X	0

Si $Q = 0$ la K no influye; si $Q = 1$ la J no influye. Esas X son los indiferentes que simplifican las ecuaciones.

✅ **Resultado final**

$Q^+ = J\overline{Q} + \overline{K}Q$ y la tabla de arriba. Comprobada con las 8 combinaciones (Q, J, K) y con las 4 transiciones.

`Demostración D8`

D8 · Un contador módulo N divide la frecuencia entre N

✔ verificado

Demuestra que la salida de un contador módulo N (con una señal que vale 1 en un estado) tiene frecuencia f_{clk}/N y que se necesitan $n = \lceil \log_2 N \rceil$ biestables.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: relacionar cuenta, periodo y número de biestables. **Teoría:** T8.

1 Frecuencia

El contador pasa por N estados y vuelve al inicial **una vez cada N flancos**. Si una señal vale 1 en un solo estado, esa señal se repite cada N ciclos de reloj: periodo $N\,T_{clk}$, frecuencia f_{clk}/N. Un biestable T con $T = 1$ es el caso $N = 2$.

2 Número de biestables

Con n biestables hay 2^n combinaciones. Hacen falta N estados distintos: $2^n \geq N$, es decir $n \geq \log_2 N$. El mínimo entero es $n = \lceil \log_2 N \rceil$. Si N no es potencia de 2 sobran $2^n - N$ estados: hay que decidir adónde van (ver el **bloqueo** en los simulacros).

✅ **Resultado final**

$f_{out} = f_{clk}/N$ y $n = \lceil \log_2 N \rceil$. Comprobado para $N = 3, 5, 6, 10$.

Demostración D9

D9 · Contador síncrono binario: $T_i = Q_{i-1} \cdots Q_0$

✔ verificado

Demuestra que un contador binario síncrono de n bits con biestables T usa $T_i = Q_{i-1}Q_{i-2}\cdots Q_0$ (con $T_0 = 1$).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar cuándo conmuta cada bit al sumar 1. **Teoría:** T8.

1 Sumar 1 en binario

Al sumar 1 el bit 0 siempre cambia. El bit i cambia solo si **todos los inferiores valían 1** (hay acarreo hasta él), como en $0111 + 1 = 1000$.

2 Biestable T

Un T conmuta cuando $T = 1$. Así que $T_i = Q_{i-1} \cdots Q_0$, una AND de todos los bits menores. $T_0 = 1$ (AND vacía).

✅ Resultado final

$T_0 = 1, T_1 = Q_0, T_2 = Q_1Q_0, T_3 = Q_2Q_1Q_0$, … Comprobado con los 16 valores de un contador de 4 bits.

Demostración D10

D10 · Estados de los registros anillo, Johnson y LFSR

✔ verificado

Demuestra: (a) un registro en anillo de n biestables tiene n estados; (b) un contador Johnson tiene $2n$; (c) un LFSR de n bits tiene **como máximo** $2^n - 1$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: contar los estados que visita cada registro. **Teoría:** T9.

1 (a) Anillo

Un único 1 que circula: hay n posiciones posibles y tras n desplazamientos vuelve al inicio. n estados.

2 (b) Johnson

La salida se realimenta **negada**. Empezando en $0\ldots0$ se van llenando de unos desde un lado (n estados) y luego se van vaciando (n estados más). Total $2n$.

3 (c) LFSR

La realimentación es una XOR de algunos bits. Con n bits hay 2^n valores, pero el estado $00\ldots0$ **se queda atascado** (la XOR de ceros es 0). Los demás 2^n-1 valores son el máximo posible. Se alcanza solo con ciertas posiciones de realimentación (polinomios primitivos).

✅ Resultado final

Anillo n, Johnson $2n$, LFSR $\leq 2^n-1$. Comprobados por simulación para $n=1\ldots8$ (anillo y Johnson) y con $n=3$ a 7 (LFSR, ciclo máximo de 2^n-1).

Demostración D11

D11 · Cuántas funciones hay de n variables y por qué un MUX universal

✔ verificado

Demuestra que hay 2^{2^n} funciones de n variables y que un multiplexor $2^k{:}1$ puede implementar cualquier función de $k+1$ variables.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: dos conteos con el mismo truco: tabla de verdad. **Teoría:** T3, T5.

1 Número de funciones

La tabla de verdad tiene 2^n filas y en cada una la salida es 0 o 1, de forma independiente. Total $2\cdot2\cdots2=2^{2^n}$. Con $n=2$: $2^4=16$; con $n=4$: $2^{16}=65\,536$.

2 MUX 4:1 y tres variables

Se eligen A, B como selección. Para cada valor de (A,B) la función es una función de **solo** C, y solo hay cuatro: 0, 1, C y $\overline{C}$. Basta cablear una de ellas a cada entrada de datos.

3 Generalización

Un MUX 2^k:1 usa k variables como selección. Para cada valor, la función restante depende de **una** variable más y solo puede ser 0, 1, x o $\overline{x}$.

✅ Resultado final

2^{2^n} funciones; MUX 2^k:1 implementa cualquier función de $k+1$ variables (comprobado con las 256 funciones de 3 variables).

Demostración D12

D12 · Número de chips de memoria

✔ verificado

Una memoria de $16\text{K} \times 16$ se construye con chips de $4\text{K} \times 8$. ¿Cuántos chips hacen falta y cómo se conectan?

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: ampliar palabras y ampliar profundidad. **Teoría:** T7.

1 Anchura

Se necesitan 16 bits de ancho con chips de 8: $16/8 = 2$ chips **en paralelo** (comparten dirección, aportan 8 bits cada uno).

2 Profundidad

$16\text{K}/4\text{K} = 4$ bloques. Se unen los 12 bits bajos de dirección a todos los chips y los 2 bits altos entran en un **decodificador 2:4** que activa la selección de un bloque.

3 Total

$4 \cdot 2 = 8$ chips. Equivale a $\dfrac{2^{14} \cdot 16}{2^{12} \cdot 8} = 8$.

✅ Resultado final

8 chips: 4 bloques de profundidad con 2 chips en paralelo cada uno. Dirección: 14 bits (12 a los chips + 2 al decodificador).

REPASO

Autoevaluación

Test por temas con explicación y tarjetas para memorizar definiciones y fórmulas.

Test por temas

Responde sin mirar la teoría. Al elegir una opción se marca la correcta y se muestra la **explicación**: léela aunque hayas acertado. Si fallas dos preguntas de un tema, vuelve a él antes de seguir.

🎯 Tu puntuación

0 correctas de 0 respondidas

Tema 2 · Sistemas de numeración, operaciones y códigos (4 preguntas)

¿Cuál es el rango de un entero en complemento a dos con 6 bits?

- A de -31 a $+32$
- B de -32 a $+32$
- C de 0 a 63
- D de -32 a $+31$

Con $n = 6$: $-2^5 = -32$ hasta $2^5 - 1 = 31$.

En C2 de 8 bits sumas $01100100 + 00110010$. ¿Qué ocurre?

- A Hay un error de paridad
- B Se produce acarreo y el resultado es correcto
- C Hay desbordamiento: dos positivos dan un resultado con MSB igual a 1
- D No hay problema: 150 cabe en 8 bits

$100 + 50 = 150 > 127$. El resultado 10010110 se lee como -106.

$0100\,0111$ en BCD representa:

- **A** 47
- **B** 0x47 en decimal, es decir, 71
- **C** 71
- **D** 4,7

Cada grupo de 4 bits es una cifra decimal: 4 y 7.

Para 3 bits, ¿qué código de Gray sigue a 011?

- **A** 000
- **B** 100
- **C** 010
- **D** 111

La secuencia es 000, 001, 011, 010, 110, 111, 101, 100.

Tema 3 · Álgebra de Boole y funciones lógicas (3 preguntas)

Según De Morgan, $\overline{A \cdot B + C}$ es igual a:

- **A** $\overline{A} + \overline{B} + C$
- **B** $(\overline{A} + \overline{B})\,\overline{C}$
- **C** $(\overline{A} + \overline{B}) + \overline{C}$
- **D** $\overline{A}\,\overline{B} + \overline{C}$

Se cambia + por · y se niega cada término: $\overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B})\overline{C}$.

En un mapa de Karnaugh, ¿cuántas casillas puede tener un grupo válido?

- **A** 1, 2, 4, 8...
- **B** Solo números pares
- **C** Cualquier número
- **D** Hasta un máximo de 4

Los grupos son potencias de 2.

¿Cuál es la forma simplificada de $AB + A\overline{B} + \overline{A}B$?

- **A** $\overline{A} + \overline{B}$
- **B** $A \oplus B$
- **C** $A + B$
- **D** AB

$AB + A\overline{B} = A$, y queda $A + \overline{A}B = A + B$.

Tema 4 · Familias lógicas (3 preguntas)

Una puerta tiene $V_{OH} = 2{,}7$ V y la siguiente exige $V_{IH} = 2{,}0$ V. El margen de ruido en alto es:

- **A** 0,7 V
- **B** 4,7 V
- **C** 0,3 V
- **D** 2,0 V

$NM_H = V_{OH} - V_{IH} = 0{,}7$ V.

Una salida da $I_{OL} = 16$ mA y $I_{OH} = 0{,}8$ mA; cada entrada pide $I_{IL} = 0{,}8$ mA e $I_{IH} = 40$ μA. El fan-out es:

A 40

B 1

C 16

D 20

$16/0{,}8 = 20$ y $0{,}8\ \text{mA}/40\ \mu\text{A} = 20$. El mínimo es 20.

Una puerta CMOS conmuta a 2 MHz con $C = 10$ pF y $V_{DD} = 3$ V. Su potencia dinámica es:

A 0,18 mW

B 1,8 mW

C 0,9 mW

D 0,06 mW

$P = 10 \cdot 10^{-12} \cdot 9 \cdot 2 \cdot 10^{6} = 1{,}8 \cdot 10^{-4}$ W $= 0{,}18$ mW.

Tema 5 · Funciones de la lógica combinacional (3 preguntas)

Para implementar una función de 4 variables con un solo multiplexor, ¿cuál es el mínimo?

A Imposible con un solo MUX

B MUX 4:1

C MUX 16:1 y nada más

D MUX 8:1

Con $2^{n-1} = 8$ entradas de datos y 3 variables en selección; la cuarta va a los datos.

Un decodificador 3:8 tiene la entrada 101. ¿Qué salida se activa?

A S_6

B S_1

C S_5

D S_3

$101_2 = 5$. Se activa la salida S_5.

Un sumador de 8 bits con acarreo en cascada tiene un retardo de 15 ns por etapa. El retardo del acarreo es:

A 120 ns

B 23 ns

C 15 ns

D 8 ns

$8 \cdot 15 = 120$ ns: el acarreo atraviesa las 8 etapas.

Tema 6 · Biestables (flip-flops) y dispositivos relacionados (3 preguntas)

Un biestable JK tiene $Q = 1$ y recibe $J = 0$, $K = 0$. Tras el flanco:

A $Q = 1$ y luego conmuta

B $Q = 1$ (mantiene)

C $Q = 0$

D Estado prohibido

$J = K = 0$ mantiene. Con $Q^+ = J\overline{Q} + \overline{K}Q = 1$.

Quieres pasar de $Q = 1$ a $Q^+ = 0$ con un biestable T. ¿Qué pones en T?

A $T = X$

B $T = 1$

C $T = 0$

D No es posible

Para cambiar de estado, $T = 1$.

Un biestable T con $T = 1$ recibe un reloj de 8 MHz. La salida tiene frecuencia:

A 2 MHz

B 8 MHz

C 4 MHz

D 16 MHz

Conmuta en cada flanco: la frecuencia se divide por 2.

Tema 7 · Memorias y circuitos lógicos programables (3 preguntas)

¿Cuántas líneas de dirección tiene una memoria de 32K×8?

A 15

B 32

C 5

D 8

$32\text{K} = 2^5 \cdot 2^{10} = 2^{15}$.

¿Cuántos chips de 4K×4 hacen falta para una memoria de 8K×8?

A 2

B 16

C 4

D 8

A lo ancho, $8/4 = 2$; a lo largo, $8\text{K}/4\text{K} = 2$. Total $2 \cdot 2 = 4$.

Una ROM de 8 posiciones y 2 bits puede implementar:

A Un sumador completo (3 entradas, 2 salidas)

B Un contador de 8 bits

C Un biestable

D Cualquier función de 4 entradas

Tiene 3 líneas de dirección (entradas) y 2 bits de dato (salidas).

Tema 8 · Contadores (3 preguntas)

¿Cuántos biestables necesita un contador de módulo 100?

A 100

B 10

C 6

D 7

$2^6 = 64 < 100 \leq 2^7 = 128$.

Un contador binario síncrono de 4 bits tiene $Q_3Q_2Q_1Q_0 = 0111$. Tras el siguiente flanco:

A 1111

B 0110

C 1000

D 0000

$T_3 = Q_2Q_1Q_0 = 1, T_2 = Q_1Q_0 = 1, T_1 = Q_0 = 1, T_0 = 1$: todos conmutan.

Un contador asíncrono de 5 bits con $t_{pd} = 10$ ns por etapa: ¿cuánto tarda en estabilizarse como máximo?

A 5 ns

B 32 ns

C 10 ns

D 50 ns

$5 \cdot 10 = 50$ ns.

Tema 9 · Registros de desplazamiento (2 preguntas)

Un contador Johnson de 5 bits tiene:

A 5 estados

B 10 estados

C 32 estados

D 25 estados

$2n = 10.$

Un registro de desplazamiento de 8 bits recibe un dato serie. ¿Cuántos flancos de reloj hacen falta para cargar una palabra completa?

A 3

B 8

C 16

D 1

Entra un bit por flanco.

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos (4 preguntas)

Un autómata tiene 6 estados. ¿Cuántos biestables necesita como mínimo?

A 3

B 6

C 4

D 2

$2^2 = 4 < 6 \leq 2^3 = 8.$

En una máquina de Mealy, la salida:

A Depende solo del estado

B Depende solo de la entrada

C Depende del estado y de la entrada actual

D Es siempre síncrona

Esa es la definición; en Moore depende solo del estado.

Con biestables D, ¿cuál es la entrada D de un biestable que debe pasar de 0 a 1?

A $D = \overline{Q}$

B $D = 0$

C $D = X$

D $D = 1$

En un biestable D, $D = Q^{+}$.

Un pulsador rebota 10 ms. Con reloj de 100 MHz, ¿cuántos ciclos debe esperar el filtro para ser fiable?

A Al menos 10^{6}

B 10^{9}

C 100

D 10^{3}

$10 \cdot 10^{-3}\ \mathrm{s} \cdot 100 \cdot 10^{6}\ \mathrm{Hz} = 10^{6}$ ciclos.

Tema 11 · Descripciones funcionales y estructurales (2 preguntas)

Un archivo VHDL que describe un sumador con la expresión `s <= a + b;` es una descripción:

- **A** Física
- **B** De transistores
- **C** Estructural
- **D** Funcional (de comportamiento)

Dice qué calcula, no con qué puertas.

¿Qué paso del flujo de diseño comprueba que el circuito hace lo que debe, sin tener en cuenta los retardos?

- **A** La simulación funcional
- **B** La síntesis
- **C** La implementación
- **D** La programación de la placa

La simulación funcional ignora los retardos reales; la temporal los incluye.

Tema 12 · Introducción a los dispositivos lógicos programables (2 preguntas)

Una LUT de 5 entradas almacena:

- **A** 16 bits
- **B** 32 bits
- **C** 5 bits
- **D** 25 bits

$2^5 = 32$ posiciones de un bit.

¿Qué hace falta para que un diseño aparezca en los LEDs de la placa?

A Solo simularlo

B Asignar las señales a los pines de la FPGA

C Cambiar el reloj de la placa

D Nada más que sintetizarlo

Sin las restricciones de pines, la herramienta coloca las señales donde quiere.

Tema 13 · Introducción al diseño digital con VHDL (3 preguntas)

En VHDL, `y <= a and b;` describe:

A Una puerta AND siempre presente

B Un proceso secuencial

C Una variable temporal

D Una instrucción que se ejecuta una sola vez

Es una asignación concurrente: hardware que existe de forma permanente.

¿Qué provoca un `if` sin `else` dentro de un proceso combinacional?

A Una puerta NOT

B Un error de sintaxis siempre

C Un latch inferido

D Un contador

La salida mantiene su valor en la rama no cubierta, y eso es memoria.

Un contador módulo 10 en VHDL necesita como mínimo una señal de:

A 5 bits

B 10 bits

C 3 bits

D 4 bits

$\lceil \log_2 10 \rceil = 4$.

Tarjetas para memorizar

Pulsa una tarjeta para darle la vuelta. Intenta contestar **en voz alta** antes de girarla.

¿Rango de un número en C2 de n bits?
-2^{n-1} a $2^{n-1} - 1$.

¿Cómo se niega un número en C2?
Invertir todos los bits y sumar 1.

¿Cuándo hay desbordamiento en C2?
Cuando sumas dos números del mismo signo y el resultado tiene signo contrario.

¿Corrección en la suma BCD?
Si la cifra pasa de 9 (o hay acarreo), sumar 0110.

¿Qué caracteriza al código Gray?
Entre dos códigos consecutivos cambia un solo bit.

¿Enuncia De Morgan?
$\overline{A + B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.

¿Cuántas funciones lógicas distintas hay de n variables?
2^{2^n}.

¿Qué son puertas universales?
NAND y NOR: con una sola se construye cualquier función.

¿Por qué las columnas de Karnaugh van en orden 00,01,11,10?

Para que las casillas vecinas difieran en un solo bit (código Gray).

¿Cómo se usan las casillas indiferentes?

Solo si permiten hacer un grupo mayor; no hay que cubrirlas.

¿Margen de ruido en nivel alto?

$NM_H = V_{OH} - V_{IH}$.

¿Cómo se calcula el fan-out?

El menor de I_{OL}/I_{IL} y I_{OH}/I_{IH}.

¿Para qué sirve el colector abierto?

Permite unir varias salidas en un cable (AND cableada) usando una resistencia pull-up.

¿Qué es el estado Z?

Alta impedancia: la salida queda desconectada. Permite compartir un bus.

¿De qué depende la potencia dinámica de CMOS?

De la capacidad, de la tensión al cuadrado y de la frecuencia.

¿Ecuación del acarreo de un sumador completo?

$C_{out} = AB + C_{in}(A \oplus B)$.

¿Cómo se resta con un sumador?

$A + \overline{B} + 1$: se invierte B y se pone $C_{in} = 1$.

¿Qué número de selección necesita un MUX de 8 a 1?

3 bits ($2^3 = 8$).

¿Qué es cada salida de un decodificador?

Un minitérmino de las entradas.

¿Qué hace un codificador de prioridad?

Entrega el código de la entrada activa de mayor índice cuando hay varias.

¿Ecuación característica del JK?

$Q^+ = J\overline{Q} + \overline{K}Q$.

¿Qué hace un JK con J=K=1?

Conmuta en cada flanco.

¿Entradas del JK para pasar de 1 a 0?

$J = X$ (cualquiera) y $K = 1$.

¿Qué es el tiempo de setup?

Tiempo que la entrada debe estar estable antes del flanco de reloj.

¿Qué diferencia hay entre latch y flip-flop?

El latch es transparente mientras está habilitado; el flip-flop muestrea solo en el flanco.

¿Cuántos bits tiene una memoria de $2^k \times m$?

$m \cdot 2^k$ bits.

¿Cuántas líneas de dirección tiene una memoria de 16K posiciones?

14 ($2^{14} = 16384$).

¿Qué diferencia hay entre ROM y RAM?

La ROM solo se lee y no se borra al apagar; la RAM se lee y escribe y (volátil) se borra.

¿Qué plano es fijo en una PAL?

El plano OR.

¿Cuántos biestables necesita un contador de módulo 60?

6 ($2^5 = 32 < 60 \leq 64$).

¿Ecuación de T_2 en un contador síncrono binario?

$T_2 = Q_1 Q_0$.

¿Ventaja del contador síncrono?

No acumula retardos de propagación y no genera glitches.

¿Qué es un glitch?

Un valor erróneo fugaz mientras los biestables de un contador asíncrono cambian en cadena.

¿Frecuencia de salida de un divisor por N?

f_{clk}/N.

¿Cuántos estados tiene un contador Johnson de n bits?

$2n$.

¿Qué entra en el primer biestable de un contador en anillo?

La salida del último.

¿Qué es SIPO?

Registro serie-entrada, paralelo-salida.

¿Periodo máximo de un LFSR de n bits?

$2^n - 1$.

¿Diferencia entre Moore y Mealy?

En Moore la salida solo depende del estado; en Mealy, del estado y de la entrada.

¿Cuántos biestables para 9 estados?

4 ($2^3 = 8 < 9 \leq 16$).

¿Entradas D para un estado destino Q^+?

$D = Q^+$.

¿Entrada T para pasar de Q a Q^+?

$T = Q \oplus Q^+$.

¿Qué haces con las combinaciones de código sin usar?

Las marcas como indiferentes (X) en Karnaugh.

¿Qué es una descripción estructural?

Dice de qué componentes consta el circuito y cómo se conectan.

¿Qué es una descripción funcional?

Dice qué hace el circuito sin indicar su estructura interna.

¿Qué hace la síntesis?

Convierte la descripción en puertas y biestables.

¿Qué almacena una LUT de 4 entradas?

16 bits: la tabla de verdad de una función de 4 variables.

¿Qué ventaja tiene la FPGA sobre un microcontrolador?

Paralelismo real: cada bloque es hardware que funciona a la vez.

¿Qué reloj trae la placa Basys?

50 MHz.

¿Qué pasa con una FPGA SRAM al apagarse?

Pierde la configuración y hay que volver a cargarla.

¿Qué es una entity?

La interfaz del módulo: nombre y puertos.

¿Qué es una architecture?

La descripción de lo que hay dentro del módulo.

¿Qué infiere VHDL si no asignas una salida en todas las ramas?

Un latch.

¿Cuándo toma efecto una asignación a una signal?

Al terminar el proceso (no al instante).

¿Qué sentencia detecta un flanco de subida?

`rising_edge(clk)`.

CONSULTA

Formulario por temas

Las fórmulas y reglas que hay que saber de memoria, tema a tema. Al final, la versión de una página para imprimir.

Esto es lo que debes **saber sin mirar**. Si una fórmula te suena a chino, vuelve a la teoría del tema: aprender una fórmula sin saber de dónde sale falla en el examen.

Tema 2 · Sistemas de numeración, operaciones y códigos

- N combinaciones $\Rightarrow n = \lceil \log_2 N \rceil$ bits.
- C2 de n bits: $-2^{n-1} \le N \le 2^{n-1} - 1$.
- $-N = \overline{N} + 1$ (invertir y sumar 1).
- Gray: $g = b \oplus (b \gg 1)$.
- Suma BCD: si cifra > 9, sumar 0110.

Tema 3 · Álgebra de Boole y funciones lógicas

- $\overline{A + B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.
- $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$.
- $A \oplus B = A\overline{B} + \overline{A}B$.
- n variables: 2^n filas y 2^{2^n} funciones distintas (con 4 variables, 65536).

Tema 4 · Familias lógicas

- $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
- fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$.
- $P_{din} = C\,V_{DD}^2\,f$.
- Retardo de una ruta: suma de los t_{pd} de sus puertas.

Tema 5 · Funciones de la lógica combinacional

- $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$.
- $A - B = A + \overline{B} + 1$.
- MUX $2^k : 1$ implementa cualquier función de $k + 1$ variables.
- Salida i de un decodificador $= m_i$ (minitérmino).

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

- $Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$.

- $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
- Divisor por 2: $f_{out} = f_{clk}/2$.

Tema 7 · Memorias y circuitos lógicos programables

- bits totales $= 2^k \cdot m$; $k = \log_2 N$.
- Chips necesarios $= \dfrac{N_{total}}{N_{chip}} \cdot \dfrac{m_{total}}{m_{chip}}$.
- Una ROM $2^n \times m$ implementa m funciones de n variables.

Tema 8 · Contadores

- $n = \lceil \log_2 N \rceil$; estados no usados: $2^n - N$.
- $f_{out} = f_{clk}/N$.
- Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$.
- Asíncrono: $t_{total} = n \cdot t_{pd}$.

Tema 9 · Registros de desplazamiento

- Anillo: n estados; Johnson: $2n$; LFSR máximo: $2^n - 1$.
- Serie → paralelo de n bits: n ciclos de reloj.

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

- $n = \lceil \log_2(\#\text{estados}) \rceil$.
- Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$.
- D: $D = Q^+$. T: $T = Q \oplus Q^+$. JK: usar tabla de excitación.

Tema 13 · Introducción al diseño digital con VHDL

- Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos.
- Bits de un contador: $\lceil \log_2 N \rceil$.
- Una `case` completa en un proceso combinatorio: añade siempre `when others`.

T5 · Bloques combinacionales (tabla de decisión)

Quiero...	Uso...
Sumar dos números de n bits	n sumadores completos en cascada (acarreo de uno al siguiente)
Restar $A - B$	Sumador con B invertida y $C_{in} = 1$

Quiero...	Uso...
Comparar	Comparador: salidas $A > B$, $A = B$, $A < B$
Elegir una entre 2^k	Multiplexor con k bits de selección
Activar una salida según un código	Decodificador n a 2^n
Mostrar un dígito	Decodificador BCD a 7 segmentos
Dividir/multiplicar por 2^k	Desplazar k posiciones (sin puertas)

T11 y T12 · Descripciones y lógica programable

- Un mismo circuito se describe con **ecuaciones**, **tabla de verdad**, **diagrama de estados**, **bloques** o **código** (VHDL). Los distintos niveles de abstracción son comportamiento, flujo de datos y estructura.
- LUT de k entradas: memoria de 2^k bits; implementa cualquier función de k variables.
- Una función de más de k variables se reparte entre varias LUT.

La versión de **una sola página**, para imprimir, está justo debajo y en el PDF `formulario.pdf`.
Comprueba en las normas de tu convocatoria si puedes llevar un formulario al examen.

CONSULTA

Formulario de una página

Todo lo esencial en una hoja: numeración, Boole, bloques, biestables, contadores, autómatas y VHDL.

Tema	Lo imprescindible
T2 Numeración	$n = \lceil \log_2 N \rceil$ bits. C2 de n bits: $-2^{n-1} \ldots 2^{n-1} - 1$; $-N = \overline{N} + 1$. Desbordamiento: acarreo que entra al signo $\neq$ acarreo que sale. Gray: $g = b \oplus (b \gg 1)$. BCD: cifra > 9 suma 0110.
T3 Boole	$\overline{A+B} = \overline{A}\,\overline{B}$; $\overline{AB} = \overline{A} + \overline{B}$. $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$. $A \oplus B = A\overline{B} + \overline{A}B$. Karnaugh: grupos de 2^k unos adyacentes (también en los bordes); las X solo se usan si ayudan. NAND-NAND = suma de productos.
T4 Familias	$NM_H = V_{OH} - V_{IH}$; $NM_L = V_{IL} - V_{OL}$. Fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$. $P_{din} = CV_{DD}^2 f$.
T5 Combinacional	Sumador completo: $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$. $A - B = A + \overline{B} + 1$. MUX 2^k:1 implementa funciones de $k+1$ variables. Decodificador: salida $i = m_i$.
T6 Biestables	$Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$. Excitación JK: $0 \to 0$: $J = 0, K = X$; $0 \to 1$: $J = 1, K = X$; $1 \to 0$: $J = X, K = 1$; $1 \to 1$: $J = X, K = 0$. $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
T7 Memorias	bits $= 2^k \cdot m$. Chips $= \frac{N}{N_c} \cdot \frac{m}{m_c}$. ROM $2^n \times m$ = m funciones de n variables.
T8 Contadores	$n = \lceil \log_2 N \rceil$; $f_{out} = f_{clk}/N$. Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$. Asíncrono: $t = n\,t_{pd}$. Estados no usados: $2^n - N$ (¡cuidado con el bloqueo!).
T9 Registros	Anillo: n estados. Johnson: $2n$. LFSR máximo: $2^n - 1$. Serie a paralelo de n bits: n ciclos.
T10 Autómatas	Estados $\to n = \lceil \log_2(\#) \rceil$ biestables. Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$. D: $D = Q^+$. T: $T = Q \oplus Q^+$. Detector con solapamiento: el estado es el mayor prefijo que es sufijo.

Tema	Lo imprescindible
T12–T13 FPGA y VHDL	LUT de k entradas: 2^k bits. Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos. `case` completo: `when others`. Lo concurrente (fuera de `process`) ocurre a la vez.

CONSULTA

Glosario

Los términos de la asignatura, en orden alfabético y con una definición corta.

Úsalo cuando un enunciado de examen use una palabra que no recuerdes. Las definiciones son cortas a propósito: para entender el concepto, ve al tema indicado en el índice.

Acarreo anticipado

Técnica que calcula los acarreos en paralelo para acelerar un sumador.

Alta impedancia (Z)

Estado de una salida tri-state en el que está eléctricamente desconectada.

Architecture

Parte del código VHDL que define el funcionamiento interno.

Autómata síncrono

Circuito secuencial cuyos biestables cambian todos con el mismo reloj.

BCD

Decimal codificado en binario: cada cifra decimal en 4 bits.

Biestable

Circuito de dos estados estables que almacena un bit.

Carga paralela

Entrada que fuerza al contador a un valor dado.

CLB

Bloque lógico configurable de una FPGA (LUT y biestables).

Codificación de estados

Asignación de un código binario a cada estado.

Codificador de prioridad

Da el código de la línea activa de mayor prioridad.

Colector abierto

Salida que solo fuerza el 0; el 1 lo pone una resistencia externa.

Complemento a dos

Representación de enteros con signo: el MSB pesa -2^{n-1}.

Contador asíncrono

Contador en el que cada biestable usa como reloj la salida del anterior.

Contador en anillo

Registro de desplazamiento realimentado con un único 1 que circula.

Contador Johnson

Registro de desplazamiento que realimenta la salida negada; $2n$ estados.

Contador módulo N

Contador que recorre N estados y vuelve al inicial.

Contador síncrono

Contador en el que todos los biestables reciben el mismo reloj.

Código Gray

Código en el que dos palabras consecutivas difieren en un solo bit.

Decodificador

Activa una única salida según el código binario de entrada.

Desbordamiento

El resultado de una operación no cabe en el número de bits disponible.

Descripción estructural

Descripción por componentes e interconexiones.

Descripción funcional

Descripción por comportamiento.

Diagrama de estados

Grafo con un nodo por estado y una flecha por transición.

Distancia de Hamming

Número de bits en que difieren dos palabras de un código.

EDA

Herramientas de diseño electrónico asistido por ordenador.

Entity

Parte del código VHDL que define las entradas y salidas de un módulo.

Estado

Resumen de la historia del circuito que basta para decidir qué hará.

Fan-out

Número máximo de entradas que una salida puede alimentar.

Fin de cuenta

Salida que avisa de que el contador llega a su último estado, para encadenar contadores.

Flanco

Instante en que la señal de reloj pasa de 0 a 1 (subida) o de 1 a 0 (bajada).

FPGA

Dispositivo lógico reprogramable formado por bloques lógicos y una red de interconexión.

Implicante primo

Grupo de unos que no se puede ampliar más en el mapa de Karnaugh.

Indiferente (X)

Combinación de entradas que no ocurre o cuya salida da igual.

Latch inferido

Memoria no deseada que aparece cuando una señal no se asigna en todos los casos.

LFSR

Registro de desplazamiento con realimentación lineal (XOR) que genera secuencias pseudoaleatorias.

LUT

Tabla de consulta: memoria que almacena la tabla de verdad de una función de pocas variables.

Margen de ruido

Cuánto ruido tolera una conexión sin que se interprete mal el nivel lógico.

Maxitérmino

OR con todas las variables que vale 0 en una sola fila de la tabla.

Metaestabilidad

Estado intermedio e impredecible tras violar los tiempos de setup o hold.

Minitérmino

AND con todas las variables que vale 1 en una sola fila de la tabla.

MSI

Integración de media escala: bloques como sumadores, multiplexores o decodificadores.

Multiplexor

Selector que copia a la salida una de varias entradas según un código de selección.

PAL

Matriz con plano AND programable y plano OR fijo.

Paridad

Bit extra que hace par (o impar) el número de unos de una palabra.

PLA

Matriz lógica programable con plano AND y plano OR programables.

Proceso

Bloque `process` de VHDL que describe comportamiento.

Puerta universal

Puerta con la que se puede construir cualquier función lógica (NAND o NOR).

RAM

Memoria de lectura y escritura, volátil.

Rebote

Oscilaciones breves de un contacto mecánico al abrir o cerrar.

Registro de desplazamiento

Cadena de biestables en la que cada uno toma el valor del anterior en cada flanco.

Restricciones (UCF)

Fichero que asocia las señales del diseño con los pines físicos de la FPGA.

Retardo de propagación

Tiempo que tarda la salida en reaccionar a un cambio de la entrada.

ROM

Memoria de solo lectura: guarda una tabla fija.

Setup / hold

Tiempos de preparación y mantenimiento de las entradas alrededor del flanco.

Síntesis

Transformación de una descripción de alto nivel en una red de puertas y biestables.

Síntesis

Conversión de la descripción en puertas y biestables.

Tabla de excitación

Dice qué entradas del biestable hacen falta para ir de Q a Q^+.

VHDL

Lenguaje de descripción de hardware.

Volátil

Que pierde su contenido al quitar la alimentación.

CONSULTA

Plan de estudio

Un plan de 4 semanas y uno de 7 días, con qué hacer cada sesión y cómo saber si vas bien.

Ningún plan sustituye a hacer problemas con lápiz. La regla de oro: **cada hora de teoría va seguida de media hora de ejercicios** de ese tema.

> 💡 **Antes de empezar**
>
> No hay fechas aquí a propósito: usa la de tu convocatoria. Cuenta hacia atrás desde ella y reparte las semanas. Marca lo hecho en la lista de progreso de El examen y cómo estudiar.

Plan de 4 semanas (1–2 horas al día)

Semana	Temas y bloques	Qué haces	Cómo sabes que vas bien		
1 · Base	[[t02	T2]] numeración y [[t03	T3]] Boole y Karnaugh	Teoría + tests de autoeval + hoja de Karnaugh K1–K5	Haces un mapa de 4 variables con indiferentes sin dudar
2 · Combinacional	[[t04	T4]] familias y [[t05	T5]] bloques	Sumador, comparador, MUX y displays; hoja K6–K17; primer examen resuelto (examenes)	Diseñas el recinto o la media de cuatro números sin mirar
3 · Secuencial	[[t06	T6]]–[[t09	T9]] biestables, memorias, contadores y registros	Tablas de excitación; contadores con JK; simulacro 1	Tienes la tabla de excitación del JK de memoria

Semana	Temas y bloques	Qué haces	Cómo sabes que vas bien			
4 · Autómatas y VHDL	[[t10	T10]], [[t11	T11]], [[t12	T12]], [[t13	T13]] y las prácticas	Hoja de autóma detecto con solapar revisar simulac

Plan exprés de 7 días

Día	Qué haces		
1	[[t02	T2]] y [[t03	T3]]: numeración, C2, BCD, Boole y Karnaugh. 10 mapas.
2	[[t05	T5]]: sumador, comparador, MUX, decodificador y display. Examen de diciembre completo (examenes).	
3	[[t06	T6]] y [[t08	T8]]: biestables y contadores; tabla de excitación JK. Hoja de

Día	Qué haces						
			autómatas A1, A2.				
4	[[t10	T10]]: autómatas. Detectores A3, A5 y A7; sumador serie A9.					
5	[[t07	T7]], [[t09	T9]], [[t12	T12]] y [[t13	T13]]: memorias, registros, FPGA y VHDL. Repasa las prácticas ([[p1	Práctica 1]] a [[p567	Prácticas 5–7]]).
6	**Simulacro 1** en 2 horas (simulacros) y corrección. Repite los fallos.						
7	**Simulacro 2**, formulario de una página y glosario. Descansa pronto.						

Cómo repartir el tiempo según los exámenes

El recuento de problemas por tipo está en El examen y cómo estudiar. Con solo tres exámenes no es una regla, pero orienta: **bloques MSI**, **aritmética y códigos** y **autómatas** aparecen en los tres; los contadores con biestables, en uno. Dedica la mitad del tiempo a esos tres grupos.

El día del examen

- Lee **todos** los enunciados y empieza por el que más seguro estés.
- En cada problema, **dibuja lo que te piden** (esquema, diagrama de estados, tabla) y explica en una frase por qué.
- Un cronograma sin explicación no cuenta: escribe qué ocurre en cada flanco.
- Guarda **10 minutos** para revisar: ¿cabe el resultado en los bits que has elegido? ¿Has cubierto el solapamiento? ¿Has puesto reset?

Guía no oficial de Electrónica Digital (UPCT) · Última actualización: 8 de noviembre de 2026. Los resultados numéricos y las ecuaciones se han comprobado con código.